
FraudGuard

*An Explainable Ensemble Learning System
for Credit Card Fraud Detection*

Final Project – Group 510

Deep and Machine Learning

Submitted by:

Chen Kadouri (ID: redacted in Github version.)

Noam Galili (ID: redacted in Github version.)

Shahar Solima (ID: redacted in Github version.)

Angel Sasin (ID: redacted in Github version.)

Asher Pe'er (ID: redacted in Github version.)

Supervisor:

Dr. Raz Lin

college of management

August 2026

Contents

1	Introduction and Related Work	1
1.1	Background and Motivation	1
1.2	Relevant Academic Literature	1
1.3	Similar Industrial Systems	2
1.4	Project Objectives and Contribution	3
2	Data	4
2.1	Dataset Research and Selection	4
2.2	Credit Card Transactions Fraud Detection Dataset	4
2.2.1	Dataset Structure	4
2.2.2	Class Distribution	5
2.3	Data Preparation and Feature Engineering	6
2.3.1	Data Cleaning and Feature Selection	6
2.3.2	Engineered Features	6
2.3.3	Categorical Encoding and Missing Values	7
2.3.4	Final Model Feature Representation	7
3	Preliminary Experimental Exploration	9
3.1	Experimental Motivation and Evaluation Strategy	9
3.2	Preliminary Model Exploration	9
3.2.1	Deep Neural Network with Entity Embeddings	9
3.2.2	Autoencoder-Based Anomaly Detection	10
3.2.3	GAN-Based Fraud Generation	10
3.2.4	Early XGBoost Classification	11
3.3	Preliminary Experimental Findings	11
4	Predictive Model Development and Ensemble Architecture	12
4.1	Systematic Model Development Methodology	12
4.2	Individual Candidate Model Evaluation	13
4.2.1	Random Forest	13
4.2.2	LightGBM	16
4.2.3	XGBoost	18

4.2.4	CatBoost	21
4.2.5	Neural Network	24
4.2.6	Isolation Forest	28
4.3	Comparative Evaluation of Candidate Models	29
4.3.1	Comparative Predictive Performance	30
4.3.2	Precision-Recall Analysis	31
4.3.3	ROC Analysis	32
4.3.4	Computational Performance	33
4.3.5	Transition to Ensemble Learning	34
4.4	Out-of-Fold Stacking Ensemble	35
4.4.1	Out-of-Fold Prediction Generation	36
4.4.2	Construction of the OOF Meta-Feature Dataset	37
4.4.3	Logistic Regression Meta-Model	38
4.4.4	OOF Evaluation of the Stacking Layer	39
4.4.5	Final Base-Model Retraining and Independent Test Inference	40
4.4.6	Five-Model Stacking Performance	42
4.4.7	OOF-Based Decision-Threshold Optimization	43
4.4.8	Final Evaluation of the Five-Model Stacking Ensemble	44
4.5	Meta-Feature Combination Study and Ensemble Reduction	47
4.5.1	Business-Oriented Threshold Selection	47
4.5.2	Precision-Recall Characteristics of Candidate Combinations	49
4.5.3	Ensemble Complexity Analysis	49
4.5.4	Selection of the Three-Model Ensemble	50
4.5.5	Analysis of the Selected Three-Model Ensemble	52
4.5.6	Comparison with the Original Five-Model Stack	56
4.5.7	Final Model-Reduction Decision	56
4.6	Final Production Model Selection and Inference Validation	57
4.7	Explainability Layer	58
4.7.1	SHAP-Based Feature Attribution	59
4.7.2	Distribution of SHAP Contributions	61
4.7.3	Transaction-Level Explanation	62
4.7.4	Explainability Output and System Integration	63
4.8	Chapter Summary	64

5	FraudGuard System Implementation	65
5.1	System Architecture	66
5.2	End-to-End Transaction Processing Flow	66
5.3	Web Monitoring Environment	68
5.3.1	Command Center	68
5.3.2	Overview	69
5.3.3	Analytics	70
5.3.4	Geo Risk	71
5.3.5	Services	72
5.3.6	Live Detection	72
5.4	Continuous Transaction Simulation	74
5.5	Fraud Alerts and Telegram Integration	74
5.6	Chapter Summary	75
6	Business Plan and “Shark Tank” Competition	76
6.1	“Shark Tank” Academic Competition	76
6.2	Market Opportunity and Business Model	76
6.3	Target Customers and Go-to-Market Strategy	76
6.4	Competitive Advantage	77
7	Discussion, Conclusions, Limitations and Future Work	77
7.1	Discussion	77
7.2	Conclusions	78
7.3	Limitations	78
7.4	Future Work	78
	Project Repository and Supplementary Materials	79

1 Introduction and Related Work

1.1 Background and Motivation

Credit card fraud represents a significant challenge within modern electronic payment systems, where fraud detection methods must distinguish fraudulent activity from large volumes of legitimate customer transactions. This task is particularly challenging because fraud datasets are typically highly imbalanced, with fraudulent transactions representing only a small fraction of overall activity. Consequently, high overall classification accuracy does not necessarily indicate effective fraud detection, making evaluation measures that consider the minority fraud class particularly important [1].

The financial impact of payment-card fraud further demonstrates the relevance of this problem. According to The Nilson Report, worldwide payment-card fraud losses reached approximately \$33.41 billion in 2024 and are projected to reach \$41.06 billion annually by 2030 [2]. Effective fraud detection must therefore identify fraudulent transactions while also limiting false-positive classifications of legitimate activity.

Machine Learning and Deep Learning provide data-driven approaches for learning patterns from transaction data rather than relying exclusively on manually defined rules. However, different predictive models may capture different patterns and demonstrate different strengths when applied to the same fraud-detection problem. This motivated FraudGuard to investigate not only the performance of individual models, but also whether combining their prediction signals through ensemble learning could provide a more effective fraud-detection approach.

FraudGuard was developed around two central challenges. First, achieving strong detection of the minority fraud class while controlling false-positive classifications. Second, determining whether combining multiple predictive models could improve performance compared with relying on individual models. These challenges guided the project from individual model experimentation toward ensemble learning, model-combination analysis, and the selection of a final fraud-detection architecture.

1.2 Relevant Academic Literature

Credit card fraud detection has been widely studied as a challenging Machine Learning problem due to the highly imbalanced nature of financial transaction data. Previous research has shown that conventional evaluation based primarily on overall accuracy can be misleading when the positive class is rare, since strong performance on the majority class may conceal poor detection of fraudulent transactions. Precision-Recall analysis is therefore particularly relevant for evaluating classifiers under substantial class imbalance [1].

Research on financial fraud detection has explored a range of Machine Learning approaches, including tree-based classifiers and ensemble methods. Ensemble learning is particularly relevant because it combines information from multiple predictive models rather than relying on the behavior of a single classifier. Stacked generalization extends this idea by using the predictions of base models as inputs to a higher-level model that learns how those prediction signals should be combined [3].

Another important consideration in ensemble development is the generation of predictions used to train the meta-model. Out-of-Fold predictions allow meta-features to be generated for training samples using models that were not trained on those same samples, providing a suitable methodology for developing the Stacking architecture while reducing the risk of data leakage between the base-model and meta-model training stages [4].

In addition to predictive performance, interpretability is relevant when applying Machine Learning to fraud detection. SHAP provides a framework for attributing model predictions to individual input features [5]. This provides an academic foundation for incorporating an explainability component alongside the predictive architecture, allowing individual fraud predictions to be examined in terms of the features that influenced the model output.

Together, these areas of research provided the academic foundation for the technical approach investigated in FraudGuard.

1.3 Similar Industrial Systems

Several commercial platforms apply Machine Learning, behavioral analytics, and real-time decisioning to financial fraud detection and prevention:

- **Feedzai** provides an AI-powered transaction fraud platform for financial institutions. Its system analyzes transactional and behavioral information across payment channels, builds customer risk profiles, and supports real-time fraud detection and decisioning. Feedzai also emphasizes reducing false positives and providing explainable fraud decisions [6].
- **Featurespace** provides fraud-detection technology based on behavioral analytics and Machine Learning. Its platform models individual customer behavior in real time and is designed to identify deviations associated with card and payment fraud while reducing unnecessary customer intervention and false-positive classifications [7].
- **SAS Fraud Management** provides an enterprise fraud-management platform using analytics and Machine Learning to monitor payments and other transactions for suspicious behavior. SAS also provides fraud decisioning capabilities designed for fraud detection, prevention, investigation, and enterprise-scale financial-crime management [8].

These systems demonstrate that Machine Learning, real-time transaction analysis, behavioral modeling, explainability, and reduction of false positives are important components of modern industrial fraud-detection platforms. However, the descriptions above are based primarily on publicly available information provided by the companies themselves and should not be interpreted as an independent benchmark comparison between these commercial systems and FraudGuard.

1.4 Project Objectives and Contribution

The development of FraudGuard followed an earlier project direction, PersistenceAV, which focused on applying Deep Learning to detect malware persistence through operating-system behavioral data. During development, the team encountered limitations in reliably determining whether analyzed malware samples had successfully established persistence and in obtaining suitable enterprise cybersecurity data for the planned system. Since reliable labels and appropriate data were necessary for model development and evaluation, the project was redirected toward credit card fraud detection.

The primary objective of FraudGuard was to develop and systematically evaluate an end-to-end fraud-detection approach capable of maintaining strong detection of fraudulent transactions while reducing false-positive classifications of legitimate activity. The project investigated multiple Machine Learning and Deep Learning models and examined whether combining their predictive information could improve performance compared with relying on individual models.

The project contributes a structured experimental framework encompassing individual-model evaluation, leakage-aware Out-of-Fold Stacking, model-combination analysis, and decision-threshold selection.

This process resulted in a final Stacking architecture combining Random Forest, CatBoost, and XGBoost through a Logistic Regression meta-model. The selected predictive architecture was further extended with SHAP-based explainability and integrated into a functional system capable of processing transactions and presenting fraud predictions in real time, maintaining transaction history, and generating Telegram notifications for transactions classified as fraudulent.

Unlike the commercial platforms discussed in Section 1.3, FraudGuard was developed as an academic end-to-end fraud-detection framework rather than a complete enterprise fraud-management platform. Its contribution lies in the systematic development and evaluation of the predictive architecture, its extension with explainability, and its integration into a functional software prototype, providing a complete experimental path from data preparation and model development to real-time fraud-detection demonstration.

2 Data

2.1 Dataset Research and Selection

Before model development, multiple publicly available credit card fraud datasets were examined to identify a dataset suitable for the objectives of FraudGuard. The candidate datasets differed in size, structure, feature representation, and the amount of information provided about individual transaction attributes.

A central consideration during this process was feature interpretability. Several of the examined fraud datasets protect sensitive financial information by anonymizing or transforming transaction attributes. Although such datasets can still be used for predictive modeling, the resulting variables may not provide a clear indication of what each value represents. This presented a limitation for FraudGuard, since the project aimed not only to classify transactions but also to analyze meaningful transaction characteristics and later provide interpretable explanations for individual predictions.

The dataset selected for the project was the Credit Card Transactions Fraud Detection dataset available on Kaggle [9]. The dataset contains simulated credit card transactions and, unlike several of the examined datasets with anonymized or transformed features, provides descriptive attributes related to the transaction, merchant, customer, time, and geographical location, together with an explicit binary fraud label. This combination made the dataset suitable for FraudGuard because the meaning of individual features could be understood directly, supporting both supervised model development and the project’s later feature-engineering and explainability stages.

For these reasons, the dataset was selected as the primary data source used throughout FraudGuard. The following sections describe its structure, class distribution, and the separation of training and test data.

2.2 Credit Card Transactions Fraud Detection Dataset

2.2.1 Dataset Structure

The selected dataset is provided through two CSV files, `fraudTrain.csv` and `fraudTest.csv`. Each raw transaction contains 23 columns representing transaction, merchant, customer, temporal, geographical, identification, and fraud-label information [9].

The raw records are not passed directly to the predictive models. Before model training or inference, each transaction passes through the project’s preprocessing and feature-engineering pipeline. During this process, unnecessary fields are removed, relevant information is transformed, additional features are derived from the original transaction data, and missing values are handled according to the preprocessing logic. This process transforms each raw 23-column transaction record into the final 16-feature representation used by the predictive models. The exact preparation and feature-engineering steps are described later in this chapter.

Property	Transactions	Raw columns	Target	Final model features
Training Data	1,296,675	23	is_fraud	16
Test Data	555,719	23	is_fraud	16

Table 1: Structure of the training and test datasets before and after the FraudGuard data-preparation pipeline.

2.2.2 Class Distribution

The selected dataset exhibits a substantial imbalance between legitimate and fraudulent transactions. In the training data, only 7,506 of 1,296,675 transactions (0.5789%) are labeled as fraudulent. The separate test data contains 2,145 fraudulent transactions among 555,719 total transactions (0.3860%) [9]. Therefore, fraud transactions constitute less than 1% in both datasets, demonstrating a substantial class imbalance. Therefore, fraud transactions constitute less than 1% in both datasets, demonstrating a substantial class imbalance.

This distribution is an important characteristic of the data used throughout FraudGuard and is taken into account during the subsequent model-development and evaluation stages.

The two files serve different roles throughout the project. `fraudTrain.csv` is used during model development, while `fraudTest.csv` is maintained as a separate dataset for evaluating the developed models on transactions not used during training. The specific training procedures applied during model development are described in the methodology chapter.

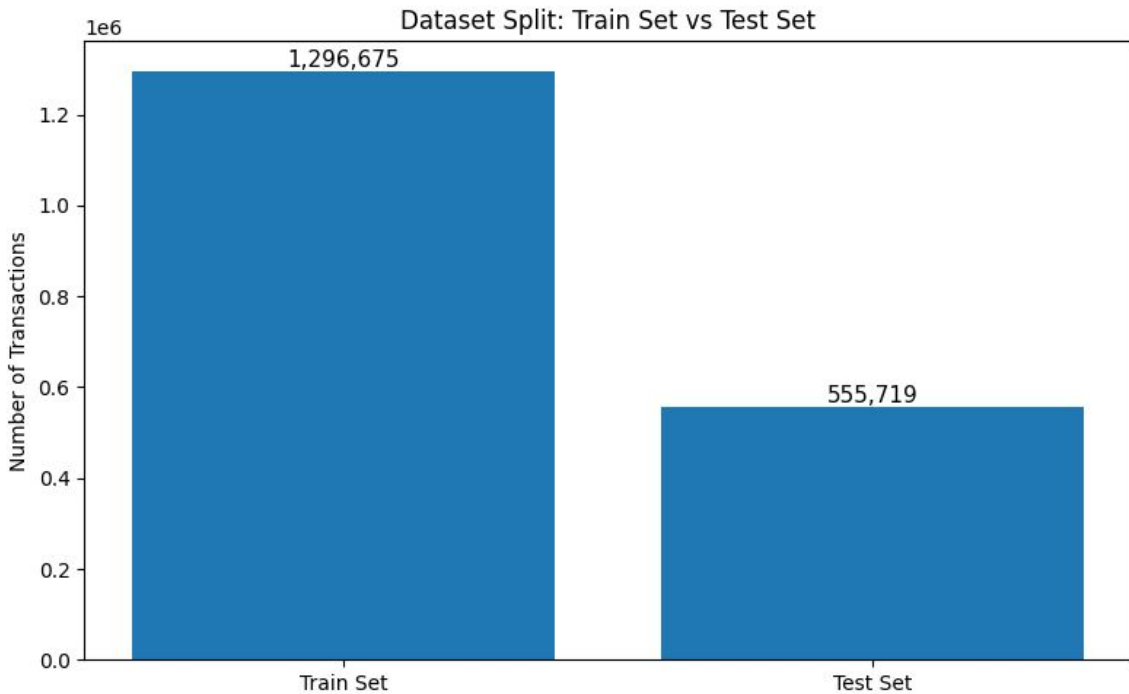


Figure 1: Distribution of legitimate and fraudulent transactions in the FraudGuard training and test datasets.

2.3 Data Preparation and Feature Engineering

2.3.1 Data Cleaning and Feature Selection

Before model development, the raw transaction records are prepared by removing fields that are not retained in the final model input. The index-like column Unnamed: 0 is removed when present, and the target variable is_fraud is separated from the predictive features.

Several original attributes are also excluded from the final feature set. The fields trans_date_trans_time and dob are first used to derive new information before being removed, while cc_num, first, last, street, city, state, zip, trans_num, and unix_time are removed from the model input.

Raw Field(s)	Treatment
Unnamed: 0	Removed
trans_date_trans_time	Used for feature engineering, then removed
dob	Used for feature engineering, then removed
cc_num, first, last, street, city, state, zip, trans_num, unix_time	Removed from model input
is_fraud	Separated as the target variable

Table 2: Treatment of raw fields during data preparation.

2.3.2 Engineered Features

The FraudGuard preprocessing pipeline derives six additional features from information available in the original transaction record. These features capture temporal, customer, and geographical characteristics that are not represented directly in the final form required by the models.

Engineered Feature	Derived From	Description
hour	trans_date_trans_time	Hour in which the transaction occurred
day_of_week	trans_date_trans_time	Day of the week of the transaction
month	trans_date_trans_time	Month in which the transaction occurred
is_weekend	day_of_week	Indicates whether the transaction occurred on a weekend
age	dob and transaction date	Cardholder's age at the time of the transaction
distance_km	lat, long, merch_lat, merch_long	Geographic distance between the cardholder and merchant locations

Table 3: Treatment of raw fields during data preparation.

The temporal features are extracted from `trans_date_trans_time`, while age is calculated using the cardholder’s date of birth relative to the transaction date. The `distance_km` feature is calculated from the customer and merchant geographical coordinates using the **Haversine distance** [10].

$$a = \sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)$$

$$c = 2 \arctan 2 \left(\sqrt{a}, \sqrt{1-a} \right)$$

$$distance_km = 6371 \cdot c$$

Figure 2: Haversine equation

2.3.3 Categorical Encoding and Missing Values

Following feature engineering, the remaining attributes are separated into categorical and numerical features. Categorical variables are transformed using Scikit-learn’s `OrdinalEncoder` [11], allowing them to be processed by the predictive models. Categories that were not observed when fitting the encoder are represented using the value -1.

Missing numerical values are handled using **median values calculated from the training data**. These values are subsequently applied when preparing other data, ensuring that missing-value handling is based on information obtained during training rather than from unseen data.

Together, these preprocessing steps transform the cleaned transaction attributes into a numerical representation suitable for model input.

2.3.4 Final Model Feature Representation

Following data cleaning, feature engineering, and preprocessing, each transaction is represented by **16 features** used as input to the predictive models. The final feature set contains a combination of original transaction attributes and features derived during the preparation stage.

Feature	Type
merchant	Original
category	Original
amt	Original
gender	Original
lat	Original
long	Original
city_pop	Original
job	Original
merch_lat	Original
merch_long	Original
hour	Engineered
day_of_week	Engineered
month	Engineered
is_weekend	Engineered
age	Engineered
distance_km	Engineered

Table 4: Final 16-feature representation used by the FraudGuard predictive models.

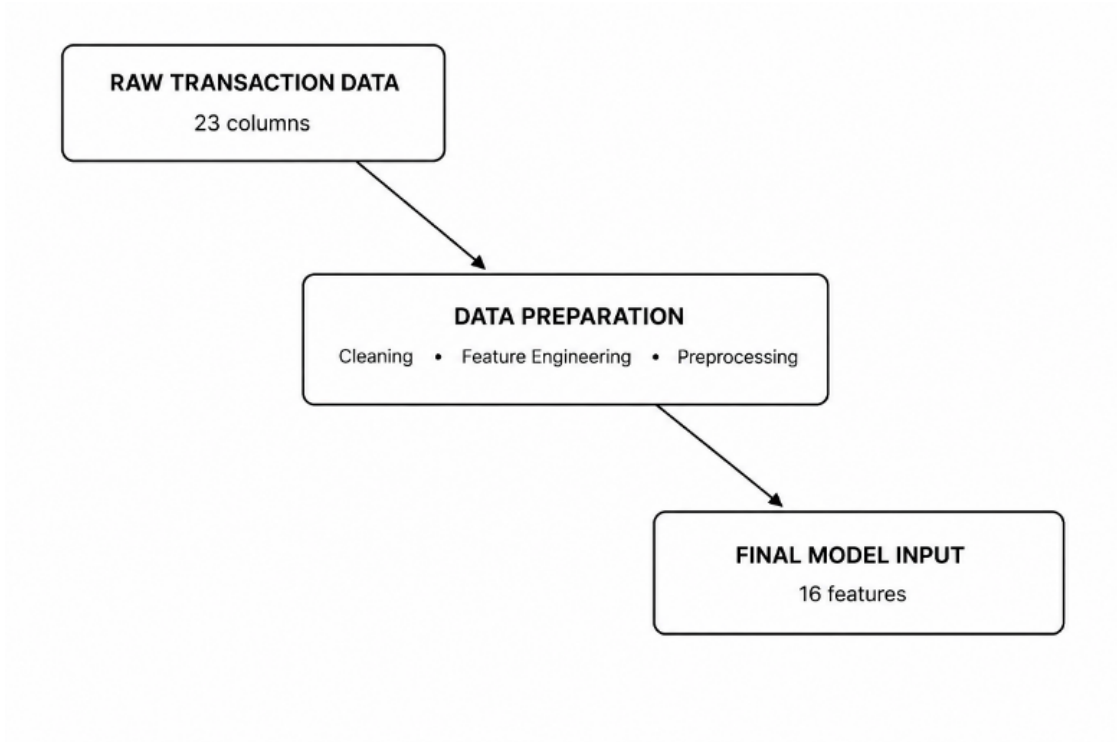


Figure 3: Transformation of the 23-column raw transaction record into the final 16-feature model representation.

3 Preliminary Experimental Exploration

3.1 Experimental Motivation and Evaluation Strategy

The development of FraudGuard followed an iterative experimental methodology in which the evaluation strategy and predictive architecture were progressively refined according to empirical results. The process began with the independent development and evaluation of several candidate Machine Learning and Deep Learning approaches on the selected credit-card transaction dataset. These initial experiments included conventional supervised classification, deep neural architectures, unsupervised anomaly detection, and generative approaches.

During the initial experiments, **Accuracy** was considered as a primary performance measure. However, the severe class imbalance of the dataset revealed that overall Accuracy provided an inadequate representation of fraud-detection capability. Since legitimate transactions constitute more than 99% of the observations, a classifier could achieve very high Accuracy while providing limited detection of the minority fraud class [?]. The evaluation methodology was therefore expanded to include **Balanced Accuracy**, which accounts for predictive performance across both classes [12].

Further experimentation demonstrated that Balanced Accuracy alone was also insufficient for selecting a practically suitable fraud-detection model. In particular, some configurations achieved comparatively strong detection performance while generating a substantial number of **False Positives**, incorrectly classifying legitimate transactions as fraudulent. Consequently, the evaluation strategy evolved toward direct analysis of the **Confusion Matrix**, with particular emphasis on **Recall** and **Precision**. Recall was used to quantify the proportion of actual fraudulent transactions successfully detected, while Precision measured the proportion of transactions classified as fraudulent that were genuinely fraudulent [?]. This enabled the experiments to explicitly examine the trade-off between detecting fraud and limiting false alarms.

3.2 Preliminary Model Exploration

Prior to the systematic ensemble-development stage, four distinct modeling approaches were investigated to evaluate their suitability for the FraudGuard classification task. These experiments encompassed supervised gradient boosting, Deep Learning, unsupervised anomaly detection, and generative modeling. Their empirical performance provided the basis for refining the subsequent model-development strategy.

3.2.1 Deep Neural Network with Entity Embeddings

The Deep Neural Network experiment investigated a multi-input architecture designed for heterogeneous tabular transaction data. High-cardinality categorical variables were represented through learned **Entity Embeddings**, while continuous numerical attributes were normalized separately [13]. The resulting representations were processed by a Fully Connected Neural Network consisting of three hidden layers with **256, 128, and 64 units**, incorporating ReLU activation, Batch Normalization, L2 regularization, and Dropout.

The network was optimized using Adam with Binary Cross-Entropy loss.

On the separate test dataset, the model achieved **97.25% Accuracy**, **91.70% Balanced Accuracy**, **0.9788 ROC-AUC**, and **0.6482 PR-AUC**. For the fraud class, however, Precision was only **10.96%**, while Recall reached **86.11%**. The Confusion Matrix contained **1,847 True Positives**, **298 False Negatives**, and **14,999 False Positives**.

The experiment therefore demonstrated an important limitation of interpreting high aggregate performance as evidence of effective fraud detection. Although the network detected a substantial proportion of fraudulent transactions, the low Precision and high False Positive count indicated that many legitimate transactions were incorrectly classified as fraudulent.

3.2.2 Autoencoder-Based Anomaly Detection

The Autoencoder experiment investigated fraud detection from an **unsupervised anomaly-detection perspective**. Rather than learning a direct mapping between transaction features and the fraud label, the Autoencoder learned to reconstruct transaction patterns, with reconstruction error subsequently used as an anomaly score [14].

Following reconstruction-error threshold optimization, the selected threshold was approximately **0.5547**. At this operating point, the model achieved approximately **60.86% Balanced Accuracy**, **9% fraud Precision**, **23% fraud Recall**, and **12% F1-Score**.

Although threshold optimization improved fraud detection relative to the initial Autoencoder configuration, the resulting Recall remained substantially limited. The experiment therefore indicated that reconstruction-error-based anomaly detection was less effective for the selected dataset than the stronger supervised approaches investigated during subsequent development.

3.2.3 GAN-Based Fraud Generation

The GAN-based experiment investigated a different strategy for addressing class imbalance: **generating synthetic fraudulent samples** to increase the representation of the minority class during classifier training using a Generative Adversarial Network (GAN) [15]. Generated fraud samples were combined with legitimate training observations and subsequently used to train a fraud classifier.

When evaluated against the original test data at a decision threshold of **0.50**, the resulting classifier achieved a **Balanced Accuracy of 88.01%**.

The experiment established that generative modeling could be incorporated into the fraud-detection workflow as a mechanism for augmenting the minority class. However, it remained an exploratory branch of the project and was not retained in the subsequent ensemble architecture.

3.2.4 Early XGBoost Classification

The early XGBoost experiment investigated supervised gradient-boosted classification using XGBoost [16], while accounting for class imbalance through `scale_pos_weight`. At the default classification threshold of **0.50**, the model achieved approximately **93.46% Balanced Accuracy**.

Threshold optimization was subsequently performed using Balanced Accuracy as the optimization criterion. A threshold of approximately **0.27** increased Balanced Accuracy to approximately **93.87%**, with fraud Recall of approximately **94%**, Precision of approximately **5%**, and an F1-Score of approximately **10%**.

The experiment demonstrated strong sensitivity to fraudulent transactions, but the combination of high Recall and very low Precision indicated that this sensitivity was obtained at the cost of a substantial number of False Positives. It also provided early evidence that the **decision threshold itself constitutes an important component of fraud-classification performance**, a consideration that was revisited during final model selection.

3.3 Preliminary Experimental Findings

Initial Experiment	Balanced Accuracy	Fraud Precision	Fraud Recall	Fraud F1
DNN with Entity Embeddings	91.70%	10.96%	86.11%	19.45%
Autoencoder	60.86%	≈9%	≈23%	≈12%
GAN-Based Experiment	88.01%	≈3.97%	≈83.9%	≈7.58%
Early XGBoost	93.87%	≈5%	≈94%	≈10%

Table 5: Comparative performance of the initial FraudGuard modeling experiments.

The preliminary experiments revealed substantial differences in the behavior of the evaluated learning approaches. While several configurations achieved strong fraud sensitivity, this was frequently accompanied by low Precision and a considerable False Positive burden. The experiments also demonstrated that classification performance was sensitive to the selected decision threshold, particularly in the early XGBoost evaluation.

Based on these findings, the preliminary approaches were not carried forward as the predictive architecture of FraudGuard. The Deep Neural Network exhibited strong fraud Recall but generated an excessive number of False Positives, while the Autoencoder demonstrated insufficient fraud-detection capability. The GAN-based approach remained an exploratory data-augmentation strategy rather than a component of the subsequent predictive ensemble. The early XGBoost experiment demonstrated stronger fraud sensitivity and highlighted the importance of decision-threshold selection; however, it represented an initial experimental configuration rather than the systematically configured XGBoost model evaluated in the subsequent development stage.

These outcomes did not imply that the underlying modeling families were inherently unsuitable. Rather, the preliminary experiments identified limitations in the initial configurations and established the evaluation criteria required for more systematic model develop-

ment. Consequently, FraudGuard progressed to a **systematic multi-model evaluation framework**, in which candidate models were trained and assessed under a common experimental methodology to determine their individual and combined suitability for the fraud-detection task.

4 Predictive Model Development and Ensemble Architecture

4.1 Systematic Model Development Methodology

The systematic model-development stage presented in this chapter was informed by the **preliminary experiments described in Chapter 3**. Those experiments were not treated as the final candidate-model evaluation; rather, they served to identify limitations in the initial configurations and to establish the evaluation criteria required for subsequent development. Based on these findings, FraudGuard progressed to a systematic **multi-model development stage**, in which multiple candidate models were trained and evaluated under a common methodology. Some modeling families explored during the preliminary stage, including **Neural Networks and XGBoost**, were revisited within this systematic framework using configurations specific to the subsequent candidate-model evaluation.

Rather than continuing to optimize a single classifier in isolation, this stage investigated whether models based on different learning mechanisms could provide complementary predictive signals for subsequent ensemble construction. Six candidate architectures were evaluated: **Random Forest, LightGBM, CatBoost, XGBoost, a Neural Network, and Isolation Forest**. Random Forest, LightGBM, CatBoost, XGBoost, and the Neural Network were trained as supervised classifiers using the known `is_fraud` target, whereas Isolation Forest was evaluated as an unsupervised anomaly-detection method and therefore did not learn directly from the fraud labels. The supervised implementations incorporated class-balancing mechanisms where appropriate to account for the substantial imbalance between legitimate and fraudulent transactions.

To ensure a consistent comparison across candidate models, the systematic evaluation stage applied a **common experimental framework**. Model performance was assessed on the separate test dataset using **Precision, Recall, F1-Score, Balanced Accuracy, ROC-AUC, PR-AUC, and Confusion Matrix analysis**. For the initial comparison of the supervised classifiers, probability outputs were converted into binary predictions using a common decision threshold of **0.50**. This standardized evaluation procedure enabled differences in model performance to be attributed to their predictive behavior rather than to different evaluation criteria or operating thresholds.

4.2 Individual Candidate Model Evaluation

Model	Learning Paradigm	Modeling Approach
Random Forest	Supervised ML	Bagging-based tree ensemble
LightGBM	Supervised ML	Gradient-boosted decision trees
CatBoost	Supervised ML	Gradient-boosted decision trees
XGBoost	Supervised ML	Gradient-boosted decision trees
Neural Network	Supervised DL	Fully connected neural architecture
Isolation Forest	Unsupervised ML	Anomaly detection

Table 6: Candidate model architectures evaluated during the multi-model development stage.

4.2.1 Random Forest

Random Forest is a **supervised ensemble-learning algorithm** composed of multiple decision trees whose predictions are aggregated to produce the final classification output [17]. By combining the decisions of many trees rather than relying on a single tree, the model provides a more robust prediction and reduces sensitivity to the structure of any individual estimator.

Within FraudGuard, Random Forest was implemented as a **probabilistic binary classifier**, producing an estimated fraud probability for each transaction in addition to its final classification.

In the FraudGuard experiments, Random Forest was configured with **250 decision trees**. The trees were allowed to grow without an explicitly defined maximum depth (`max_depth=None`), while a minimum of **two samples per leaf** was imposed to constrain the formation of highly specific terminal nodes.

To account for the substantial class imbalance in the transaction dataset, the classifier used `class_weight='balanced_subsample'`. Under this configuration, class weights are recalculated for the bootstrap sample used to construct each individual tree, increasing the relative influence of the minority fraud class during model fitting. The implementation also enabled parallel tree construction through `n_jobs=-1`. These settings are explicitly defined in the project notebook.

Parameter	Value
<code>n_estimators</code>	250
<code>max_depth</code>	None
<code>min_samples_leaf</code>	2
<code>class_weight</code>	<code>balanced_subsample</code>
<code>n_jobs</code>	-1

Table 7: Random Forest hyperparameter configuration used in the FraudGuard experiments.

In addition to producing a binary classification, the Random Forest model generates a **class-probability estimate** for each transaction. In FraudGuard, the probability associated with the positive class represents the model’s estimated likelihood that a transaction is fraudulent. During standalone evaluation, this probability can be converted into a binary prediction by comparing it with a specified decision threshold.

For the subsequent Stacking architecture, however, the continuous probability estimate is more informative than the binary decision alone. The Random Forest fraud probability is therefore preserved as the meta-feature `rf_prob`. This allows the Logistic Regression meta-model to receive information about the **strength of the Random Forest prediction**, rather than receiving only a hard Fraud or Legitimate classification.

Under the common evaluation framework defined in Section 4.1, Random Forest produced the following results on the separate test dataset.

Metric	Random Forest
Precision	0.9433
Recall	0.7207
F1-Score	0.8171
Balanced Accuracy	0.8603
ROC-AUC	0.9867
PR-AUC	0.8671

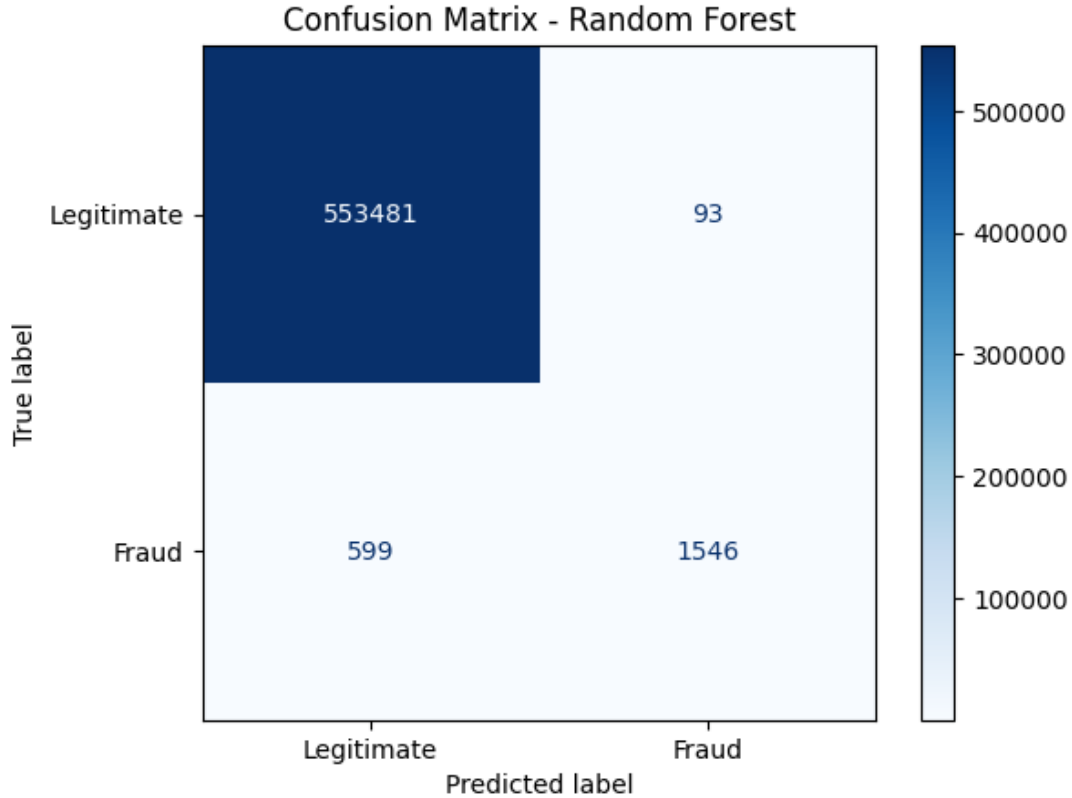


Figure 4: Confusion Matrix of the Random Forest classifier on the separate test dataset at a decision threshold of 0.50.

The Random Forest model demonstrated a **high-precision, comparatively conservative fraud-detection profile**. At the evaluated decision threshold, the model achieved a fraud Precision of **0.9433**, indicating that the substantial majority of transactions classified as fraudulent were indeed fraudulent. This behavior is also reflected in the Confusion Matrix, where only **93 legitimate transactions were incorrectly classified as fraud**.

However, this high Precision was accompanied by a lower Recall of **0.7207**. Of the **2,145 fraudulent transactions** in the test dataset, Random Forest correctly detected **1,546**, while **599 fraud cases remained undetected**. The model therefore exhibited a clear trade-off between limiting False Positives and maximizing fraud detection.

These results demonstrate the predictive characteristic that made Random Forest relevant to the subsequent ensemble-development stage: the model provided a **high-Precision, low-False-Positive fraud signal that differed from the higher-Recall operating profiles observed in other candidate models**. Its continuous probability output was therefore retained as a candidate meta-feature for the subsequent Stacking architecture.

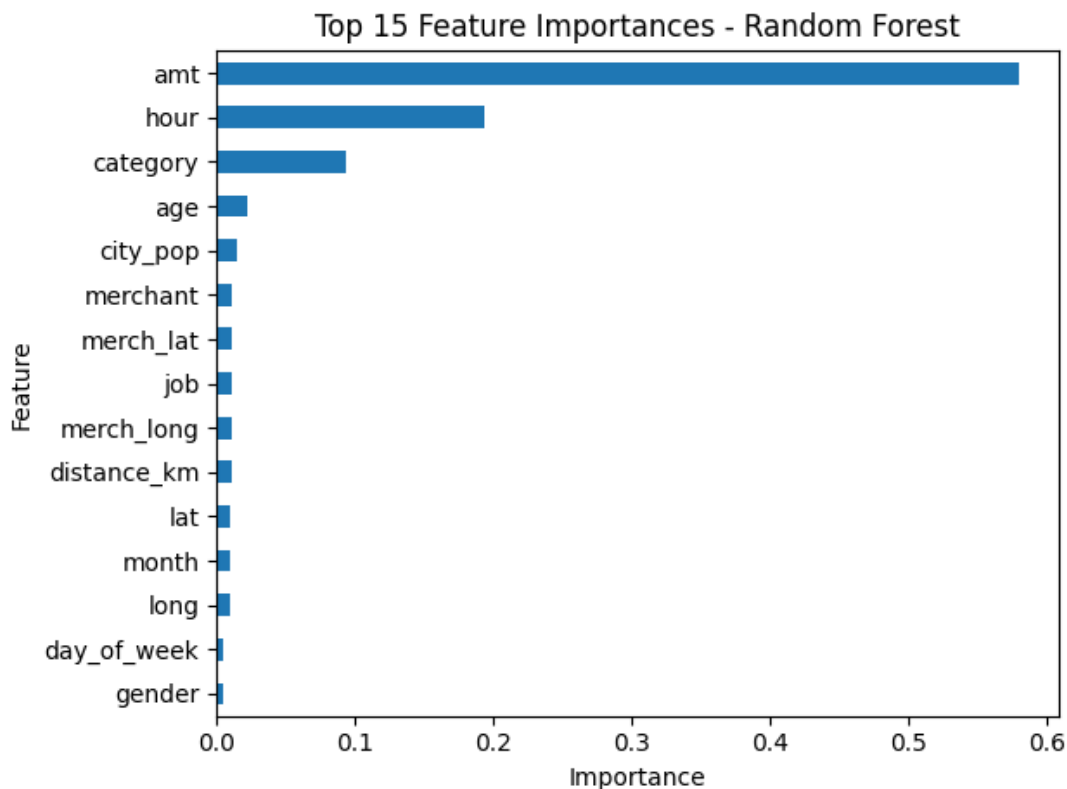


Figure 5: Top 15 feature importances identified by the Random Forest classifier.

The Random Forest feature-importance analysis shows that transaction amount (amt) was the dominant feature, followed by transaction hour (hour) and merchant category (category). The substantial difference between the three leading features and the remaining variables indicates that the Random Forest model assigned considerably greater importance to a relatively small subset of the available feature space.

The prominence of amt and hour also highlights the contribution of both original transaction information and engineered temporal information to the Random Forest model. In

particular, hour, which was derived during the feature-engineering stage, emerged as the second most influential feature according to the model’s internal importance measure.

4.2.2 LightGBM

LightGBM is a supervised gradient-boosting algorithm that constructs an ensemble of decision trees sequentially [18]. Gradient boosting develops the predictive function iteratively, with successive trees learning to reduce the errors remaining from earlier stages of the ensemble.

Within FraudGuard, LightGBM was implemented as a probabilistic binary classifier and evaluated as one of the candidate models for subsequent Stacking. The model produced a continuous fraud-probability estimate for each transaction, which was later represented by the meta-feature `lgbm_prob` during the Out-of-Fold Stacking stage.

The LightGBM candidate was configured with 350 boosting iterations and a learning rate of 0.05, with each tree permitted up to 64 leaves and no explicit maximum depth. The configuration also incorporated balanced class weighting during training.

Parameter	Value
<code>n_estimators</code>	350
<code>learning_rate</code>	0.05
<code>num_leaves</code>	64
<code>max_depth</code>	-1
<code>class_weight</code>	balanced
<code>n_jobs</code>	-1

Table 8: LightGBM hyperparameter configuration used in the FraudGuard experiments.

Under the common evaluation framework defined in Section 4.1, the trained LightGBM produced the following results:

Metric	LightGBM
Precision	0.4564
Recall	0.8956
F1-Score	0.6047
Balanced Accuracy	0.9457
ROC-AUC	0.9967
PR-AUC	0.8714

Table 9: Performance of the LightGBM classifier on the separate test dataset at a decision threshold of 0.50.

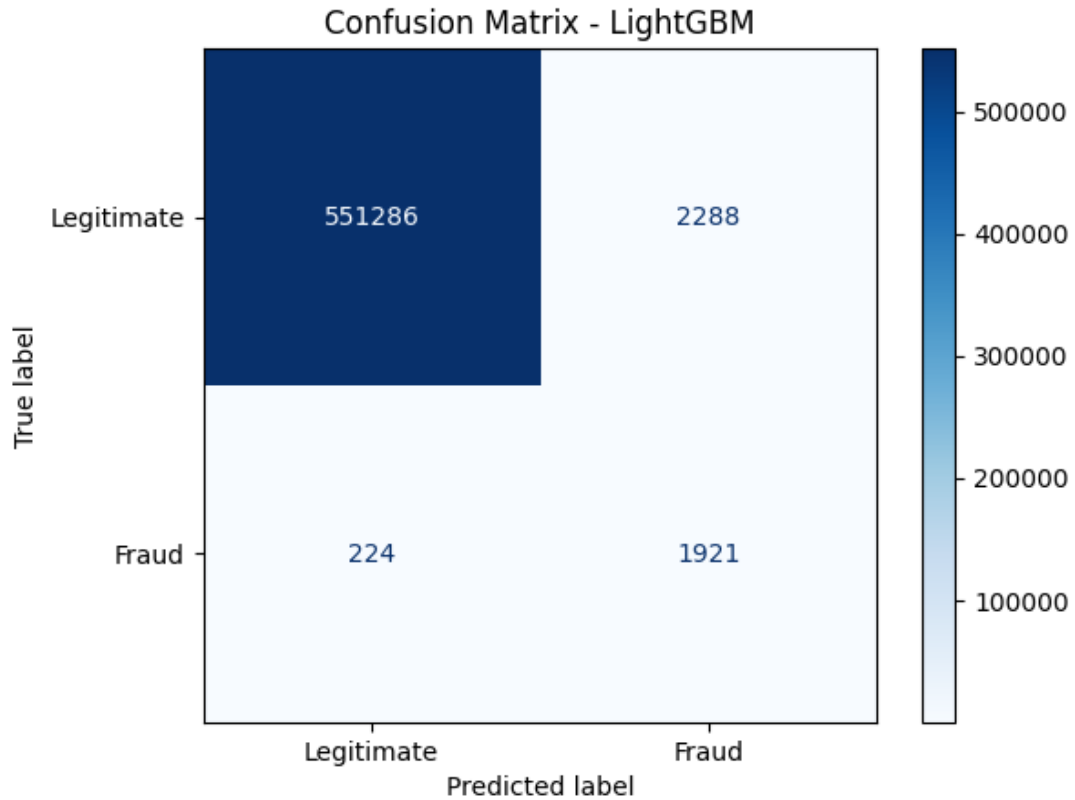


Figure 6: Confusion Matrix of the LightGBM classifier on the separate test dataset at a decision threshold of 0.50.

The LightGBM results demonstrate a **high-sensitivity fraud-detection profile**, achieving strong Recall while producing a lower Precision. This indicates that the model successfully identified a large proportion of fraudulent transactions, but at the cost of a greater number of legitimate transactions being incorrectly flagged as fraud. Despite this trade-off, its strong Balanced Accuracy, ROC-AUC, and PR-AUC supported its retention for the subsequent ensemble-development stage.

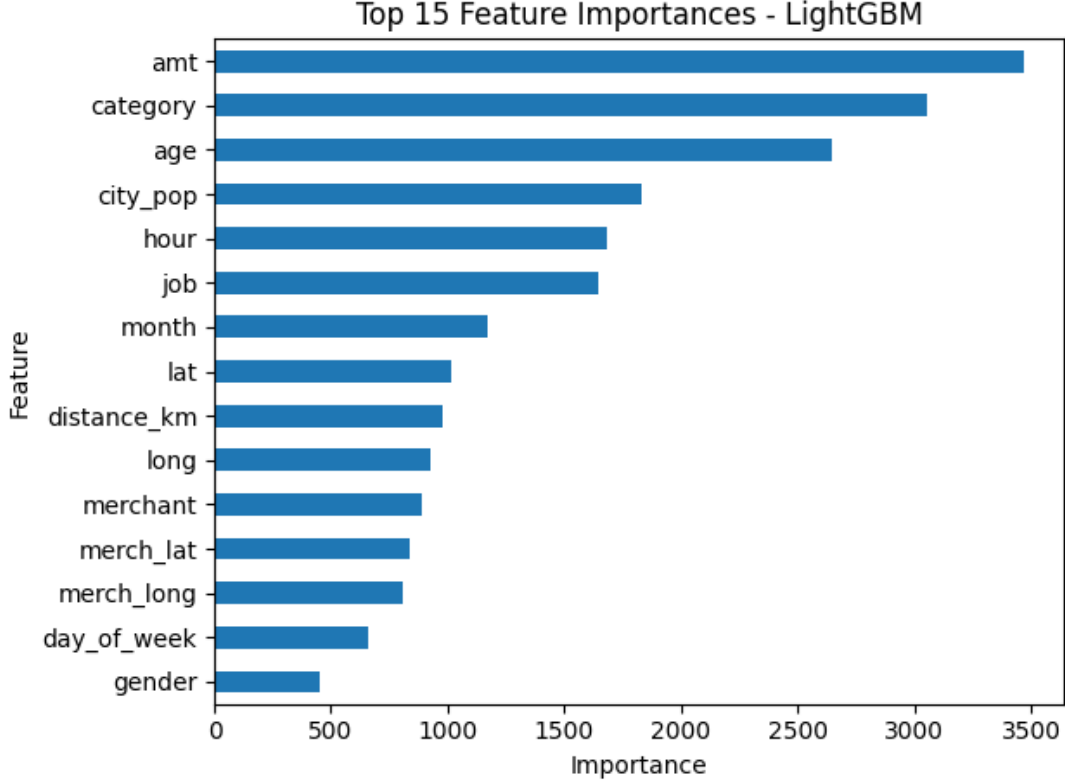


Figure 7: Top 15 feature importances identified by the LightGBM classifier.

The LightGBM feature-importance analysis shows that **transaction amount (amt)** received the **highest importance**, followed by **merchant category (category)** and **customer age (age)**. Unlike a highly concentrated importance profile, however, LightGBM distributed substantial importance across several additional variables, including `city_pop`, `hour`, and `job`.

The ranking indicates that LightGBM incorporated information from a relatively broad set of transaction, customer, temporal, and geographical characteristics. Notably, engineered features such as `age`, `hour`, and `distance_km` appear among the higher-ranked variables, demonstrating that the feature-engineering stage contributed information that was actively utilized by the model.

4.2.3 XGBoost

XGBoost is a **supervised gradient-boosting algorithm** that constructs an ensemble of decision trees through an iterative optimization process [16]. Successive trees are added to improve the current predictive function by addressing residual classification errors, allowing the ensemble to progressively capture complex nonlinear relationships within the feature space.

Following the preliminary XGBoost experiment described in Chapter 3, XGBoost was revisited during the systematic model-development stage under the standardized candidate-model evaluation framework. The configuration presented in this section therefore represents the subsequent systematic XGBoost candidate rather than the initial exploratory

experiment.

Within FraudGuard, XGBoost was implemented as a **probabilistic binary classifier** and evaluated as one of the candidate models for subsequent Stacking. Its continuous fraud-probability output was later represented by the meta-feature `xgb_prob` during the Out-of-Fold Stacking stage.

The XGBoost classifier was configured with **350 boosting iterations** and a **learning rate of 0.05**, with individual trees restricted to a maximum depth of **6**. Row and feature subsampling were both applied at a rate of **0.9** through `subsample` and `colsample_bytree`, respectively. To address the strong class imbalance, `scale_pos_weight` was calculated from the ratio between legitimate and fraudulent transactions in the training data rather than assigned as a fixed constant. The model used `aucpr` as its evaluation metric.

Parameter	Value
<code>n_estimators</code>	350
<code>learning_rate</code>	0.05
<code>max_depth</code>	6
<code>subsample</code>	0.9
<code>colsample_bytree</code>	0.9
<code>scale_pos_weight</code>	Calculated from training-class distribution
<code>eval_metric</code>	<code>aucpr</code>
<code>n_jobs</code>	-1

Table 10: XGBoost hyperparameter configuration used in the FraudGuard experiments.

Under the common evaluation framework defined in Section 4.1, the trained XGBoost produced the following results:

Metric	XGBoost
Precision	0.3123
Recall	0.9371
F1-Score	0.4684
Balanced Accuracy	0.9645
ROC-AUC	0.9971
PR-AUC	0.8712

Table 11: Performance of the XGBoost classifier on the separate test dataset at a decision threshold of 0.50.

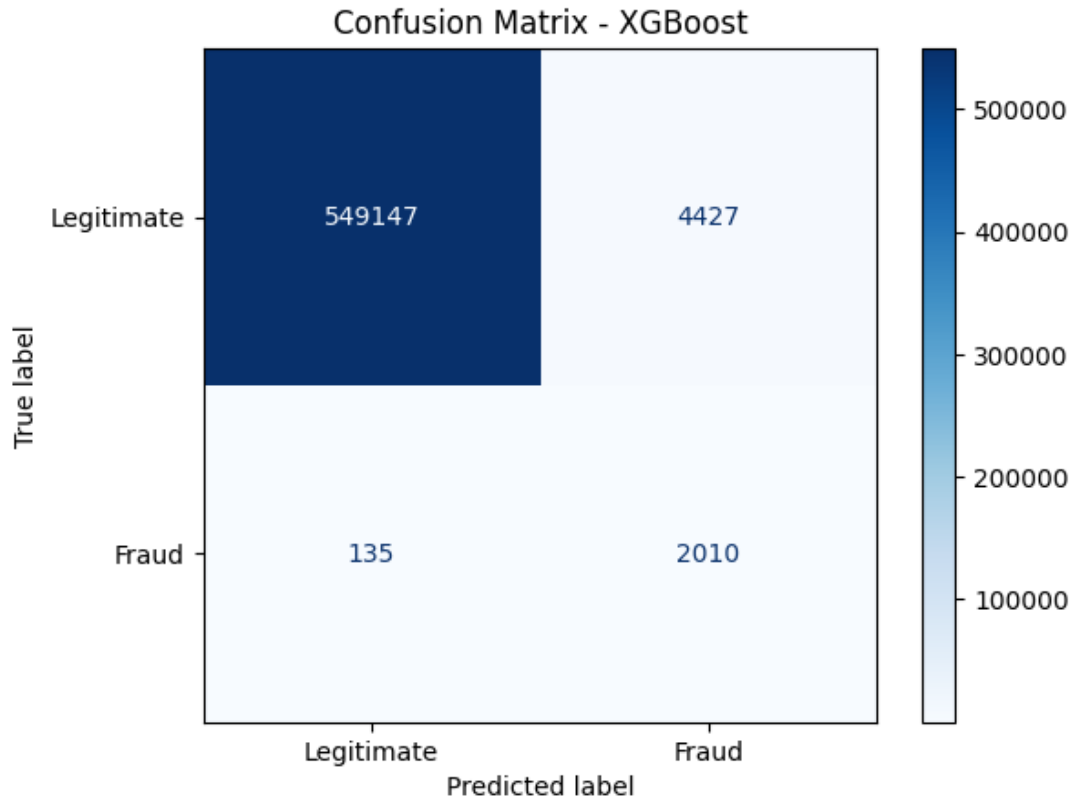


Figure 8: Confusion Matrix of the XGBoost classifier on the separate test dataset at a decision threshold of 0.50.

The XGBoost results demonstrate a **high-recall fraud-detection profile**. The model detected **2,010 of the 2,145 fraudulent transactions**, leaving only **135 fraudulent cases undetected**. This strong sensitivity was accompanied by a greater False Positive burden, with **4,427 legitimate transactions incorrectly classified as fraudulent**.

This operating behavior made XGBoost particularly relevant to the ensemble-development stage. Rather than serving as a conservative classifier, its probability output provided a **strong fraud-sensitive signal that differed from the operating characteristics of the more precision-oriented candidates**. The model was therefore retained for subsequent OOF Stacking.

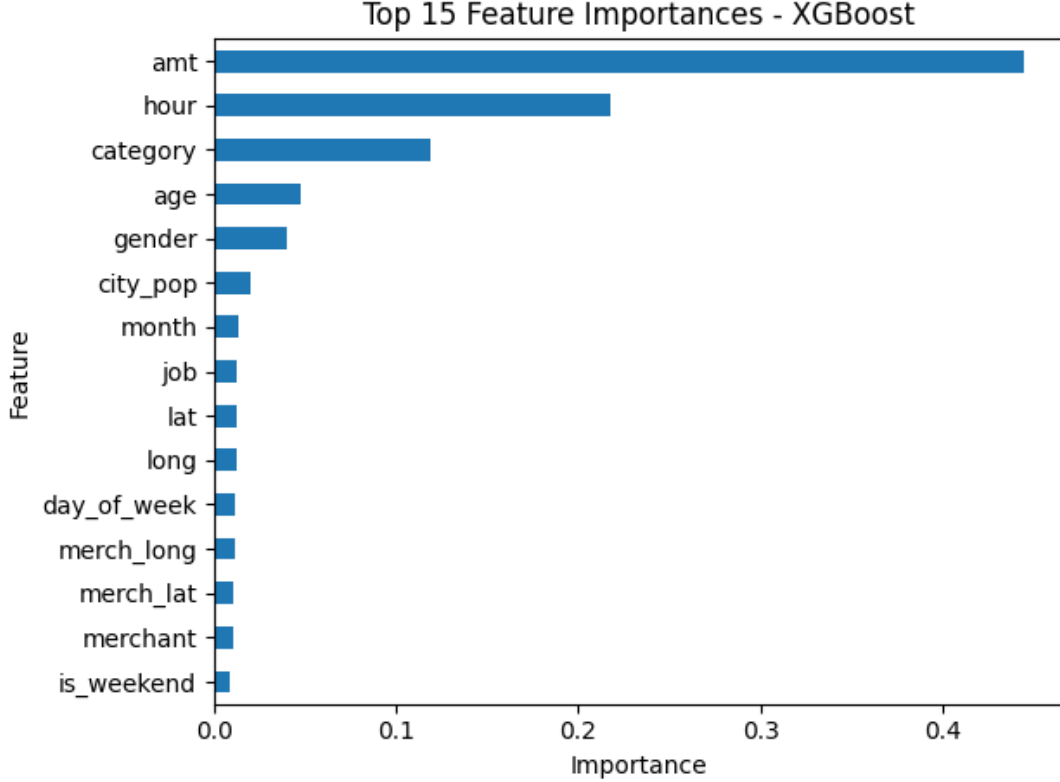


Figure 9: Top 15 feature importances identified by the XGBoost classifier.

The XGBoost feature-importance analysis shows that **transaction amount (amt)** received the highest importance, followed by **transaction hour (hour)** and **merchant category (category)**. These three variables were substantially more prominent than the remaining features, with amt exhibiting the strongest individual importance.

The ranking further indicates that the model incorporated information from multiple feature groups. Variables including age, gender, and city_pop contributed additional predictive information, while engineered temporal variables such as hour, month, day_of_week, and is_weekend also appeared among the fifteen ranked features. As with the other feature-importance analyses, these values quantify the relative importance assigned by the model but do not indicate the **direction** of each feature’s effect on the predicted fraud probability.

4.2.4 CatBoost

CatBoost is a **supervised gradient-boosting algorithm** based on decision trees [19], in which successive trees are constructed iteratively to improve the predictive function. Within FraudGuard, CatBoost was implemented as a **probabilistic binary classifier** and evaluated as one of the supervised candidates in the systematic model-development stage.

For each transaction, the classifier produced a continuous probability estimate for the positive fraud class. This output was subsequently represented by the meta-feature cat_prob during the **Out-of-Fold Stacking** stage, allowing the ensemble to incorpo-

rate the strength of CatBoost’s fraud prediction rather than reducing its output to a binary Fraud or Legitimate decision.

The CatBoost classifier was trained for **350 boosting iterations** with a **learning rate of 0.05** and a tree **depth of 8**. Class imbalance was addressed using `auto_class_weights="Balanced"`, automatically adjusting the relative contribution of the classes during training. The model optimized **binary Logloss**, while **PRAUC** was specified as the evaluation metric, providing an evaluation criterion suited to the project’s highly imbalanced binary-classification setting.

Parameter	Value
iterations	350
learning_rate	0.05
depth	8
auto_class_weights	Balanced
loss_function	Logloss
eval_metric	PRAUC

Table 12: CatBoost hyperparameter configuration used in the FraudGuard experiments.

Under the common evaluation framework defined in Section 4.1, the trained CatBoost produced the following results:

Metric	CatBoost
Precision	0.2957
Recall	0.9483
F1-Score	0.4508
Balanced Accuracy	0.9698
ROC-AUC	0.9971
PR-AUC	0.8694

Table 13: Performance of the CatBoost classifier on the separate test dataset at a decision threshold of 0.50.

At this threshold, CatBoost correctly identified **2,034 of the 2,145 fraudulent transactions**, corresponding to the reported Recall of approximately **94.83%**.

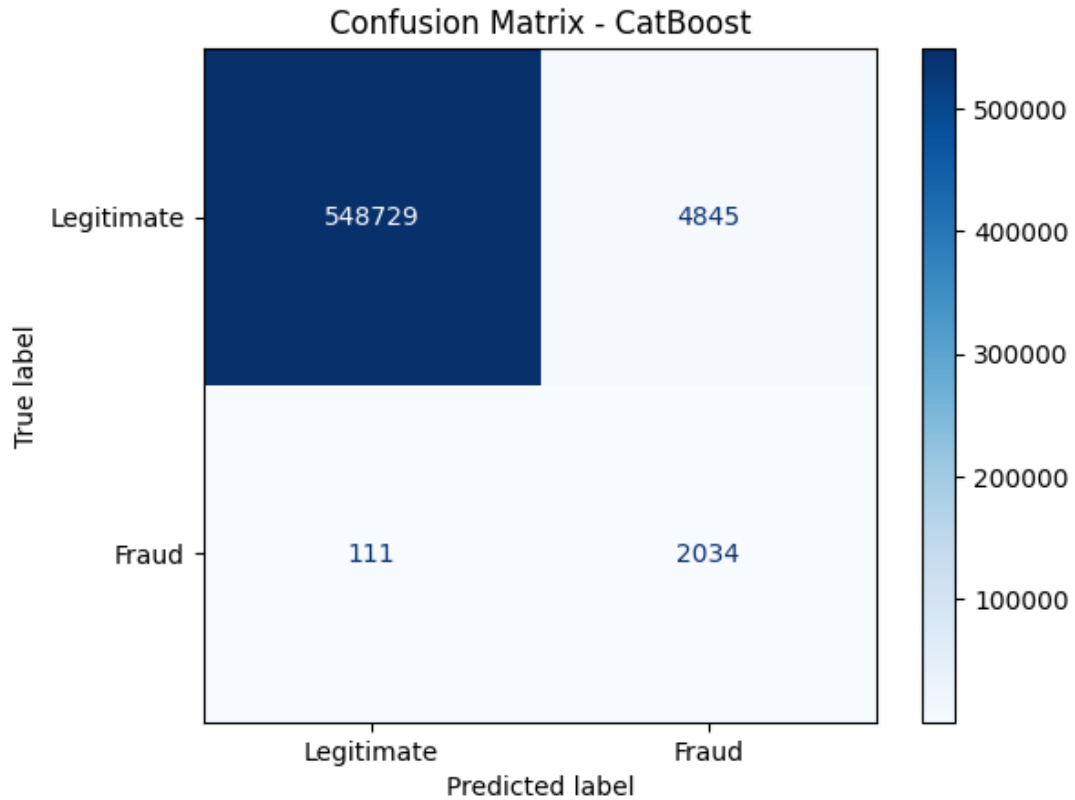


Figure 10: Confusion Matrix of the CatBoost classifier on the separate test dataset at a decision threshold of 0.50.

The defining characteristic of the CatBoost result was its **high fraud Recall**. With only **111 fraudulent transactions remaining undetected**, the classifier captured the substantial majority of fraud cases in the separate test dataset. This sensitivity was accompanied by lower Precision, indicating that achieving this level of fraud coverage also introduced a larger False Positive burden.

This behavior made CatBoost valuable beyond its standalone classification performance. Its continuous fraud-probability output provided a strongly fraud-sensitive signal and was therefore retained as `cat_prob` for investigation during the subsequent ensemble-development stage.

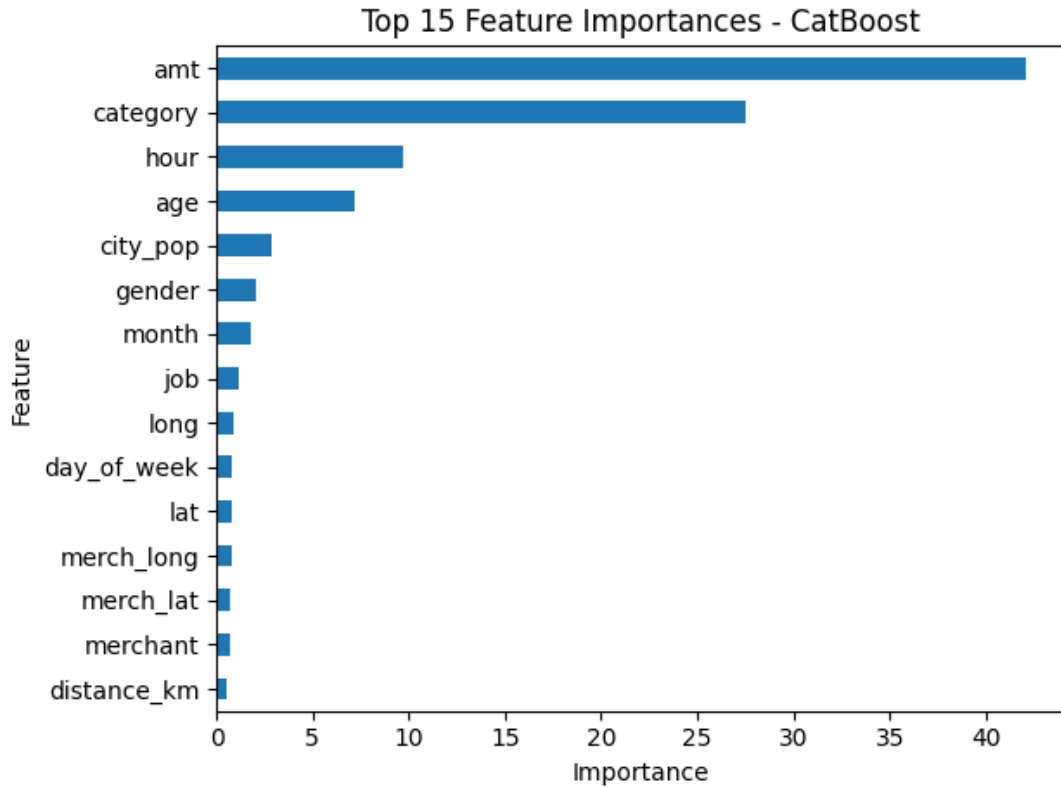


Figure 11: Top 15 feature importances identified by the CatBoost classifier.

The CatBoost feature-importance analysis shows that **transaction amount (amt)** received the highest importance, followed by **merchant category (category)** and **transaction hour (hour)**. Together, these features account for the most prominent contributions in the ranking, while the importance assigned to the remaining variables decreases considerably.

Beyond these dominant transaction characteristics, age represents the next most prominent feature, followed by smaller contributions from variables such as city_pop, gender, and month. Engineered features including hour, age, month, day_of_week, and distance_km are represented within the ranking, demonstrating that CatBoost utilized information introduced during the feature-engineering stage alongside the original transaction attributes.

As with the preceding feature-importance analyses, the ranking describes the **relative importance assigned to each variable**, but does not establish whether a particular feature value increases or decreases the predicted fraud probability. That directional interpretation is addressed separately through the project’s explainability layer.

4.2.5 Neural Network

The Neural Network introduced a **Deep Learning component** into the systematic candidate-model evaluation. Rather than partitioning the feature space through decision-tree structures, the network learns a nonlinear mapping between the transaction features and the probability of fraud through a sequence of fully connected layers and learned

transformations.

Neural-network modeling had previously been investigated during the preliminary experiments described in Chapter 3 through the Deep Neural Network with Entity Embeddings. In the systematic development stage, the Neural Network was revisited as a separate candidate configuration under the standardized evaluation framework defined in Section 4.1. The architecture presented in this section therefore represents the subsequent systematic Neural Network candidate rather than the preliminary Entity-Embedding experiment.

Within FraudGuard, the Neural Network was included to provide a predictive representation structurally different from the tree-based classifiers. Its output was a continuous fraud probability, subsequently represented by the meta-feature `nn_prob` during Out-of-Fold Stacking. This allowed the subsequent ensemble-development stage to investigate whether a Deep Learning representation could provide useful information alongside the probability outputs generated by the tree-based classifiers.

Unlike the tree-based candidates, the Neural Network received **standardized numerical inputs**. A StandardScaler was fitted to the training features and subsequently applied to the test features before they were passed through the network. Class weights were also computed from the training labels using a balanced weighting strategy to increase the influence of the minority fraud class during optimization.

The network consisted of four hidden Dense layers containing **256, 128, 64, and 32 units**, respectively. The first three hidden layers incorporated ReLU activation together with regularization through Batch Normalization and Dropout. Dropout rates of **0.30, 0.30, and 0.20** were applied following the first three hidden representations. The final layer contained a single neuron with a **sigmoid activation**, producing the fraud probability required for binary classification.

Component	Configuration
Input preprocessing	StandardScaler
Hidden Layer 1	256 units, Batch Normalization, ReLU, Dropout 0.30
Hidden Layer 2	128 units, Batch Normalization, ReLU, Dropout 0.30
Hidden Layer 3	64 units, Batch Normalization, ReLU, Dropout 0.20
Hidden Layer 4	32 units, ReLU
Output Layer	1 unit, Sigmoid
Optimizer	Adam
Initial learning rate	0.001
Loss function	Binary Cross-Entropy
Class handling	Balanced class weights

Table 14: Neural Network architecture and training configuration used in the FraudGuard systematic model evaluation.

The model was trained for a maximum of **50 epochs** using a batch size of **4,096**, with **20% of the training data reserved for validation**. Training incorporated Early Stopping based on validation PR-AUC, with a patience of five epochs and restoration of the best-performing weights. A ReduceLROnPlateau callback reduced the learning rate when validation PR-AUC stopped improving, while model checkpointing preserved the best network observed during training.

This use of **validation PR-AUC as the principal monitoring signal** is particularly relevant to the fraud-detection setting because model selection during training was tied to performance on the rare positive class rather than overall Accuracy.

Under the common evaluation framework defined in Section 4.1, the trained Neural Network produced the following results:

Metric	Neural Network
Precision	0.0742
Recall	0.9389
F1-Score	0.1375
Balanced Accuracy	0.9468
ROC-AUC	0.9867
PR-AUC	0.6898

Table 15: Performance of the Neural Network on the separate test dataset at a decision threshold of 0.50.

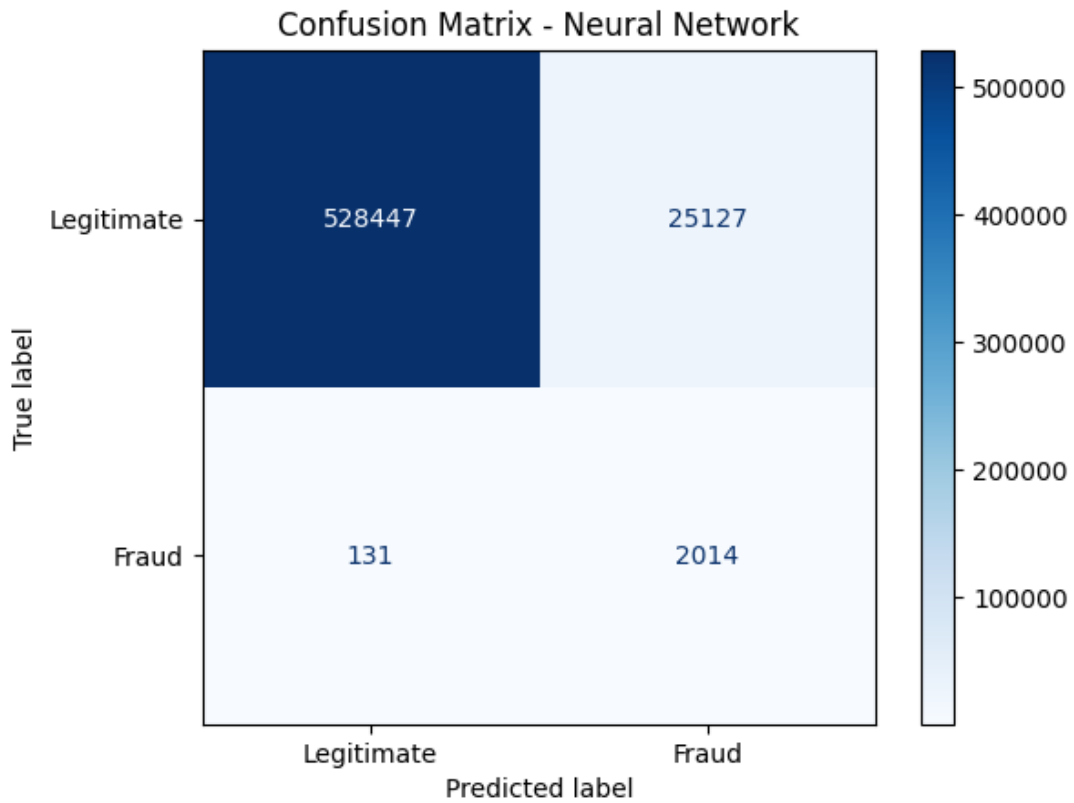


Figure 12: Confusion Matrix of the Neural Network on the separate test dataset at a decision threshold of 0.50.

The Neural Network exhibited a high-Recall but low-Precision fraud-detection profile. It correctly identified 2,023 of the 2,145 fraudulent transactions, corresponding to a Recall of 0.9431, while only 122 fraudulent transactions remained undetected. However, this high sensitivity was accompanied by 23,797 False Positives, resulting in a substantially lower Precision of 0.0783.

The model therefore demonstrated a markedly different operating profile from the more conservative classifiers evaluated in the systematic comparison. Its strong fraud sensitivity was achieved at the cost of a considerable False Positive burden, limiting its suitability as a standalone fraud classifier at the evaluated operating threshold.

Nevertheless, the Neural Network represented a **structurally distinct learning approach** from the tree-based candidate models. Its continuous fraud-probability output was therefore retained as `nn_prob` for investigation during the subsequent ensemble-development stage.

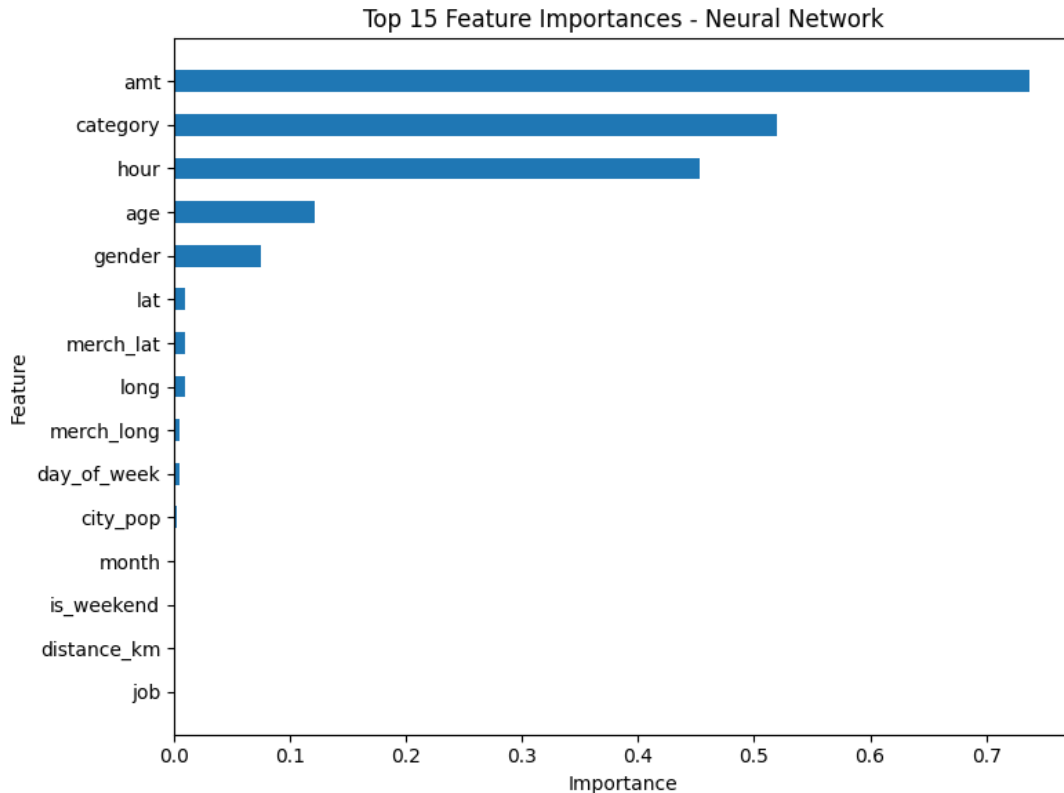


Figure 13: Top 15 feature importances identified for the Neural Network classifier.

The Neural Network feature-importance analysis indicates that transaction amount (`amt`) was the most influential input, followed by transaction category (`category`) and transaction hour (`hour`). These three features exhibited substantially greater importance than the remaining inputs, suggesting that the network relied primarily on transaction magnitude, merchant-category information, and temporal behavior when forming its fraud predictions. Among the remaining features, `age` and `gender` showed smaller contributions, while the other variables exhibited comparatively limited importance according to the applied importance analysis. The prominence of `hour` is also notable because it represents an engineered temporal feature, providing further evidence that the feature-engineering stage contributed information used by the predictive models.

4.2.6 Isolation Forest

Isolation Forest was investigated as an **unsupervised anomaly-detection approach** [20], providing a fundamentally different modeling strategy from the supervised classifiers evaluated during the same experimental stage. Rather than learning a direct mapping between transaction features and the known `is_fraud` label, the method attempts to identify observations that exhibit anomalous characteristics relative to the broader transaction population.

Its inclusion in the candidate-model evaluation allowed the project to examine whether fraudulent transactions could be identified effectively through **anomaly detection alone**, without directly learning from the fraud labels. This provided an alternative perspective to the supervised Machine Learning and Deep Learning approaches evaluated in parallel.

Although Isolation Forest was trained as an unsupervised anomaly-detection model, its predictions were evaluated against the known fraud labels in the separate test dataset, enabling direct comparison with the supervised candidate models under the common evaluation framework defined in Section 4.1.

Metric	Isolation Forest
Precision	0.0482
Recall	0.0727
F1-Score	0.0580
Balanced Accuracy	0.5336
ROC-AUC	0.8071
PR-AUC	0.0294

Table 16: Performance of the Isolation Forest model on the separate test dataset.

The experiment detected only **156 of the 2,145 fraudulent transactions**, while **1,989 fraudulent transactions remained undetected**.

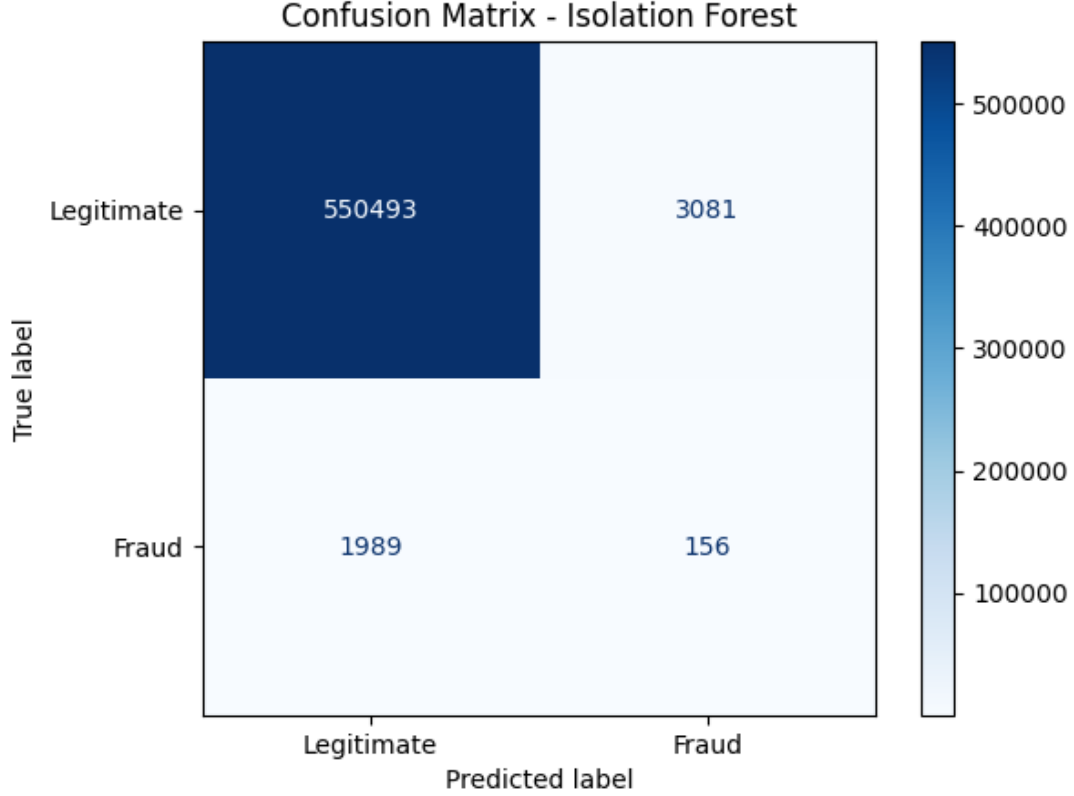


Figure 14: Confusion Matrix of the Isolation Forest model on the separate test dataset.

Isolation Forest exhibited substantially weaker fraud-detection performance than the supervised candidate models. Most importantly, its **Recall of 0.0727** indicates that the anomaly-detection approach identified only a small proportion of the fraudulent transactions. Its low **Precision of 0.0482** and **PR-AUC of 0.0294** further demonstrated limited effectiveness in separating fraudulent transactions from the overwhelmingly legitimate transaction population.

Based on these results, Isolation Forest was **excluded from subsequent ensemble development**. The project therefore continued with the five supervised candidates: Random Forest, LightGBM, CatBoost, XGBoost, and the Neural Network. Their probability outputs were subsequently investigated through the Out-of-Fold Stacking architecture, whereas Isolation Forest contributed no meta-feature to that stage.

4.3 Comparative Evaluation of Candidate Models

Following the individual evaluation of the candidate models, their performance was examined collectively to determine whether a single classifier provided a sufficiently strong fraud-detection solution or whether the observed differences between models justified an ensemble-based approach.

The comparison focused on Precision, Recall, F1-Score, Balanced Accuracy, ROC-AUC, and PR-AUC, rather than relying on conventional Accuracy. This distinction was necessary because of the substantial class imbalance in the dataset: a classifier can achieve high overall Accuracy while providing limited practical value for detecting the minority

fraud class. The comparative analysis therefore emphasized metrics that more directly characterize fraud detection and the trade-off between identifying fraudulent transactions and incorrectly flagging legitimate ones.

4.3.1 Comparative Predictive Performance

The supervised candidate models exhibited materially different operating characteristics. No individual model simultaneously dominated across all fraud-oriented evaluation criteria. Random Forest produced the strongest Precision, whereas CatBoost and XGBoost achieved substantially stronger Recall. LightGBM occupied an intermediate position between these operating profiles, while the Neural Network achieved high Recall but substantially lower Precision. Isolation Forest performed considerably below the supervised candidates and was therefore not retained for subsequent ensemble development.

This variation was particularly important because it demonstrated that the models exhibited distinct operating characteristics rather than a single uniform performance profile. Instead, they represented different trade-offs between fraud sensitivity and False Positive control. Random Forest provided a comparatively conservative fraud signal, while CatBoost and XGBoost were more sensitive to the positive fraud class. These differing operating characteristics provided the central motivation for investigating whether their continuous probability outputs could be combined rather than selecting a single standalone classifier.

Beyond these differences in threshold-specific classification performance, the continuous probability outputs of the supervised candidates were also compared to assess the degree of similarity among the predictive signals available for subsequent ensemble construction.

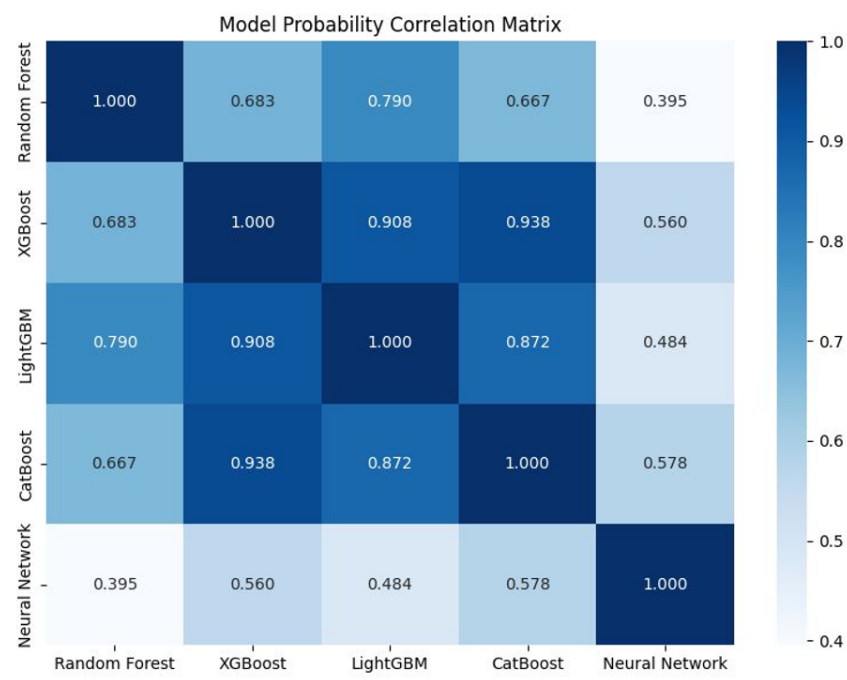


Figure 15: Pairwise correlation of fraud-probability outputs generated by the five supervised candidate models.

The probability-correlation analysis provides an additional perspective on the relationships among the supervised candidate models. The gradient-boosting classifiers exhibited comparatively strong correlations, particularly XGBoost and CatBoost (0.938), followed by XGBoost and LightGBM (0.908) and LightGBM and CatBoost (0.872). These values indicate substantial similarity in the continuous probability patterns produced by the gradient-boosting candidates.

In contrast, the Neural Network exhibited considerably lower correlations with the tree-based classifiers, ranging from 0.395 with Random Forest to 0.578 with CatBoost. Random Forest also showed lower correlations with XGBoost and CatBoost than those observed among the gradient-boosting models themselves. The probability outputs were therefore not equally correlated, providing additional motivation for experimentally investigating whether combining these signals through Stacking could improve the fraud-detection architecture.

4.3.2 Precision-Recall Analysis

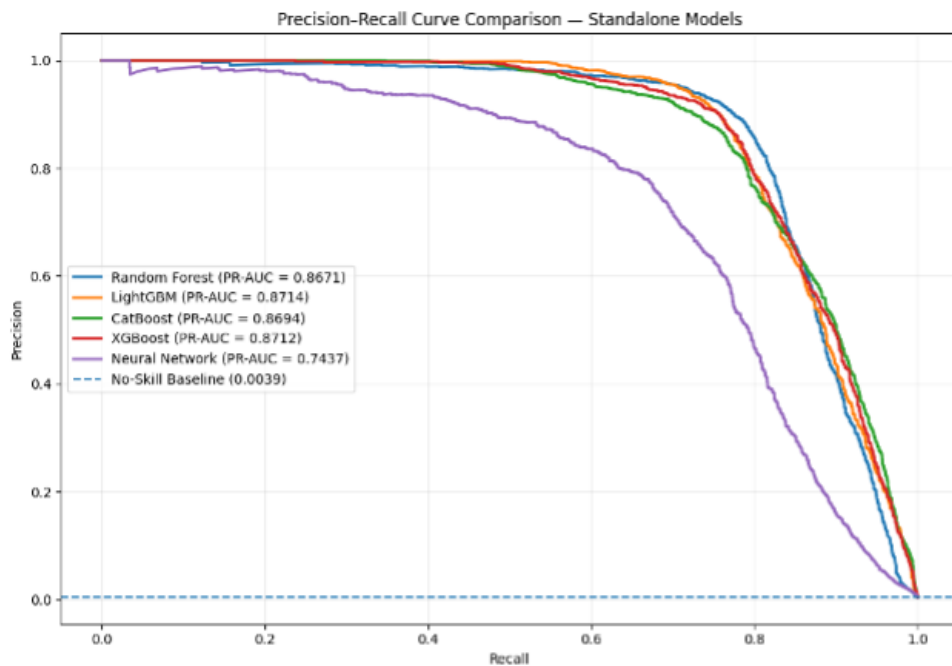


Figure 16: Precision-Recall curves of the five supervised candidate models on the separate test dataset, with the no-skill baseline shown for reference.

The Precision-Recall curves provide a threshold-independent view of the relationship between fraud detection and prediction precision. Rather than evaluating each classifier at a single decision threshold, the curves illustrate how Precision and Recall evolve as the operating threshold changes. This perspective is particularly informative for the highly imbalanced FraudGuard dataset because it focuses directly on performance with respect to the positive fraud class.

The PR-AUC results were closely grouped among the four tree-based supervised models, with LightGBM achieving 0.8714, XGBoost 0.8712, CatBoost 0.8694, and Random Forest

0.8671. The Neural Network achieved a lower PR-AUC of 0.6898, while still remaining substantially above the no-skill baseline of 0.0039. The similarity among the leading PR-AUC values further indicates that no single tree-based candidate established a decisive advantage across the complete Precision-Recall operating range.

The differences between the curves further demonstrate that the models exhibit distinct operating behaviors across possible decision thresholds. This observation later becomes important when the project moves beyond the common 0.50 threshold and evaluates the operating point of the final ensemble according to explicit fraud-detection requirements.

4.3.3 ROC Analysis

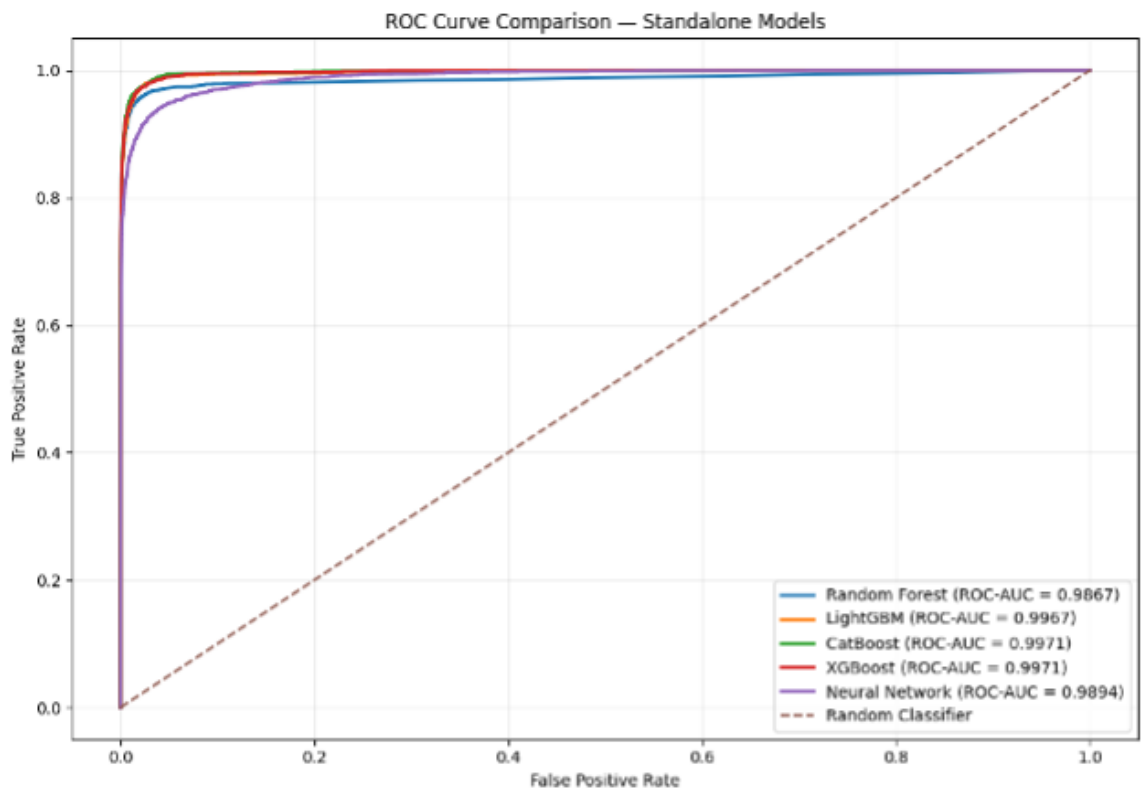


Figure 17: Receiver Operating Characteristic curves of the five supervised candidate models on the separate test dataset, with the random-classifier baseline shown for reference.

The ROC analysis provides a complementary view of discriminative performance by examining the relationship between the True Positive Rate and False Positive Rate across classification thresholds. The supervised candidates achieved strong ROC-AUC values, indicating a high overall ability to rank fraudulent transactions above legitimate transactions.

The ROC-AUC results were consistently high across all five supervised candidates. CatBoost and XGBoost achieved the highest ROC-AUC values at 0.9971, followed closely by LightGBM at 0.9967. The Neural Network achieved a ROC-AUC of 0.9867, while Random Forest achieved 0.9867. These results indicate strong ranking capability across the

supervised candidate set, despite the more pronounced differences observed in threshold-dependent Precision, Recall, and F1-Score.

However, ROC-AUC was not used in isolation to determine model suitability. In a dataset where fraudulent transactions represent only a small fraction of all observations, strong ROC-AUC can coexist with operationally significant differences in Precision and False Positive behavior. The ROC analysis was therefore considered alongside the Precision-Recall evaluation and threshold-specific classification metrics rather than serving as the sole model-selection criterion.

4.3.4 Computational Performance

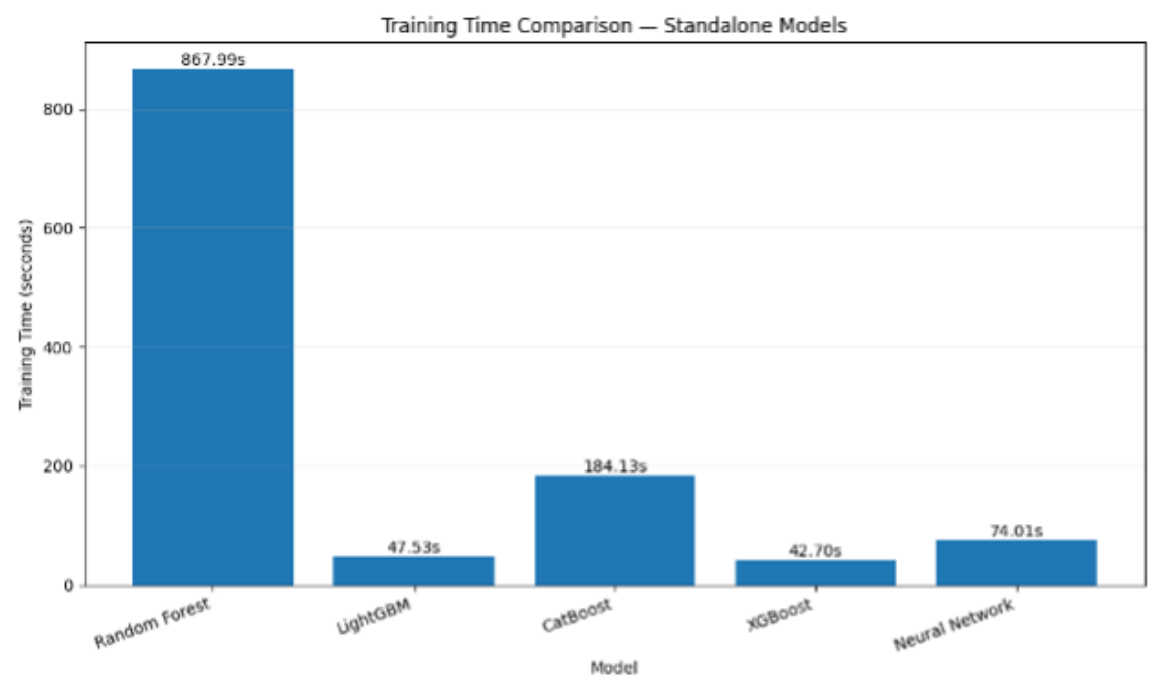


Figure 18: Training time of the five supervised candidate models during the systematic standalone-model evaluation.

The measured training times differed substantially across the candidate models. Random Forest required the longest training time at 867.99 seconds, followed by CatBoost at 184.13 seconds and the Neural Network at 74.01 seconds. LightGBM and XGBoost required 47.53 seconds and 42.70 seconds, respectively. These measurements demonstrate that the computational cost of model fitting varied considerably even among models exhibiting similarly strong predictive discrimination.

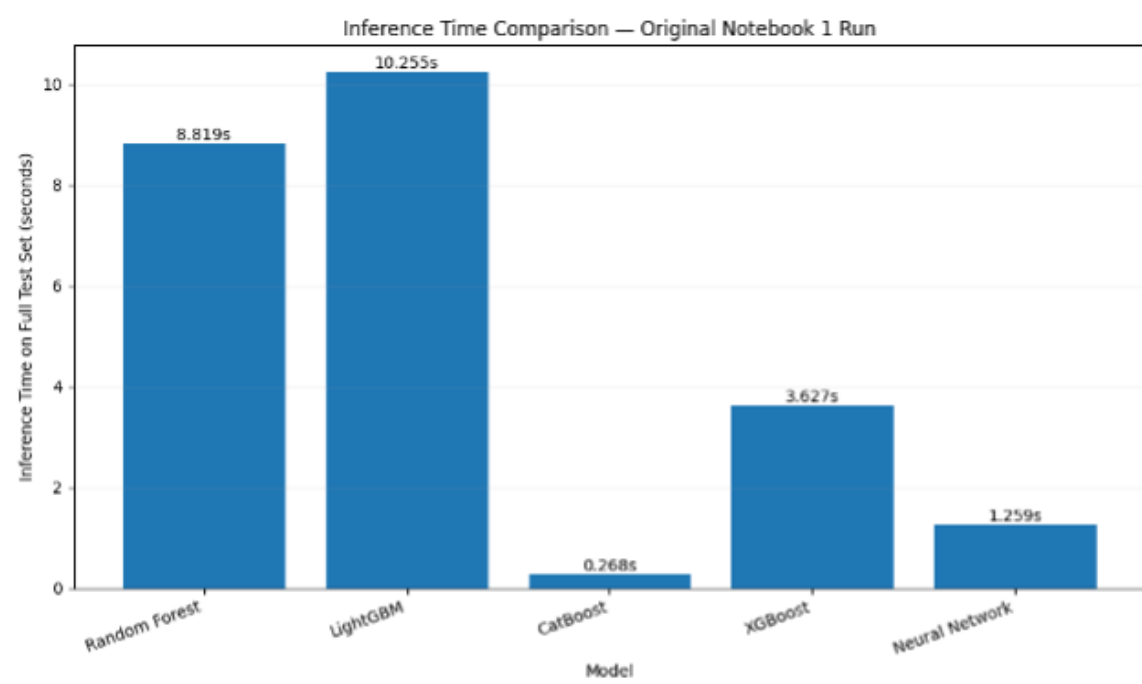


Figure 19: Inference time of the five supervised candidate models on the full separate test dataset.

Inference-time behavior followed a different pattern. LightGBM required the longest measured inference time at 10.255 seconds, followed by Random Forest at 8.819 seconds and XGBoost at 3.627 seconds. The Neural Network required 1.259 seconds, while CatBoost recorded the lowest inference time at 0.268 seconds for the evaluated test set. Thus, training cost and inference cost did not exhibit the same ordering across the candidate models.

Training and inference measurements provide complementary perspectives on computational efficiency. Training time reflects the computational cost associated with constructing or fitting each model, whereas inference time is particularly relevant to the intended operational setting because it determines the computational overhead associated with evaluating new transactions.

These measurements provided an additional system-level perspective on model development. Each base learner retained in an ensemble introduces additional computational requirements during model fitting and subsequent inference. Consequently, computational efficiency became particularly relevant during the later model-combination study, where FraudGuard investigated whether the ensemble could be reduced while preserving the desired fraud-detection behavior.

4.3.5 Transition to Ensemble Learning

The comparative evaluation demonstrated that the supervised candidate models exhibited complementary predictive characteristics rather than a single universally dominant operating profile. Random Forest provided a comparatively precision-oriented signal, while CatBoost and XGBoost exhibited substantially stronger fraud Recall. LightGBM and the Neural Network contributed additional probability estimates derived from different

learning structures.

These observations motivated the transition from standalone model evaluation to Stacking Ensemble Learning. Rather than selecting one candidate and discarding the predictive information produced by the others, the next experimental stage investigated whether their continuous fraud-probability estimates could be combined by a second-level model.

Following the exclusion of Isolation Forest, the five supervised models carried forward were Random Forest, LightGBM, CatBoost, XGBoost, and the Neural Network. Their probability outputs subsequently formed the basis of the project’s Out-of-Fold Stacking architecture.

4.4 Out-of-Fold Stacking Ensemble

Following the systematic evaluation of the candidate models, FraudGuard progressed from standalone classification to a Stacking Ensemble designed to combine the predictive information produced by multiple base learners [?]. The initial Stacking architecture incorporated the five supervised candidate models retained from the previous evaluation stage: Random Forest, LightGBM, CatBoost, XGBoost, and the Neural Network.

Rather than combining the binary classifications produced by these models, the ensemble used their continuous fraud-probability estimates. Each base model therefore contributed an estimate of the probability that a transaction belonged to the positive fraud class. These probability estimates were subsequently used as inputs to a second-level Logistic Regression meta-model, which learned how to combine the predictive signals generated by the five base learners.

A central requirement of the Stacking procedure was that the meta-model should not be trained using base-model predictions obtained from observations that those same base models had already seen during training. FraudGuard therefore implemented an Out-of-Fold (OOF) prediction procedure, in which the training dataset was partitioned into folds and predictions for each training observation were generated only by models trained without that observation.

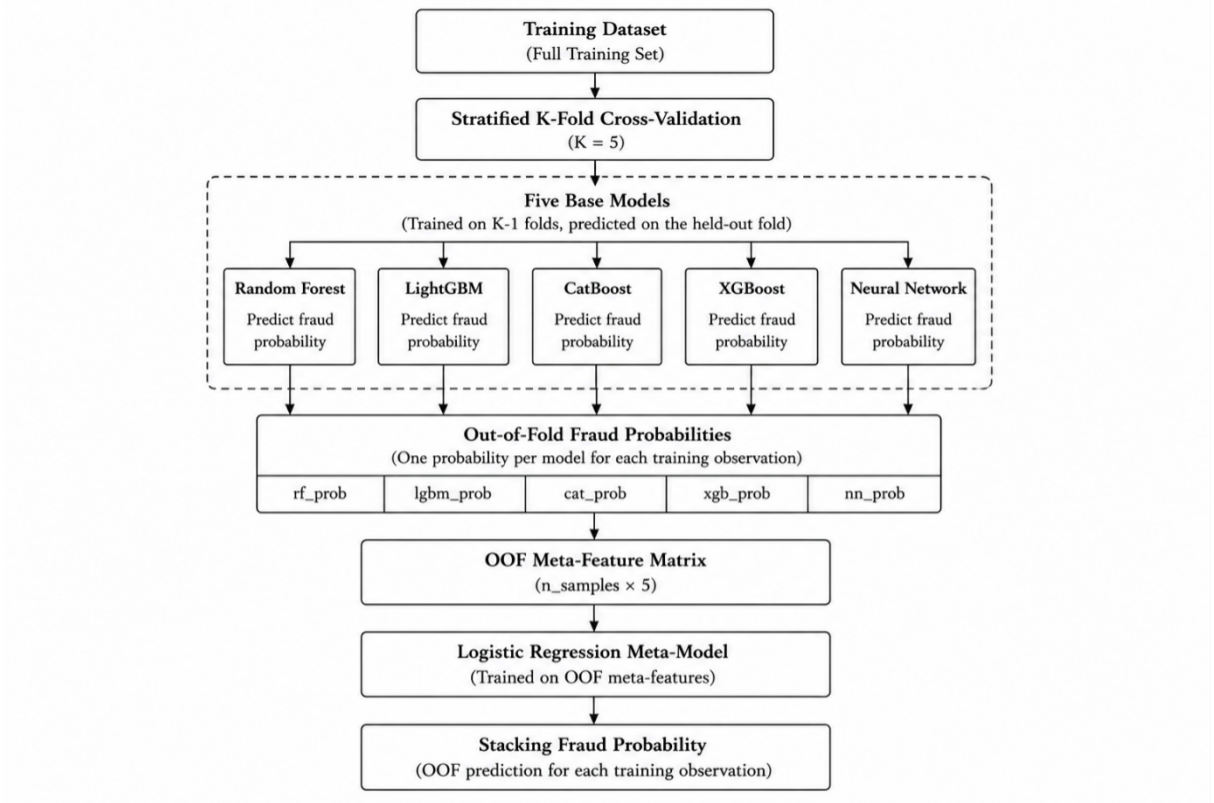


Figure 20: Out-of-Fold Stacking architecture used to construct the initial five-model FraudGuard ensemble.

4.4.1 Out-of-Fold Prediction Generation

The OOF procedure was implemented using Stratified 5-Fold cross-validation ($N_SPLITS = 5$). Stratification was used because of the substantial class imbalance in the fraud dataset, allowing the fraud-class distribution to be preserved across the folds. The folds were shuffled using a fixed random state of 42 to provide reproducible partitioning.

For each fold, the corresponding training portion was used to fit the preprocessing operations and train all five base models, while the held-out portion served as the validation subset. Each trained model then generated a continuous fraud-probability estimate for every transaction in the validation fold. These predictions were stored in the positions corresponding to the original observations.

The process can be represented as:

$$\hat{p}_i^{(m)} = f_{-k}^{(m)}(x_i), \quad i \in V_k, \quad m = 1, \dots, 5$$

where V_k denotes the validation observations belonging to fold k , $f_{-k}^{(m)}$ represents base model m trained on all training folds except fold k , and $\hat{p}_i^{(m)}$ is the resulting OOF fraud-probability estimate for transaction i .

The essential property of this procedure is that transaction is not included in the training data used to generate its corresponding OOF probability. Consequently, after all five folds have been processed, every training transaction receives one probability estimate

from each base-model family without being evaluated by a fold-specific model that was trained on that same transaction.

Within every fold, preprocessing was also fitted exclusively on the fold’s training subset before being applied to its validation subset. Categorical features were transformed using an OrdinalEncoder, configured to assign previously unseen categories the value , while missing values were handled using median values obtained from the training fold. This fold-specific preprocessing strategy maintained the separation between the training and validation portions during OOF generation.

The Neural Network required an additional preprocessing stage. A StandardScaler was fitted using the preprocessed training fold and then applied to the corresponding validation fold before Neural Network training and prediction. Class weights for the Neural Network were likewise calculated from the training labels of the corresponding fold.

4.4.2 Construction of the OOF Meta-Feature Dataset

Five OOF probability arrays were constructed, one for each supervised base model. After completion of the five folds, these arrays contained one fraud-probability estimate for every transaction in the training dataset:

$$X_{\text{meta}} = \begin{bmatrix} p_{rf}^{(1)} & p_{lgbm}^{(1)} & p_{cat}^{(1)} & p_{xgb}^{(1)} & p_{nn}^{(1)} \\ p_{rf}^{(2)} & p_{lgbm}^{(2)} & p_{cat}^{(2)} & p_{xgb}^{(2)} & p_{nn}^{(2)} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ p_{rf}^{(n)} & p_{lgbm}^{(n)} & p_{cat}^{(n)} & p_{xgb}^{(n)} & p_{nn}^{(n)} \end{bmatrix}$$

Figure 21: The five columns of X_{meta} correspond directly to the OOF probability outputs generated by the five supervised base models, as summarized in Table X.

Meta-Feature	Source Model
rf_prob	Random Forest
lgbm_prob	LightGBM
cat_prob	CatBoost
xgb_prob	XGBoost
nn_prob	Neural Network

Table 17: Out-of-Fold probability features used to construct the five-model Stacking meta-dataset.

The corresponding fraud label was retained as the target variable for training the second-level classifier. The resulting meta-dataset therefore contained five predictive inputs, representing the OOF probability estimates of the five base models, together with the original binary fraud label.

Importantly, the original transaction features were not provided directly to the Logistic Regression meta-model. At this stage, the meta-model operated exclusively on the probability outputs generated by the base learners. Its task was therefore not to relearn the original transaction-to-fraud relationship, but to learn how the five model-level signals should be combined.

Before meta-model training, the implementation explicitly verified that the OOF dataset had the same number of rows as the training labels and contained no missing OOF predictions. This ensured that every training observation had received a complete five-dimensional probability representation.

4.4.3 Logistic Regression Meta-Model

The second level of the Stacking architecture was implemented using Logistic Regression. The meta-model received the five OOF probability features and learned a weighted combination of these signals for predicting the final fraud class.

The meta-model was configured with `class_weight='balanced'`, `max_iter=1000`, and `random_state=42`. Balanced class weighting was retained at this level to account for the substantial imbalance between legitimate and fraudulent transactions in the meta-training dataset.

Parameter	Value
Meta-model	Logistic Regression
Input features	5 OOF probability features
class_weight	balanced
max_iter	1000
random_state	42

Table 18: Logistic Regression meta-model configuration used in the five-model OOF Stacking architecture.

insert:

$$\hat{P}(y = 1 \mid \mathbf{p}) = \sigma(\beta_0 + \beta_{rf}p_{rf} + \beta_{lgbm}p_{lgbm} + \beta_{cat}p_{cat} + \beta_{xgb}p_{xgb} + \beta_{nn}p_{nn})$$

with:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where p_{rf} , p_{lgbm} , p_{cat} , p_{xgb} , and p_{nn} represent the fraud-probability estimates generated by the five base models, β_0 is the intercept, and the remaining β terms are the coefficients learned by the Logistic Regression meta-model.

This formulation allows the Stacking layer to learn different contributions from the base models rather than applying a fixed majority vote or simple probability average. A larger positive learned coefficient indicates that increasing the corresponding model probability

contributes more strongly toward the meta-model’s fraud prediction, although coefficient magnitude should be interpreted within the context of the complete Logistic Regression model rather than as a standalone measure of model quality.

The implementation additionally extracted and stored the learned Logistic Regression coefficients for the five meta-features. This provided a direct mechanism for examining how the second-level classifier combined the individual probability signals.

Meta-Feature	Coefficient	Absolute Coefficient
cat_prob	5.053871	5.053871
lgbm_prob	4.546487	4.546487
xgb_prob	1.774920	1.774920
nn_prob	0.755679	0.755679
rf_prob	−0.298839	0.298839

Table 19: Logistic Regression coefficients associated with the five OOF probability meta-features.

The learned coefficients indicate that the five probability signals contributed differently to the Stacking decision function. CatBoost (cat_prob) received the largest positive coefficient (5.0539), followed by LightGBM (lgbm_prob) at 4.5465 and XGBoost (xgb_prob) at 1.7749. The Neural Network probability received a smaller positive coefficient of 0.7557, while Random Forest received a slightly negative coefficient of −0.2988. These coefficients describe how the Logistic Regression meta-model combined the five probability inputs jointly and should not be interpreted as standalone measures of the predictive quality of the individual base models.

4.4.4 OOF Evaluation of the Stacking Layer

Following meta-model training, the Logistic Regression model was first evaluated using the OOF meta-feature dataset. This intermediate evaluation examined whether the second-level model had learned useful relationships among the probability outputs generated by the five base-model families.

The OOF evaluation used the common decision threshold of 0.50 and the same fraud-oriented metrics applied throughout the systematic model-development stage.

However, this OOF evaluation was not treated as the final independent test result. Its purpose was to evaluate the behavior of the Stacking layer on the OOF-derived meta-training representation before proceeding to the completely separate test dataset.

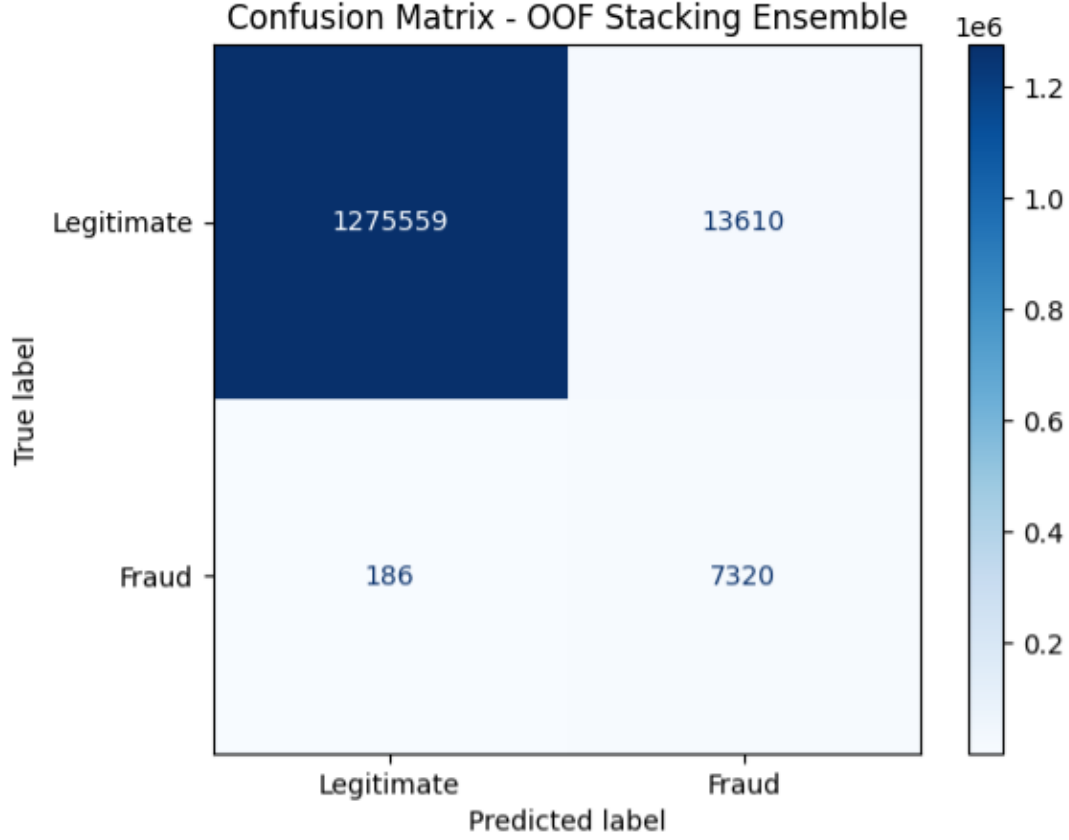


Figure 22: Confusion Matrix of the five-model Stacking meta-model evaluated on the Out-of-Fold meta-feature dataset at a threshold of 0.50.

4.4.5 Final Base-Model Retraining and Independent Test Inference

After OOF generation and training of the Logistic Regression meta-model, the five base models were retrained using the full training dataset. This step produced the final base-model instances used to generate probability estimates for the separate test dataset and subsequently for the saved production architecture.

The final preprocessing artifacts were likewise fitted using the complete training dataset and then applied to the separate test dataset. For the Neural Network, a final Standard-Scaler was fitted on the fully preprocessed training features before transforming the test features.

Each final base model generated a fraud probability for every transaction in the separate test dataset:

$$\mathbf{z}_{\text{test}} = [p_{rf}, p_{lgbm}, p_{cat}, p_{xgb}, p_{nn}]$$

These five probabilities formed the test-time meta-feature vector and were passed to the previously trained Logistic Regression meta-model:

$$\hat{P}_{\text{stack}}(y = 1) = f_{\text{meta}}(\mathbf{z}_{\text{test}})$$

This distinction between OOF meta-training and final test inference is fundamental to the Stacking procedure. During meta-model training, the five inputs associated with each training observation were generated through OOF prediction, ensuring that the corresponding base models had not been trained on that observation. For final test inference, the five probability inputs were instead generated by base models retrained on the complete training dataset. The separate test dataset was therefore not used to construct the OOF meta-training representation or to train the Logistic Regression meta-model.

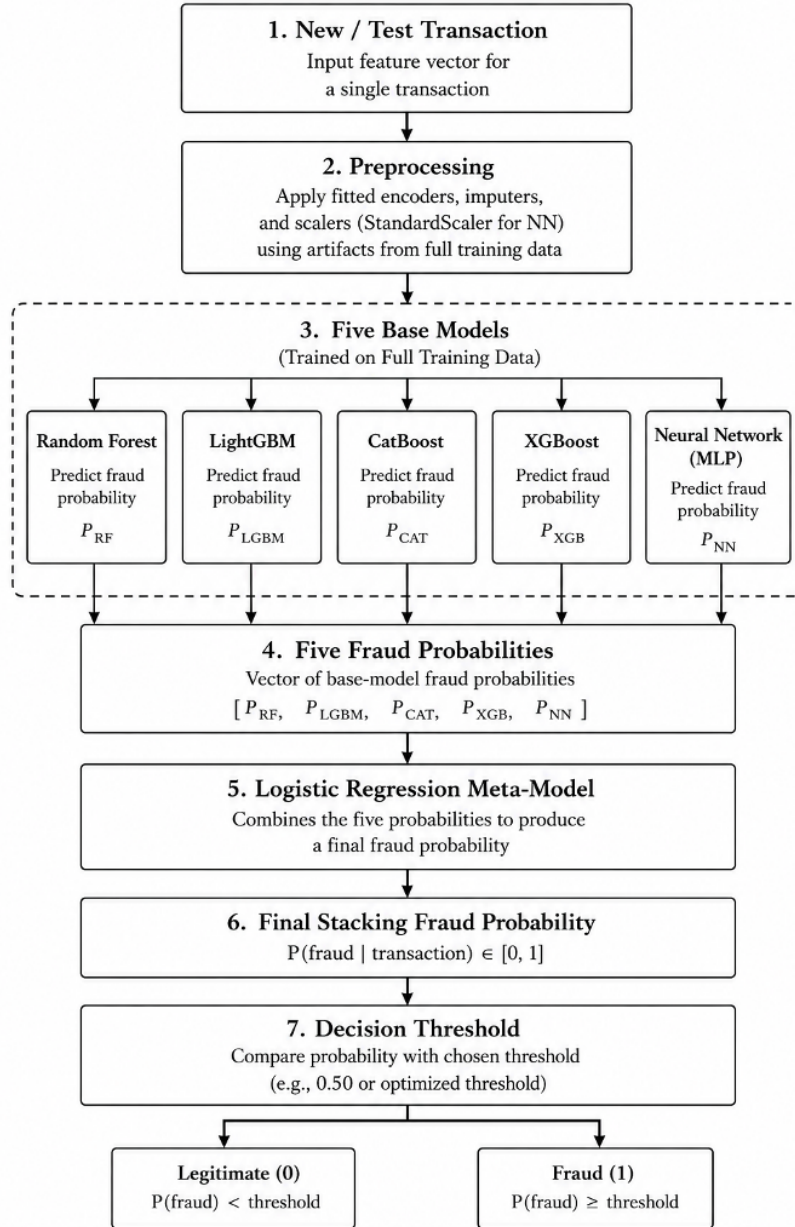


Figure 23: Final inference flow of the five-model FraudGuard Stacking Ensemble on the separate test dataset.

4.4.6 Five-Model Stacking Performance

The completed five-model Stacking architecture was evaluated on the separate test dataset using the fraud probabilities generated by the Logistic Regression meta-model. An initial evaluation was performed using the common decision threshold of 0.50, maintaining consistency with the preceding systematic candidate-model evaluation.

Performance was assessed using Precision, Recall, F1-Score, Balanced Accuracy, ROC-AUC, PR-AUC, and Confusion Matrix analysis. This evaluation established the baseline test-set operating behavior of the five-model Stacking architecture at the default threshold, providing a reference point for the subsequent OOF-based decision-threshold optimization.

Metric	Five-Model Stacking Ensemble
Precision	0.2740
Recall	0.9534
F1-Score	0.4257
Balanced Accuracy	0.9718
ROC-AUC	0.9971
PR-AUC	0.8626

Table 20: Performance of the five-model OOF Stacking Ensemble on the separate test dataset at a decision threshold of 0.50.

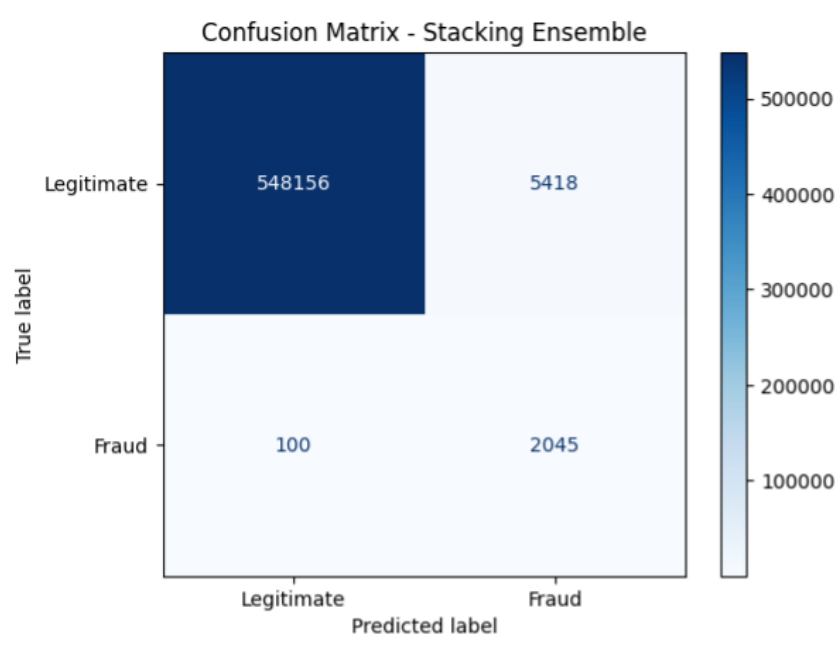


Figure 24: Confusion Matrix of the five-model OOF Stacking Ensemble on the separate test dataset at a decision threshold of 0.50.

At the default decision threshold of 0.50, the five-model Stacking Ensemble exhibited a strongly Recall-oriented operating profile. The ensemble correctly identified 2,045 of the

2,145 fraudulent transactions, corresponding to a fraud Recall of 0.9534, while only 100 fraudulent transactions were missed. However, this high fraud sensitivity was accompanied by 5,418 False Positive classifications, resulting in a substantially lower Precision of 0.2740. The resulting F1-Score was 0.4257, while Balanced Accuracy reached 0.9718.

The model nevertheless demonstrated strong threshold-independent ranking performance, achieving a ROC-AUC of 0.9971 and a PR-AUC of 0.8626. These results indicate that the ensemble was highly effective at separating fraudulent from legitimate transactions overall, while its classification behavior at the default 0.50 threshold remained heavily oriented toward maximizing fraud detection rather than minimizing False Positives. This observation motivated the subsequent evaluation of an alternative decision threshold using the OOF predictions.

4.4.7 OOF-Based Decision-Threshold Optimization

Rather than selecting a production operating threshold directly from the independent test set, FraudGuard performed threshold optimization using the predictions available from the OOF meta-training stage.

Candidate thresholds were evaluated according to F1-Score, and the threshold producing the highest OOF F1-Score was selected as the optimized operating point.

Formally:

$$t^* = \arg \max_t F_1(y_{\text{OOF}}, \hat{y}_t)$$

where t^* represents the selected OOF-derived threshold, y_{OOF} denotes the true training labels associated with the OOF predictions, and $\hat{y}_t$ represents the binary Stacking predictions obtained by applying candidate threshold t to the OOF Stacking probabilities.

The optimization procedure selected an OOF-derived decision threshold of $t^* = 0.999067$, at which the maximum OOF F1-Score reached 0.8872.

The selected OOF-derived threshold was subsequently applied to the Stacking probabilities generated for the independent test dataset. This separated threshold selection from final test evaluation, avoiding direct optimization of the operating threshold against the test labels.

Test-set outcome	Threshold 0.50	OOF-optimized threshold (0.999)
True Negatives	548,156	553,394
False Positives	5,418	180
False Negatives	100	507
True Positives	2,045	1,638

Table 21: Comparison of five-model Stacking classification outcomes at the default 0.50 threshold and the OOF-optimized decision threshold on the separate test dataset.

Application of the OOF-optimized threshold (0.999) substantially altered the operating behavior of the Stacking Ensemble. Compared with the default threshold of 0.50, the

number of False Positives decreased from 5,418 to 180, while True Negatives increased from 548,156 to 553,394. This reduction in false alarms was accompanied by a decrease in fraud detection: True Positives declined from 2,045 to 1,638, while False Negatives increased from 100 to 507. The threshold optimization therefore shifted the ensemble from the highly Recall-oriented behavior observed at the default threshold toward an operating point with substantially stronger False Positive control.

4.4.8 Final Evaluation of the Five-Model Stacking Ensemble

Following construction of the OOF meta-training dataset, training of the Logistic Regression meta-model, retraining of the five base models on the complete training dataset, and selection of the decision threshold from the OOF predictions, the five-model Stacking Ensemble was evaluated on the separate test dataset. This evaluation represents the final performance of the initial Stacking architecture before the subsequent model-reduction stage.

At the OOF-derived decision threshold of **0.999067**, the ensemble achieved a Precision of 0.9010, Recall of 0.7636, and F1-Score of 0.8266. The corresponding Balanced Accuracy was 0.8817, while the threshold-independent ROC-AUC and PR-AUC were 0.9971 and 0.8626, respectively.

Metric	Five-Model Stacking Ensemble
Precision	0.9010
Recall	0.7636
F1-Score	0.8266
Balanced Accuracy	0.8817
ROC-AUC	0.9971
PR-AUC	0.8626

Table 22: Final performance of the five-model Stacking Ensemble on the separate test dataset using the OOF-derived decision threshold.

The final operating point reflects the trade-off introduced by OOF-based threshold selection. Compared with the default threshold of 0.50, the optimized threshold reduced the number of False Positives from 5,418 to 180 and increased Precision from 0.2740 to 0.9010. This stronger False Positive control was accompanied by a reduction in Recall from 0.9534 to 0.7636, with False Negatives increasing from 100 to 507. Nevertheless, because the threshold was selected to maximize OOF F1-Score, the resulting test-set F1-Score increased from 0.4257 to 0.8266.

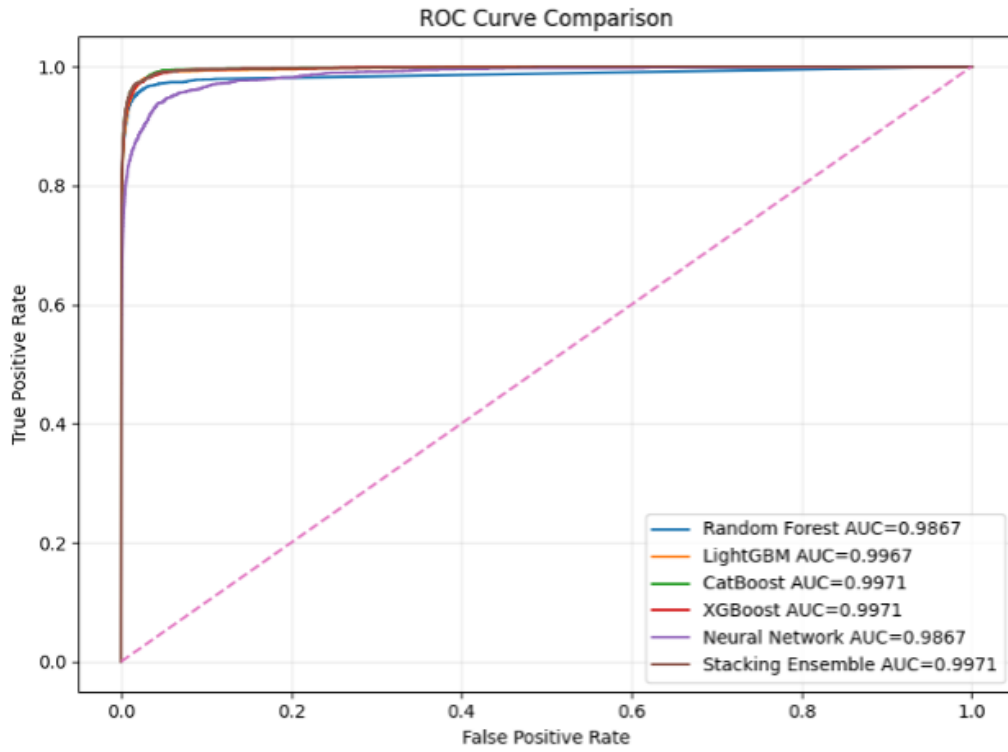


Figure 25: ROC curves of the five supervised base models and the five-model Stacking Ensemble on the separate test dataset.

The ROC analysis shows that the Stacking Ensemble retained strong overall discriminative capability, achieving a ROC-AUC of 0.9971. This value was essentially comparable to the strongest tree-based candidates, with CatBoost and XGBoost also reaching approximately 0.9971 and LightGBM reaching 0.9967. Random Forest and the Neural Network produced lower ROC-AUC values of approximately 0.9867. The Stacking procedure therefore preserved the high ranking capability of the strongest base learners, although it did not produce a substantial ROC-AUC improvement over the best individual models.

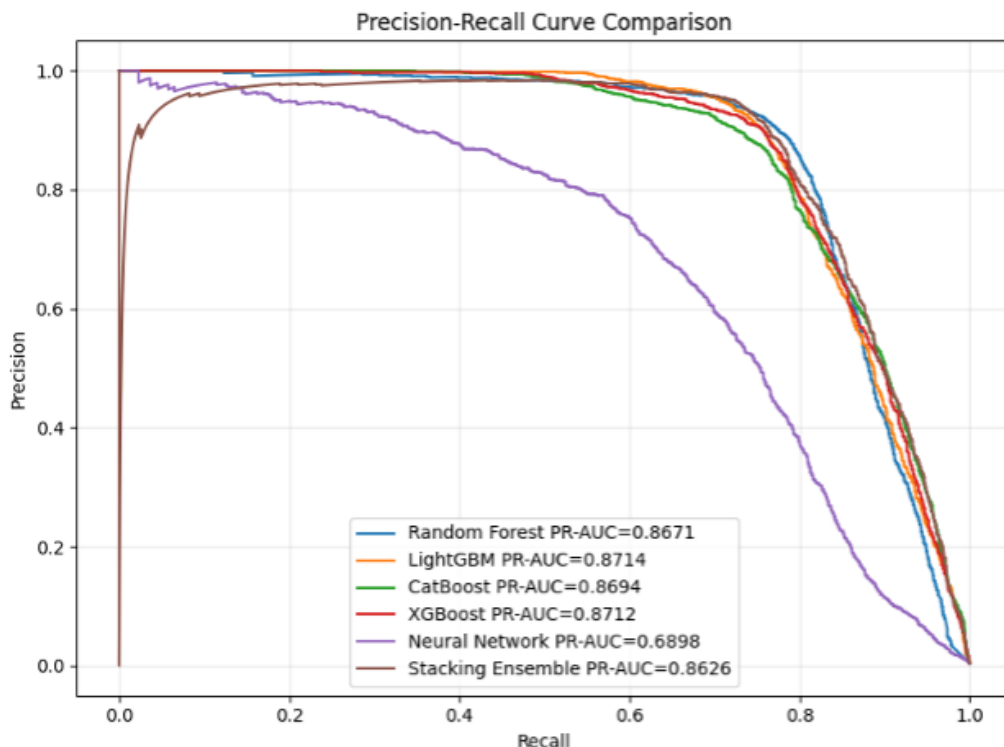


Figure 26: Precision-Recall curves of the five supervised base models and the five-model Stacking Ensemble on the separate test dataset.

The Precision-Recall analysis provides a more fraud-focused comparison under the substantial class imbalance of the dataset. The Stacking Ensemble achieved a PR-AUC of 0.8626, compared with 0.8714 for LightGBM, 0.8712 for XGBoost, 0.8694 for CatBoost, 0.8671 for Random Forest, and 0.6898 for the Neural Network. The ensemble therefore did not exceed the strongest individual tree-based models in threshold-independent PR-AUC. Its principal contribution at this stage was therefore not an improvement in threshold-independent PR-AUC, but the construction of a leakage-controlled second-level combination of the five probability signals, together with a framework for subsequent operating-threshold selection.

The OOF-derived threshold of 0.999067 was specific to the F1-maximization experiment described in this section and was not adopted as the final production threshold. The subsequent model-reduction study introduced a different operating criterion, requiring fraud Recall of at least 0.95 while prioritizing the reduction of False Positives. Consequently, the thresholds reported in Section 4.5 were selected under this new business-oriented criterion and are not directly equivalent to the F1-optimized threshold evaluated here.

Overall, the five-model OOF Stacking experiment demonstrated that heterogeneous base-model probability outputs could be combined within a leakage-controlled second-level learning framework. However, the comparative results also raised an additional architectural question: whether all five base models were necessary to preserve the desired fraud-detection behavior. The next stage therefore investigated model reduction, examining whether a smaller subset of complementary base learners could provide comparable or improved operational performance while reducing ensemble complexity.

4.5 Meta-Feature Combination Study and Ensemble Reduction

Following evaluation of the initial five-model Stacking architecture, an additional model-reduction experiment was conducted to determine whether all five base-model probability signals were required to achieve the desired fraud-detection behavior. The objective was not simply to identify the combination with the highest aggregate classification score, but to investigate whether a smaller ensemble could preserve high fraud Recall while reducing the number of False Positive classifications. The experiment operated directly on the previously generated meta-feature representations. The five available probability features were `rf_prob`, `lgbm_prob`, `cat_prob`, `xgb_prob`, and `nn_prob`, corresponding to Random Forest, LightGBM, CatBoost, XGBoost, and the Neural Network, respectively. The saved OOF probabilities were used for meta-model training, while the corresponding saved test-set probabilities were used for evaluation. Consequently, the base models themselves did not need to be retrained during this combination study; each candidate combination required only the training of a Logistic Regression meta-model on the selected probability features. All non-empty subsets of the five available meta-features were evaluated. For five candidate features, the number of possible non-empty combinations is:

$$\sum_{k=1}^5 \binom{5}{k} = 2^5 - 1 = 31$$

The experiment therefore examined 31 candidate configurations, ranging from individual probability features to the complete five-feature representation.

Experimental Component	Configuration
Available meta-features	<code>rf_prob</code> , <code>lgbm_prob</code> , <code>cat_prob</code> , <code>xgb_prob</code> , <code>nn_prob</code>
Candidate combinations	31
Meta-model	Logistic Regression
Minimum Recall requirement	0.95
Candidate threshold range	0.30–0.89
Threshold increment	0.01
Random state	42

Table 23: Experimental configuration of the FraudGuard meta-feature combination study.

4.5.1 Business-Oriented Threshold Selection

Unlike the preceding OOF threshold experiment, which selected the decision threshold according to maximum F1-Score, the model-reduction study introduced an explicit fraud-detection constraint. For each candidate meta-feature combination, decision thresholds were evaluated from 0.30 to 0.89 in increments of 0.01, and the operating point was selected subject to a minimum fraud Recall requirement of:

$$\text{Recall} \geq 0.95$$

Among configurations satisfying this constraint, the comparison prioritized fewer False Positives, followed by higher Precision and lower model complexity. This criterion reflected the operational objective of maintaining detection of at least approximately 95% of fraudulent transactions while reducing the number of legitimate transactions incorrectly flagged as fraud.

From a business perspective, this criterion explicitly balances the two principal classification errors in fraud detection: False Alarms and Missed Fraud. False Alarms correspond to False Positive classifications, in which legitimate transactions are incorrectly flagged as fraudulent, whereas Missed Fraud corresponds to False Negative classifications, in which fraudulent transactions remain undetected. The minimum Recall constraint limits the acceptable level of Missed Fraud, while the subsequent minimization of False Positives reduces the operational burden associated with False Alarms.

The resulting comparison therefore addressed a different question from conventional model ranking: rather than asking which configuration maximized a single statistical metric, it examined which combination achieved the required fraud sensitivity with the lowest False Positive burden.

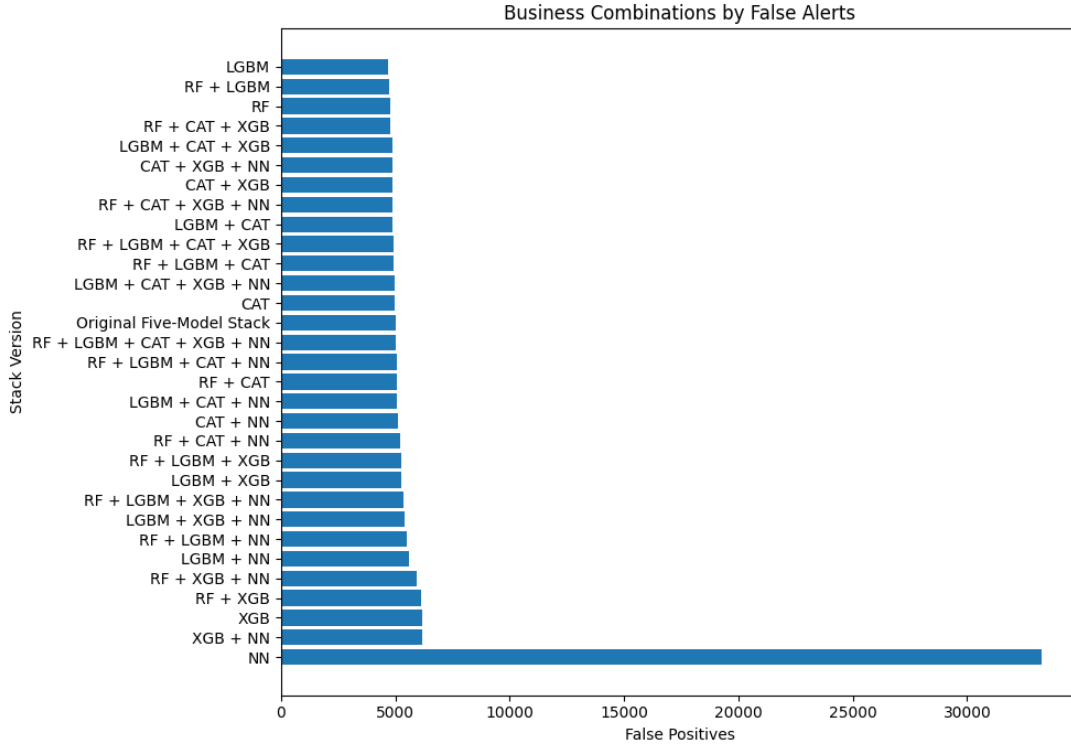


Figure 27: False Positive counts of the evaluated meta-feature combinations under the minimum fraud Recall requirement.

The combination study revealed that the number of False Positives varied across candidate meta-feature subsets even when the required Recall level was maintained. This demonstrated that additional model inputs did not automatically produce a more favor-

able operating point and motivated explicit comparison of predictive performance against ensemble complexity.

4.5.2 Precision-Recall Characteristics of Candidate Combinations

The candidate configurations were also examined in terms of their resulting Precision and Recall at their selected business-oriented thresholds. Because the threshold-selection procedure imposed a minimum Recall requirement, eligible configurations were concentrated around the required fraud-sensitivity level, while their Precision values varied according to the number of False Positives generated.

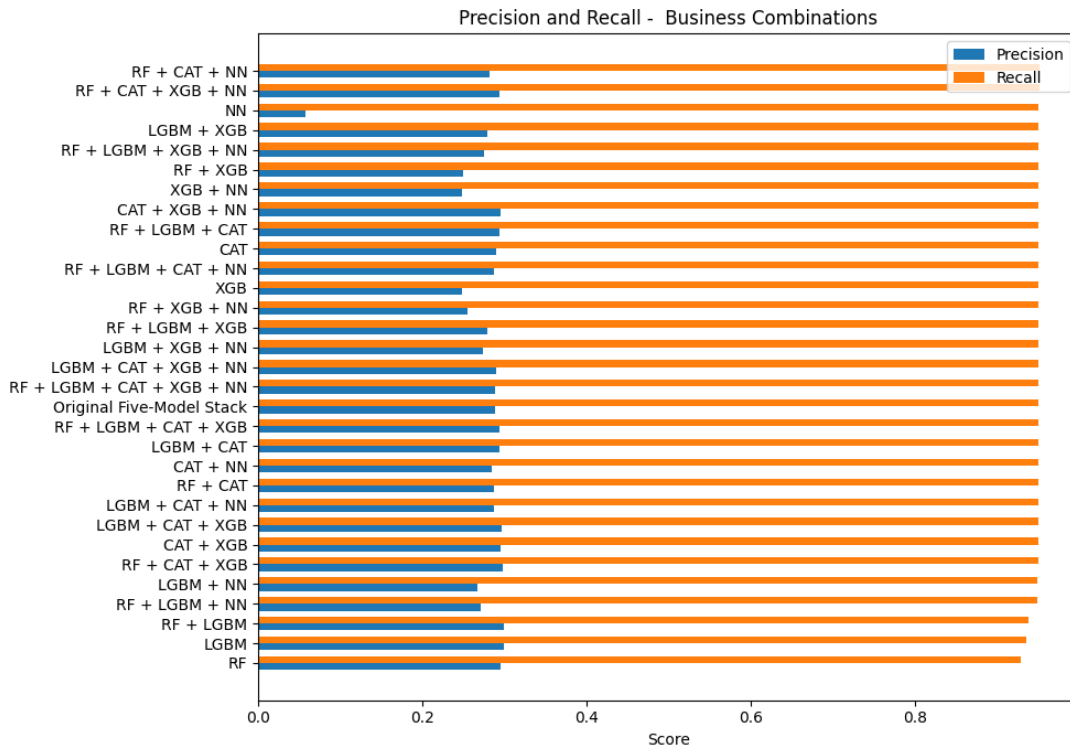


Figure 28: Precision and Recall achieved by the evaluated meta-feature combinations at their business-oriented decision thresholds.

The comparison illustrates the effect of the minimum-Recall constraint on the candidate operating points. While eligible configurations maintained Recall at or above approximately 0.95, their Precision differed because they generated different numbers of False Positive classifications. The selection problem therefore became primarily one of identifying which combination could preserve the required fraud coverage while most effectively limiting false alerts.

4.5.3 Ensemble Complexity Analysis

In addition to predictive behavior, the experiment examined whether ensemble size was associated with False Positive performance. Candidate configurations were grouped according to the number of probability features included in the corresponding stack.

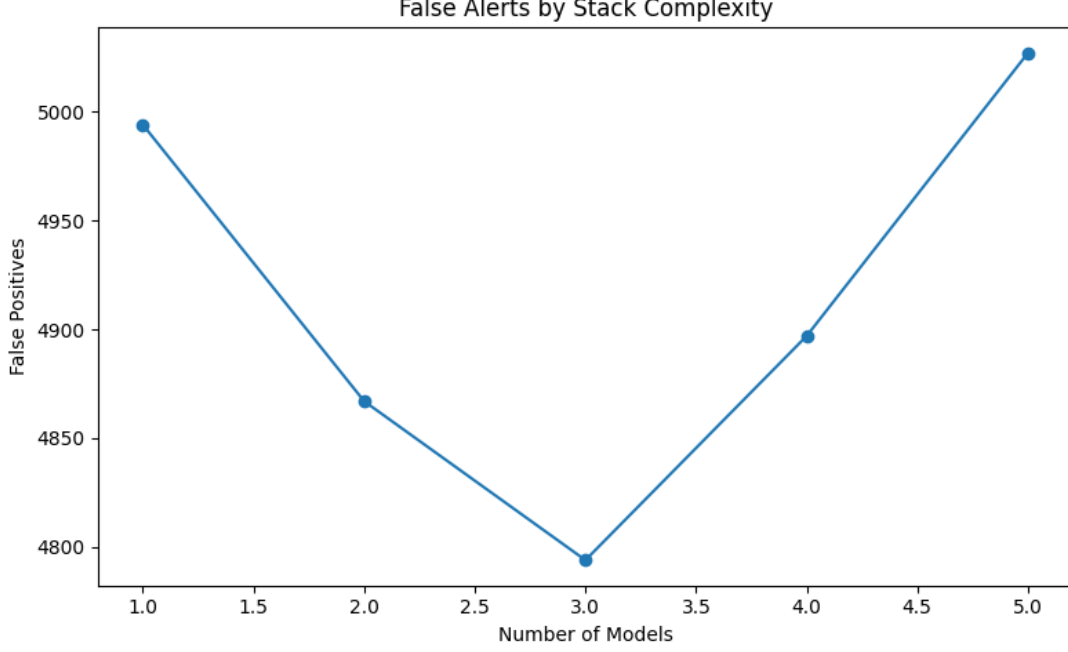


Figure 29: False Positive behavior of candidate meta-feature combinations according to ensemble size.

The complexity analysis indicates that increasing the number of base-model signals did not produce a monotonic reduction in False Positives. Within the evaluated configurations, the lowest False Positive level was associated with a three-model combination, whereas the aggregate behavior of four- and five-model configurations did not provide a corresponding reduction. This result supported further investigation of a reduced ensemble rather than retaining all five probability signals solely on the assumption that greater model complexity would improve operational performance.

4.5.4 Selection of the Three-Model Ensemble

Applying the business-oriented selection criterion identified the combination of Random Forest, CatBoost, and XGBoost as the highest-ranked candidate. The corresponding meta-feature representation was:

$$X_{\text{meta}}^{\text{reduced}} = [p_{rf}, p_{cat}, p_{xgb}]$$

For this three-model configuration, the selected decision threshold was 0.69. On the separate test dataset, the resulting operating point achieved Recall of 0.9501, Precision of 0.2983, and an F1-Score of 0.4540. Balanced Accuracy was 0.9707, ROC-AUC was 0.9975, and PR-AUC was 0.8807.

Metric	RF + CAT + XGB
Decision threshold	0.69
Precision	0.2983
Recall	0.9501
F1-Score	0.4540
Balanced Accuracy	0.9707
ROC-AUC	0.9975
PR-AUC	0.8807

Table 24: Performance of the selected Random Forest–CatBoost–XGBoost meta-feature combination at the business-oriented decision threshold.

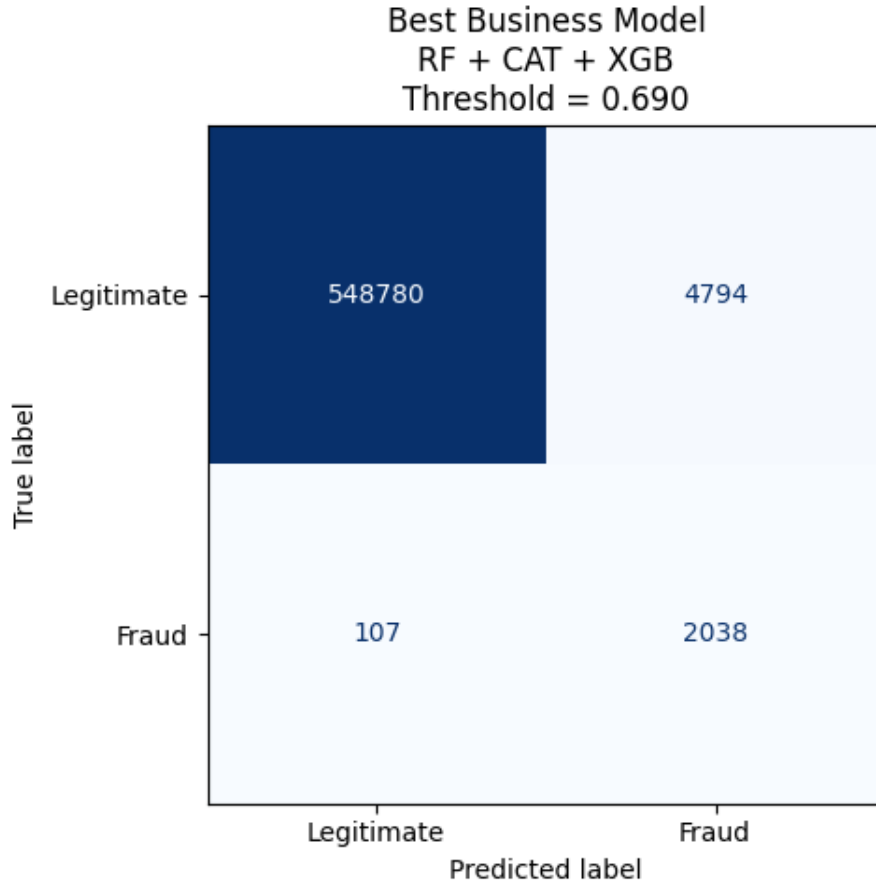


Figure 30: Confusion Matrix of the selected Random Forest–CatBoost–XGBoost ensemble on the separate test dataset at a decision threshold of 0.69.

At this operating point, the reduced ensemble correctly identified 2,038 of the 2,145 fraudulent transactions, while 107 fraud cases remained undetected. Among legitimate transactions, 4,794 were incorrectly classified as fraud, while 548,780 were correctly classified as legitimate. The resulting configuration therefore satisfied the predefined minimum Recall requirement while producing the lowest False Positive count according to the business-ranking procedure used in the combination study.

4.5.5 Analysis of the Selected Three-Model Ensemble

After selection of the Random Forest–CatBoost–XGBoost combination, the fitted Logistic Regression meta-model was examined to characterize how the three retained probability signals contributed to the final Stacking prediction. The learned parameters of the selected meta-model were an intercept of **-4.217189**, with coefficients of **1.724078** for **Random Forest**, **5.509192** for **CatBoost**, and **5.138019** for **XGBoost**.

Accordingly, the fitted decision function for the selected ensemble can be expressed as:

$$z = -4.217189 + 1.724078 p_{RF} + 5.509192 p_{CAT} + 5.138019 p_{XGB}$$

where p_{RF} , p_{CAT} , and p_{XGB} represent the fraud-probability outputs of Random Forest, CatBoost, and XGBoost, respectively. The resulting value is transformed by the Logistic Regression sigmoid function to obtain the final Stacking fraud probability. Transactions with a final probability greater than or equal to the selected production decision threshold of 0.69 are classified as fraudulent.

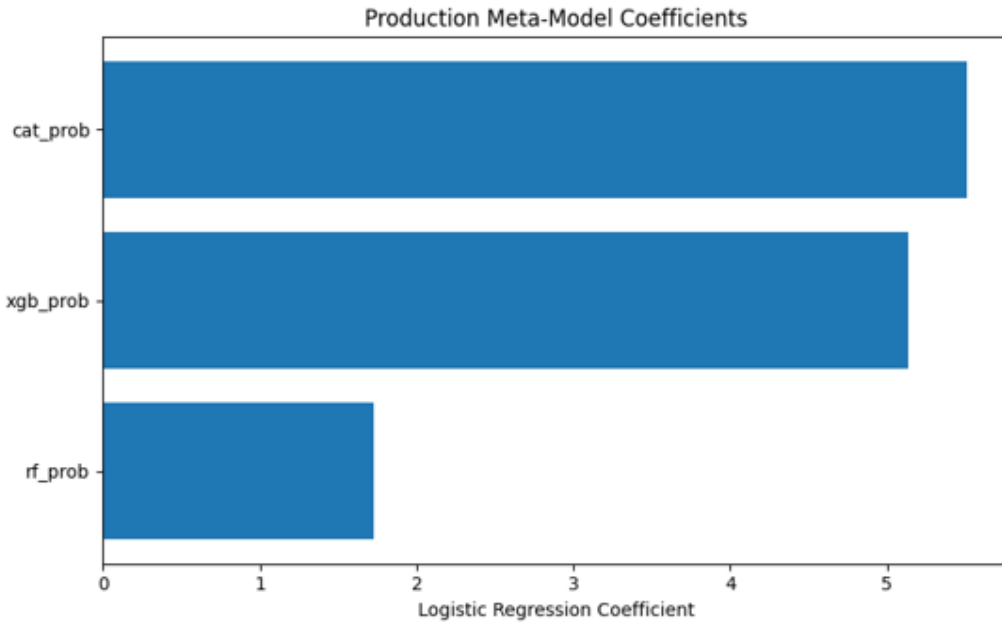


Figure 31: Logistic Regression coefficients learned for the three probability meta-features of the selected Random Forest–CatBoost–XGBoost ensemble.

All three retained probability features received positive coefficients. **CatBoost received the largest coefficient (5.509192), closely followed by XGBoost (5.138019), while Random Forest received a smaller positive coefficient (1.724078).** This indicates that, within the fitted Logistic Regression meta-model, increases in the CatBoost and XGBoost fraud probabilities produced larger changes in the meta-model’s log-odds than an equivalent increase in the Random Forest probability. Random Forest nevertheless retained a positive coefficient, indicating that its probability signal continued to contribute to the combined decision. The unequal coefficient magnitudes demonstrate

that the meta-model did not treat the retained base-model probabilities as equally informative signals, but instead learned a data-driven weighted combination of their outputs. These coefficients characterize the joint decision function of the fitted meta-model and should not be interpreted as standalone measures of the predictive quality of the individual base classifiers.

To complement this coefficient-based interpretation, the selected ensemble was also examined geometrically in the three-dimensional meta-feature space formed by the Random Forest, CatBoost, and XGBoost fraud probabilities. This representation provides a transaction-level view of how the three probability signals are jointly positioned before being combined by the Logistic Regression meta-model.

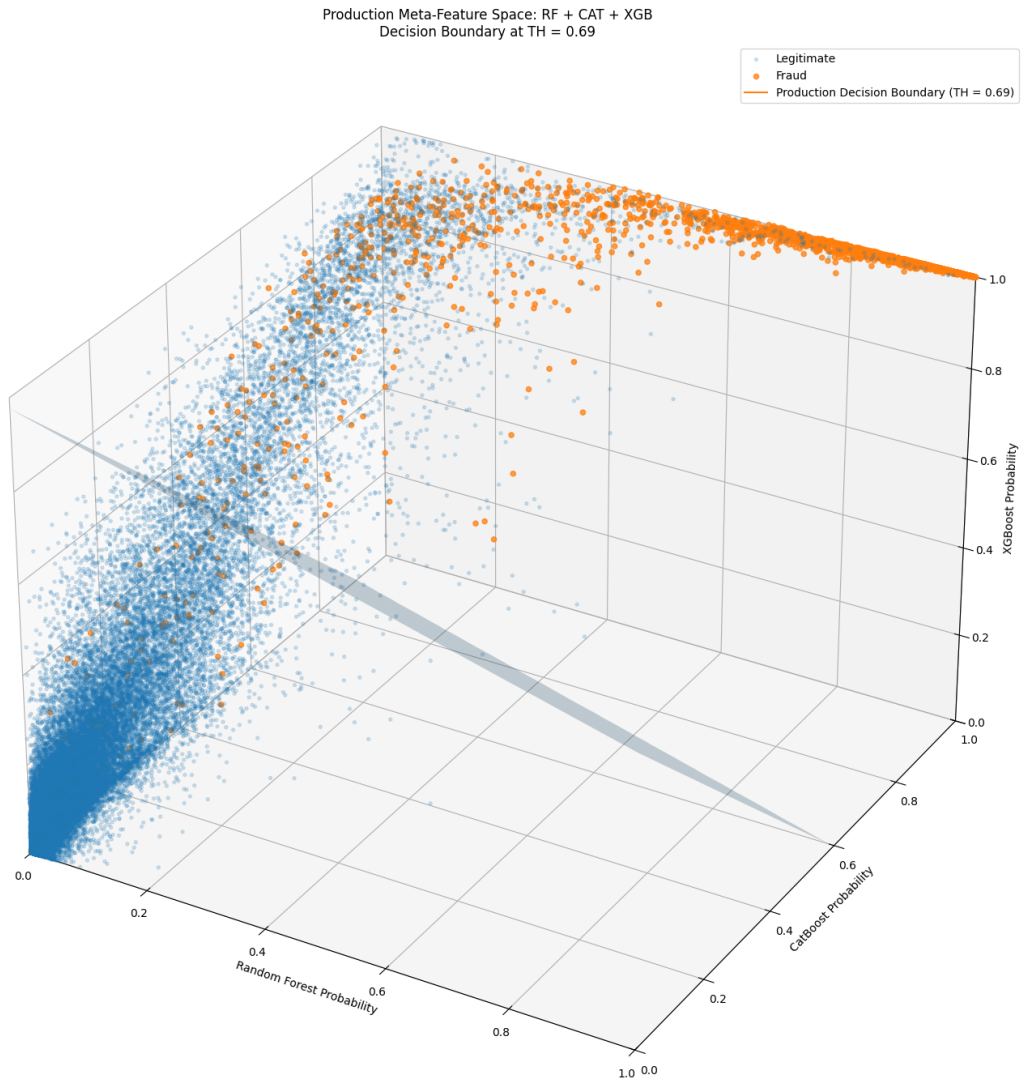


Figure 32: Three-dimensional meta-feature space of the selected Random Forest–CatBoost–XGBoost ensemble, including the Logistic Regression decision boundary associated with the selected threshold of 0.69.

The visualization illustrates how the three base-model probability estimates jointly define the input space of the Logistic Regression meta-model. The final classification is therefore

not determined by the probability generated by any individual classifier in isolation, but by the combination learned across the three-dimensional meta-feature representation.

After combination by the Logistic Regression meta-model, each transaction received a single final Stacking fraud probability. The distribution of these final probabilities was examined to characterize the separation between legitimate and fraudulent transactions and to contextualize the selected operating threshold.

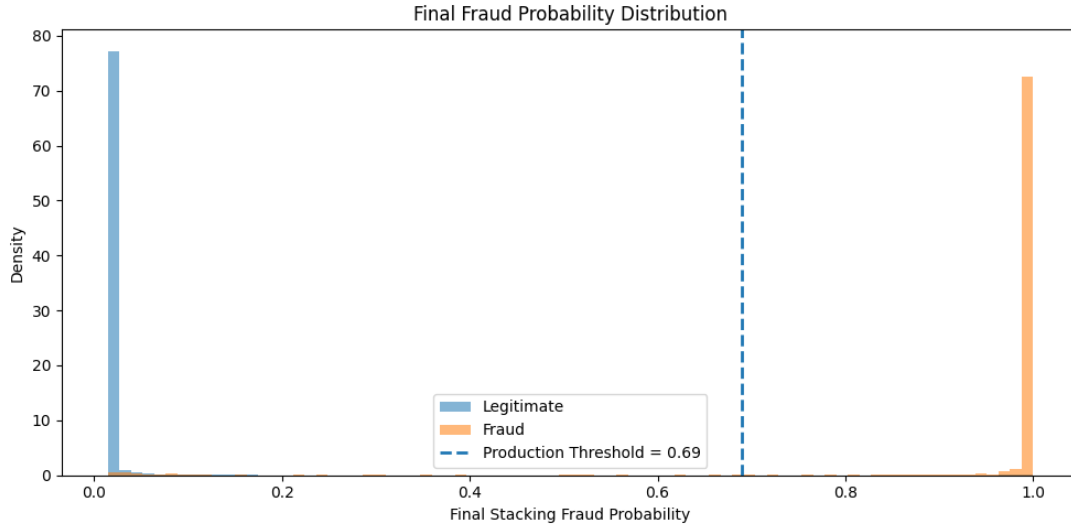


Figure 33: Distribution of final Stacking fraud probabilities for legitimate and fraudulent transactions, with the selected decision threshold of 0.69.

The probability distributions show that legitimate transactions were predominantly concentrated toward lower final fraud probabilities, whereas fraudulent transactions were predominantly concentrated toward the upper end of the probability range. The distributions were not perfectly separated, however, which is consistent with the remaining False Positive and False Negative classifications observed for the selected ensemble.

The behavior of the selected ensemble was then examined across the candidate threshold range to characterize the relationship between Precision and Recall and to identify the operating point satisfying the minimum Recall requirement.

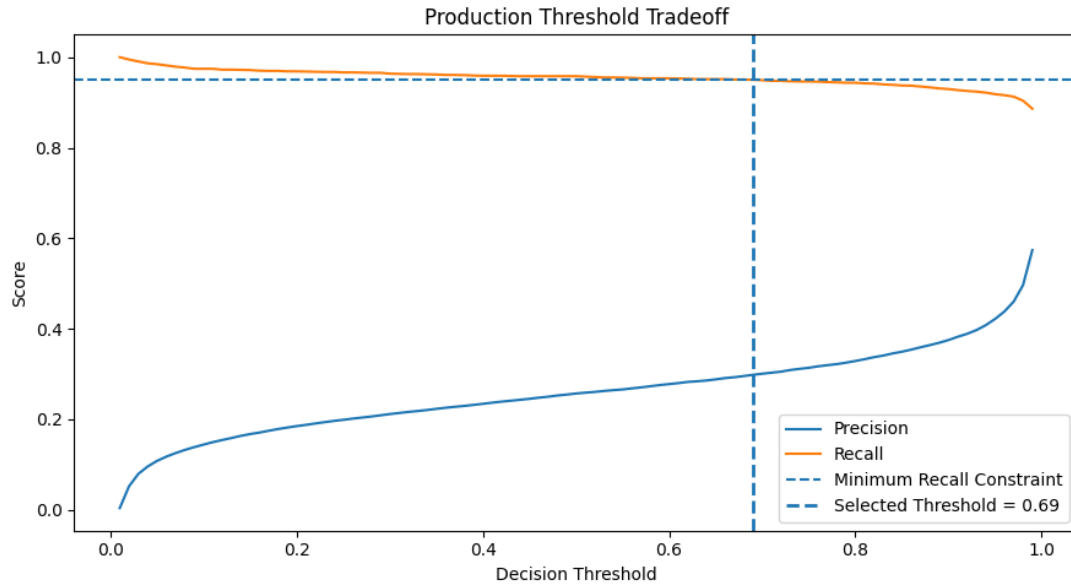


Figure 34: Precision–Recall trade-off of the selected Random Forest–CatBoost–XGBoost ensemble across candidate decision thresholds, including the minimum Recall constraint and selected threshold of 0.69.

As the decision threshold increased, Precision increased while Recall gradually decreased. The selected threshold of 0.69 represented the business-oriented operating point identified by the experiment while maintaining the required Recall of at least 0.95. This analysis illustrates that the selected ensemble architecture and its decision threshold were treated as related but distinct design choices: the meta-feature subset determined which model signals were combined, while threshold selection determined the final operational balance between fraud detection and False Positive control.

4.5.6 Comparison with the Original Five-Model Stack

Metric	Original Five-Model Stack	RF + CAT + XGB
Number of base-model signals	5	3
Threshold	0.56	0.69
Precision	0.2885	0.2983
Recall	0.9501	0.9501
F1-Score	0.4426	0.4540
Balanced Accuracy	0.9705	0.9707
ROC-AUC	0.9971	0.9975
PR-AUC	0.8626	0.8807
False Positives	5,027	4,794
False Negatives	107	107
True Positives	2,038	2,038
True Negatives	548,547	548,780

Table 25: Comparison of the original five-model Stacking architecture and the selected three-model ensemble under the common minimum-Recall threshold criterion.

Under the common business-oriented evaluation criterion, both architectures detected 2,038 fraudulent transactions and missed 107, producing the same Recall of approximately 0.9501. The reduced three-model ensemble therefore preserved the same number of Missed Fraud cases (107 False Negatives) while decreasing False Alarms from 5,027 to 4,794 False Positives, corresponding to 233 fewer legitimate transactions incorrectly flagged as fraud. Precision consequently increased from 0.2885 to 0.2983, while F1-Score increased from 0.4426 to 0.4540. The reduced configuration also used only three probability signals rather than five.

The study additionally identified CatBoost as the smallest candidate capable of satisfying the minimum Recall requirement. At a threshold of 0.73, CatBoost alone achieved Recall of 0.9506 with 4,994 False Positives. Although this demonstrated that a single-model solution could satisfy the Recall constraint, the three-model Random Forest–CatBoost–XGBoost configuration produced fewer False Positives (4,794) while maintaining comparable fraud Recall and was therefore selected by the business-ranking procedure.

4.5.7 Final Model-Reduction Decision

The meta-feature combination study demonstrated that the complete five-model representation was not required to preserve the target fraud-detection behavior under the business-oriented evaluation criterion. The selected Random Forest–CatBoost–XGBoost combination maintained Recall at approximately 0.95, while reducing False Positives relative to the original five-model architecture evaluated under the same threshold-selection rule.

The reduction also removed the LightGBM and Neural Network probability features from the subsequent ensemble architecture, decreasing the number of base-model signals from five to three. Importantly, this decision was based on the empirical combination study rather than on the standalone performance of either excluded model. Their removal therefore reflects the behavior of the probability signals within the evaluated Stacking combinations, rather than a general conclusion that LightGBM or the Neural Network were individually ineffective classifiers.

Based on these results, Random Forest, CatBoost, and XGBoost were carried forward as the selected base-model set for the subsequent FraudGuard architecture.

4.6 Final Production Model Selection and Inference Validation

The preceding experimental stages progressively refined the FraudGuard predictive architecture from individual candidate models to a five-model Out-of-Fold Stacking ensemble and, subsequently, to a reduced production configuration. Based on the systematic meta-feature combination study described in Section 4.5, the final architecture retained **Random Forest, CatBoost, and XGBoost** as the three base classifiers, with their continuous fraud-probability outputs combined by a **Logistic Regression meta-model**.

The final configuration was selected under the business-oriented criterion established during the model-reduction stage, in which candidate configurations were required to maintain fraud Recall of at least **0.95** while minimizing the number of False Positive classifications. The selected Random Forest–CatBoost–XGBoost ensemble operated at a decision threshold of **0.69**, producing **2,038 True Positives, 107 False Negatives, 4,794 False Positives, and 548,780 True Negatives** on the separate test dataset. As demonstrated in Section 4.5, this configuration provided the selected operating balance under the project’s minimum-Recall and False-Positive reduction criteria.

Table 26: Final selected FraudGuard predictive configuration.

Component	Final Selection
Base models	Random Forest, CatBoost, XGBoost
Base-model outputs	rf_prob, cat_prob, xgb_prob
Meta-model	Logistic Regression
Number of meta-features	3
Decision threshold	0.69
Minimum Recall criterion	≥ 0.95
True Positives	2,038
False Negatives / Missed Fraud	107
False Positives / False Alarms	4,794
True Negatives	548,780

The final selection therefore reflects a balance between **fraud detection, False Alarm control, and ensemble complexity**, rather than optimization of a single aggregate

performance metric. Reducing the ensemble from five to three base-model probability signals removed LightGBM and the Neural Network from the final predictive path while preserving the required fraud-detection behavior and reducing the False Positive burden observed in the corresponding five-model comparison.

The selected configuration was subsequently validated through a dedicated **inference pipeline**. Unlike the preceding experimental stages, this stage did not retrain or reselect the predictive models. Instead, the saved preprocessing components, Random Forest, CatBoost, XGBoost, Logistic Regression meta-model, selected meta-feature ordering, and production decision threshold were loaded and applied to transaction data. For each transaction, the three base classifiers generated the fraud-probability estimates `rf_prob`, `cat_prob`, and `xgb_prob`, which were supplied to the saved Logistic Regression meta-model to produce the final Stacking fraud probability. The stored decision threshold was then applied to obtain the final binary classification.

This inference-only stage demonstrated that the final FraudGuard predictive configuration could be reconstructed from its saved artifacts and executed independently of the preceding training, OOF generation, threshold-selection, and model-reduction procedures. The resulting architecture therefore represents the final predictive pipeline carried forward for subsequent system integration and explainability.

4.7 Explainability Layer

Following selection and validation of the final production ensemble, an explainability layer was incorporated to provide transaction-level interpretation of model behavior. The objective of this stage was **not to train an additional predictive model or modify the fraud classification**, but to explain which transaction features contributed to the risk signal produced for an evaluated transaction.

A distinction is required between the final Stacking model and the transaction-level explanation mechanism. The production Logistic Regression meta-model receives only the three probability features `rf_prob`, `cat_prob`, and `xgb_prob`. Consequently, it can characterize how the probability outputs of Random Forest, CatBoost, and XGBoost are combined, but it cannot directly attribute the resulting decision to the original transaction features.

To provide feature-level explanations, FraudGuard therefore applies **SHAP (SHapley Additive exPlanations)** to the CatBoost base model [21]. CatBoost was selected as the primary transaction-level explainer because it is one of the three models retained in the production ensemble, exhibited strong predictive performance during model development, and provides a tree-based structure suitable for SHAP analysis. The implementation uses a **TreeExplainer** constructed from the saved production CatBoost model.

The resulting architecture therefore maintains two distinct levels of interpretation. At the Stacking level, the final decision remains based on the Logistic Regression combination of the three base-model probabilities. At the transaction level, CatBoost SHAP values are used to identify which processed transaction features contributed most strongly to the CatBoost fraud-risk output.

4.7.1 SHAP-Based Feature Attribution

For an individual transaction, SHAP assigns a contribution value to each processed feature relative to the model’s baseline output. Positive SHAP values indicate features that move the CatBoost output toward higher fraud risk, whereas negative values indicate features that move the model output in the opposite direction.

The transaction-level CatBoost prediction can be represented additively as:

$$f(x) = \phi_0 + \sum_{i=1}^M \phi_i \quad (1)$$

where $f(x)$ represents the CatBoost model output for transaction x , ϕ_0 represents the baseline or expected model output, ϕ_i represents the SHAP contribution associated with feature i , and M denotes the number of processed input features.

The implementation constructs a SHAP explanation for the processed transaction and associates each feature with its processed value, SHAP value, absolute SHAP magnitude, contribution direction, and a readable business-oriented description. Features are subsequently ranked according to the absolute magnitude of their SHAP values, allowing the strongest transaction-level contributors to be identified.

For the global SHAP analysis, explanations were computed on a sampled subset of the separate test dataset containing up to 1,000 fraudulent transactions and 2,000 legitimate transactions. The sample was constructed using the fixed project random state, and CatBoost SHAP values were computed for these observations using the production CatBoost TreeExplainer.

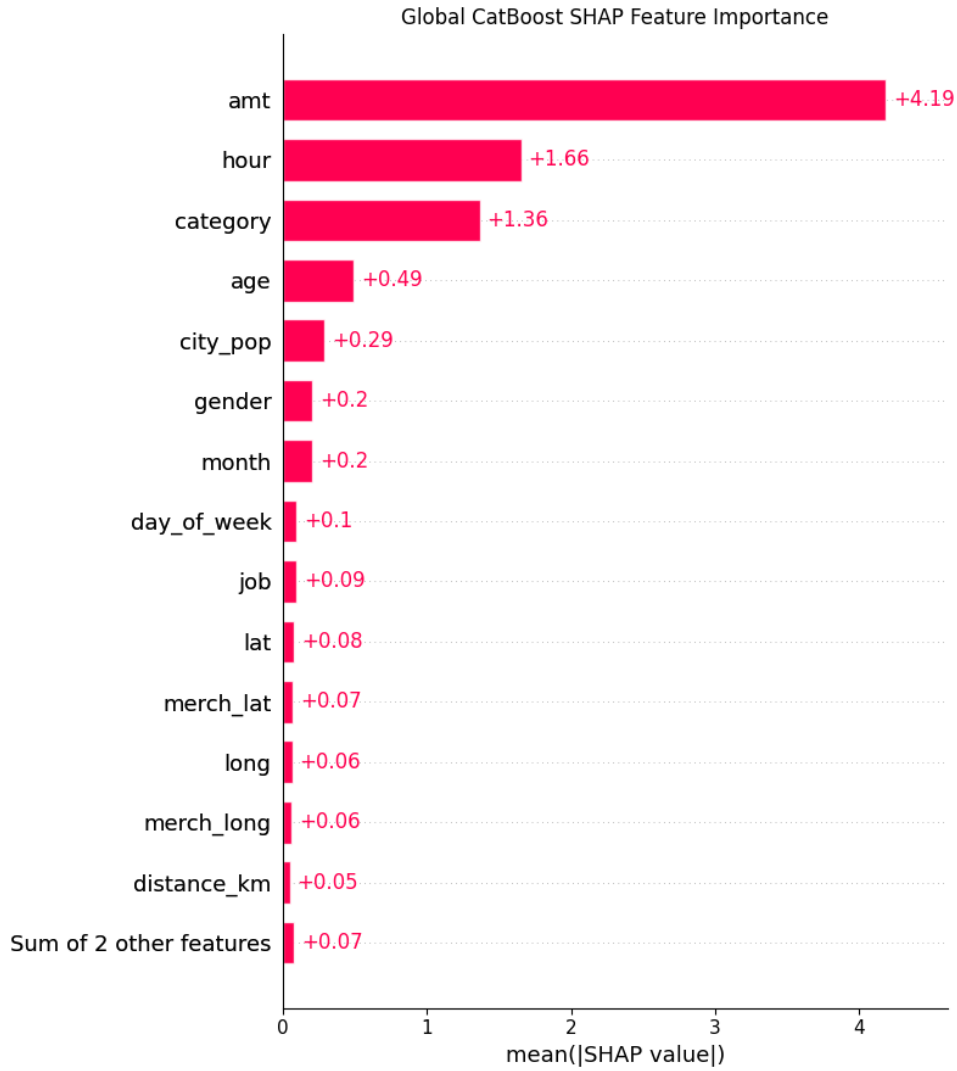


Figure 35: Distribution of CatBoost SHAP values across the sampled test transactions, showing the direction and magnitude of feature-level contributions to the model output.

The global SHAP analysis indicates that the CatBoost model relied most strongly on a relatively concentrated subset of transaction attributes. **Transaction amount (amt)** exhibited the largest mean absolute SHAP value (4.19), followed by **transaction hour (hour, 1.66)** and **transaction category (category, 1.36)**. Age showed a smaller but still visible contribution (0.49), while the remaining features exhibited substantially lower mean absolute SHAP magnitudes.

These values characterize the average magnitude of each feature’s contribution to the **Cat-Boost model output** across the analyzed observations. They should not be interpreted as causal relationships with fraud, nor as feature importance for the complete Stacking ensemble. Rather, they describe the relative influence of the original processed transaction features within the CatBoost component used for transaction-level explanation.

4.7.2 Distribution of SHAP Contributions

Global mean absolute SHAP values describe the overall magnitude of feature influence but do not indicate whether particular feature values tend to increase or decrease the CatBoost fraud-risk output. The distribution of SHAP values across transactions was therefore also examined.

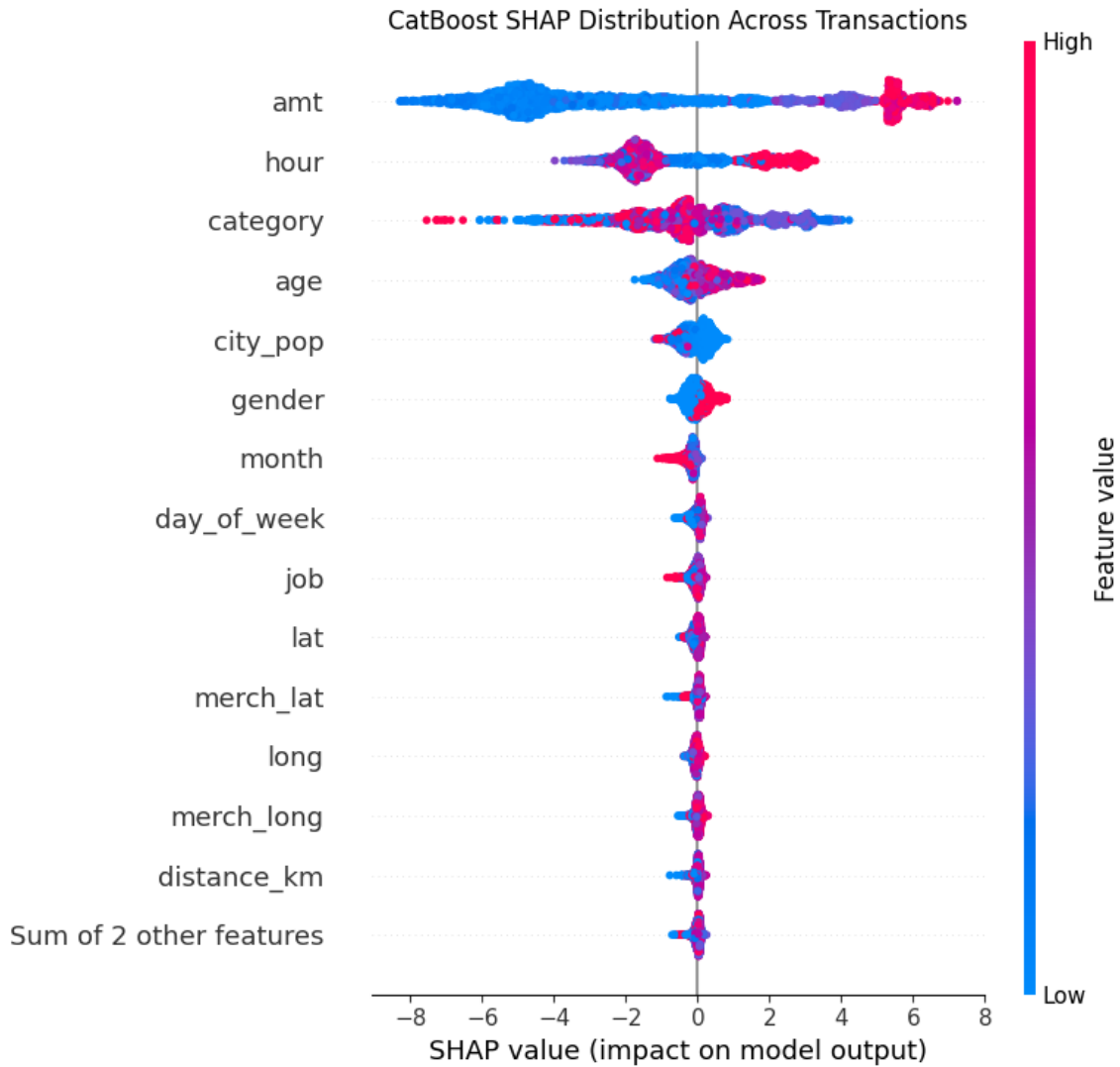


Figure 36: Distribution of CatBoost SHAP values across transactions, showing the direction and magnitude of feature-level contributions to the model output.

The SHAP distribution provides a more detailed view of model behavior by showing both the magnitude and direction of feature contributions across transactions. Each point represents a transaction-level feature contribution, with horizontal position indicating the corresponding SHAP value and feature value represented by the figure's color scale.

The broad distribution associated with `amt` is consistent with its dominant global importance, demonstrating that transaction amount can produce substantial changes in the CatBoost output. `hour` and `category` also exhibit comparatively wide SHAP distributions, confirming that their influence varies materially across transactions. In contrast,

several lower-ranked features remain concentrated close to zero, indicating more limited effects on the CatBoost output for most observations.

Importantly, the distribution demonstrates that feature influence is **transaction-dependent**. A globally important feature does not necessarily contribute in the same direction or with the same magnitude for every transaction. This motivates the use of local SHAP explanations when presenting the factors associated with a specific fraud-risk assessment.

4.7.3 Transaction-Level Explanation

The explainability layer was additionally evaluated on individual transactions to demonstrate how the global feature relationships translate into a specific model prediction. In the notebook demonstration, a known fraudulent transaction from the separate test dataset was passed through the complete production prediction and explanation pipeline.

For the demonstrated transaction, the three production base models generated fraud probabilities of approximately **0.9759 for Random Forest**, **0.9998 for CatBoost**, and **0.9999 for XGBoost**. These values formed the meta-feature vector supplied to the Logistic Regression meta-model, which produced a final Stacking fraud probability of approximately **0.9997**. Because this value exceeded the production decision threshold of **0.69**, the resulting operational decision was **BLOCK**.

The SHAP explanation was then generated independently at the CatBoost feature level to identify the transaction attributes that contributed most strongly to its fraud-risk output.

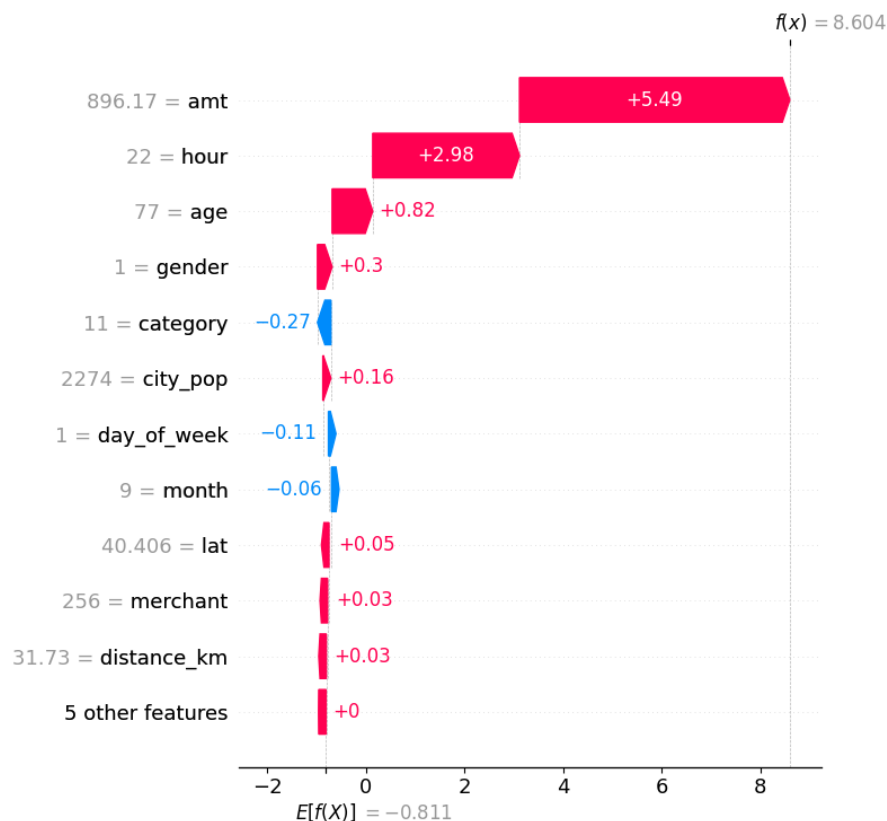


Figure 37: CatBoost SHAP waterfall explanation for the demonstrated blocked transaction, showing feature-level contributions to the base-model output.

For this transaction, the strongest positive CatBoost SHAP contribution was associated with the **transaction amount of 798.05**, with a SHAP value of approximately **+5.45**. The transaction hour of **22:00** contributed approximately **+2.84**, while age contributed approximately **+0.78**. Smaller positive contributions were associated with gender and city population. Conversely, category and month produced negative contributions of approximately -0.41 and -0.15 , respectively.

The waterfall representation illustrates how these positive and negative contributions move the CatBoost output away from its baseline value. In this case, the large positive contributions associated primarily with transaction amount and transaction hour dominated the smaller negative contributions, resulting in a substantially elevated CatBoost model output.

Crucially, this explanation describes **why the CatBoost component produced its transaction-level risk signal**. The final **BLOCK** decision was not generated directly from these SHAP values; it remained the result of the production Logistic Regression meta-model applied to the three base-model probabilities and compared against the threshold of **0.69**.

4.7.4 Explainability Output and System Integration

The implementation separates prediction, explanation, decision orchestration, and response formatting into distinct responsibilities. The prediction component performs pre-processing, obtains the Random Forest, CatBoost, and XGBoost probabilities, constructs the three-dimensional meta-feature vector, evaluates the Logistic Regression meta-model, and applies the production threshold. The explainability component subsequently uses CatBoost Tree SHAP to construct a feature-level interpretation of the processed transaction.

For internal analysis, the explanation layer ranks features according to absolute SHAP magnitude and records their contribution direction. The implementation extracts the strongest positive contributors as transaction-level risk factors and associates them with readable descriptions. This provides an analyst-oriented representation of the model behavior while retaining the underlying technical SHAP information.

The implementation additionally separates the **internal explanation from the customer-facing response**. Detailed information such as feature names, SHAP values, model internals, and the decision threshold is retained for internal interpretation and is not exposed in the customer message. For a blocked transaction, the customer-facing output instead communicates that the transaction requires verification without disclosing the internal fraud-detection logic.

This separation preserves the architectural distinction between **prediction and interpretation**: the explainability layer operates after the predictive result has been generated and does not alter the probability or decision produced by the final ensemble. SHAP therefore functions as an interpretation mechanism for model behavior rather than an additional decision-making component.

4.8 Chapter Summary

This chapter presented the systematic development and evaluation of the FraudGuard fraud-detection architecture, progressing from standalone candidate models to the final production ensemble. The comparative evaluation demonstrated distinct operating characteristics among the supervised classifiers, motivating the use of Out-of-Fold Stacking to combine their continuous fraud-probability outputs while maintaining separation between meta-model training and independent test evaluation.

Subsequent model-reduction experiments examined whether the initial five-model architecture could be simplified while preserving the required fraud-detection behavior. The resulting production configuration retained **Random Forest**, **CatBoost**, and **XGBoost**, combined through a **Logistic Regression meta-model**, with a production decision threshold of **0.69**. This configuration provided the selected balance between fraud Recall and False Positive control while reducing the number of base models required during inference.

Finally, an explainability layer based on **CatBoost SHAP values** was incorporated to provide feature-level interpretation of individual transaction risk signals without altering the ensemble’s predictive decision. Together, these stages established the final predictive and explainability components of FraudGuard for subsequent system-level integration and evaluation.

5 FraudGuard System Implementation

FraudGuard extends the predictive architecture developed in Chapter 4 into an integrated credit-card fraud-detection prototype. The system deploys the selected production Stacking Ensemble within a software environment capable of receiving transaction data, executing fraud inference, and presenting the resulting predictions through multiple application interfaces.

This chapter describes the system architecture, end-to-end transaction-processing flow, web-based monitoring environment, continuous transaction simulation, and external fraud-alert mechanism. Together, these components demonstrate the transition from offline model development to an operational fraud-monitoring prototype.

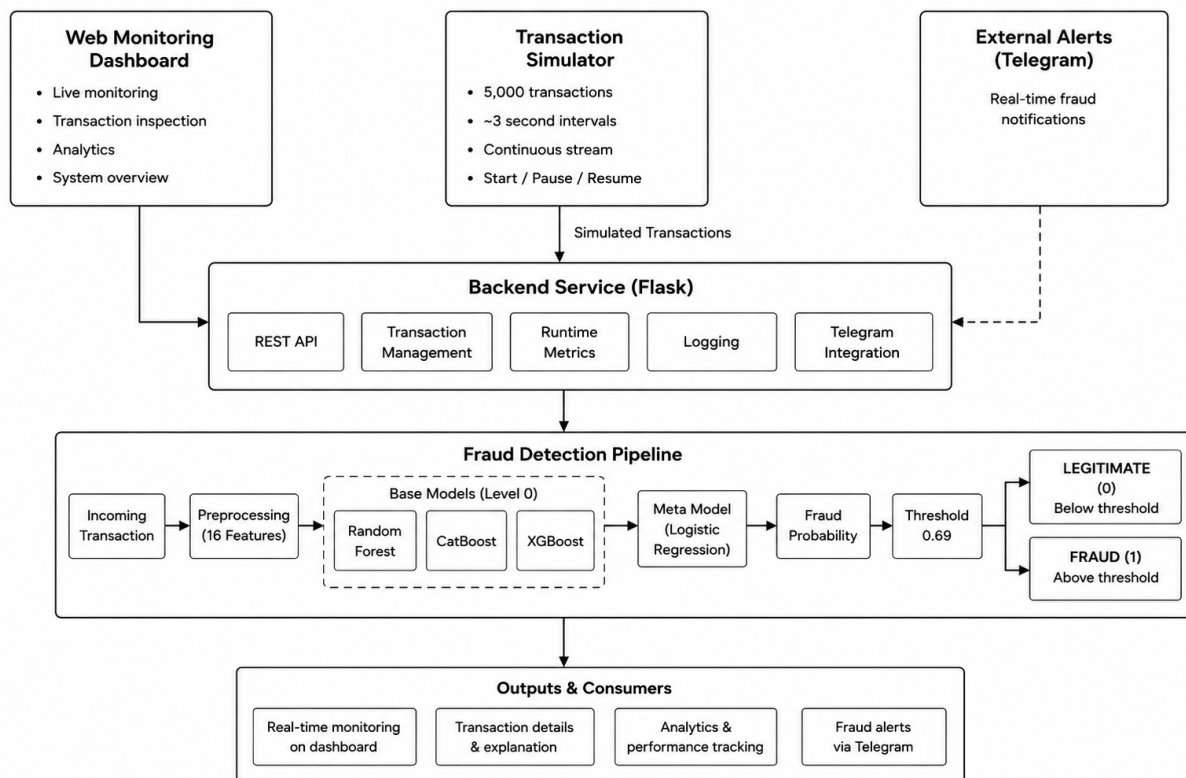


Figure 38: FraudGuard product overview, showing the web monitoring environment, backend fraud-detection service, and external notification channel.

5.1 System Architecture

FraudGuard follows a modular architecture that separates transaction acquisition, back-end processing, fraud inference, and application-level outputs. At a high level, the architecture can be summarized as:

Transaction Sources → Flask Backend → Fraud Detection Pipeline → Application Outputs

The **Flask backend** serves as the central integration layer, receiving transaction data and forwarding it to the deployed inference pipeline. The **React/Vite frontend** provides the primary monitoring environment for presenting predictions, transaction information, and system statistics. Transaction data is supplied through the continuous transaction simulator, allowing the deployed fraud-detection pipeline to be evaluated under a controlled stream of transactions.

Following inference, prediction results are exposed to the relevant application interfaces for monitoring and analysis. Transactions classified as fraudulent can additionally trigger external **Telegram notifications**.

5.2 End-to-End Transaction Processing Flow

When a transaction enters FraudGuard, the backend first prepares it using the saved preprocessing artifacts established during model development. The resulting feature representation is then passed to the deployed fraud-detection pipeline.

The processed feature vector is evaluated independently by the selected **Random Forest**, **CatBoost**, and **XGBoost** base models. Each base model produces a fraud-probability output, and these outputs are passed to the trained **Logistic Regression meta-model**, which produces the final **Stacking probability**. The production threshold of **0.69** is then applied to this probability to determine the final classification as legitimate or fraudulent.

The resulting prediction is returned to the application layer together with the relevant transaction and model information. The web interface can then display the result for monitoring and inspection, while a fraudulent classification can additionally activate the system's fraud-alert functionality.

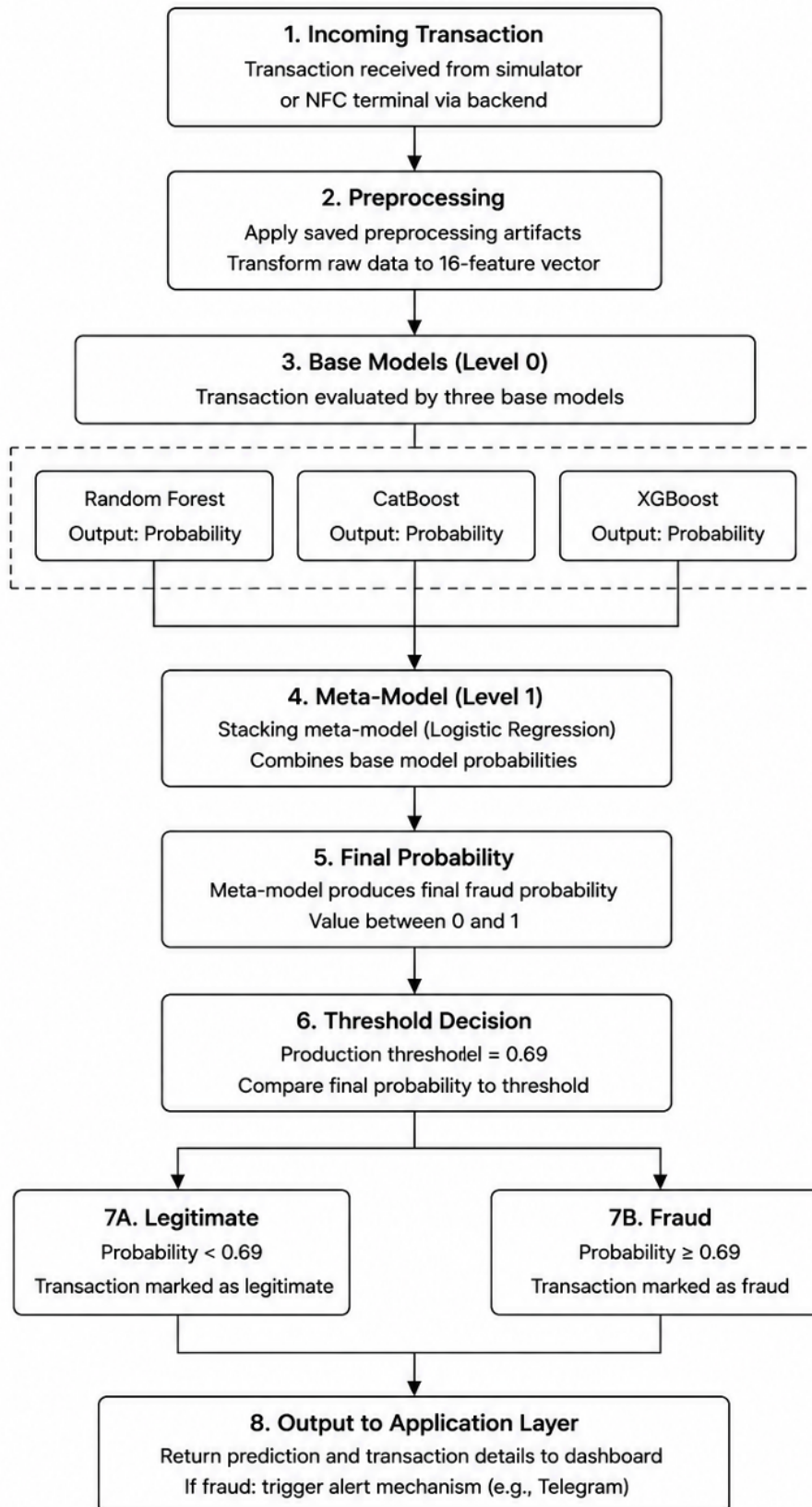


Figure 39: End-to-end FraudGuard transaction processing flow, from preprocessing and base-model inference through Stacking aggregation, threshold-based classification, and application-level output.

5.3 Web Monitoring Environment

FraudGuard includes a web-based monitoring environment implemented using **React** and **Vite** [22, 23]. The interface provides access to the outputs of the deployed fraud-detection system and allows users to monitor processed transactions, inspect individual predictions, and review system-level information.

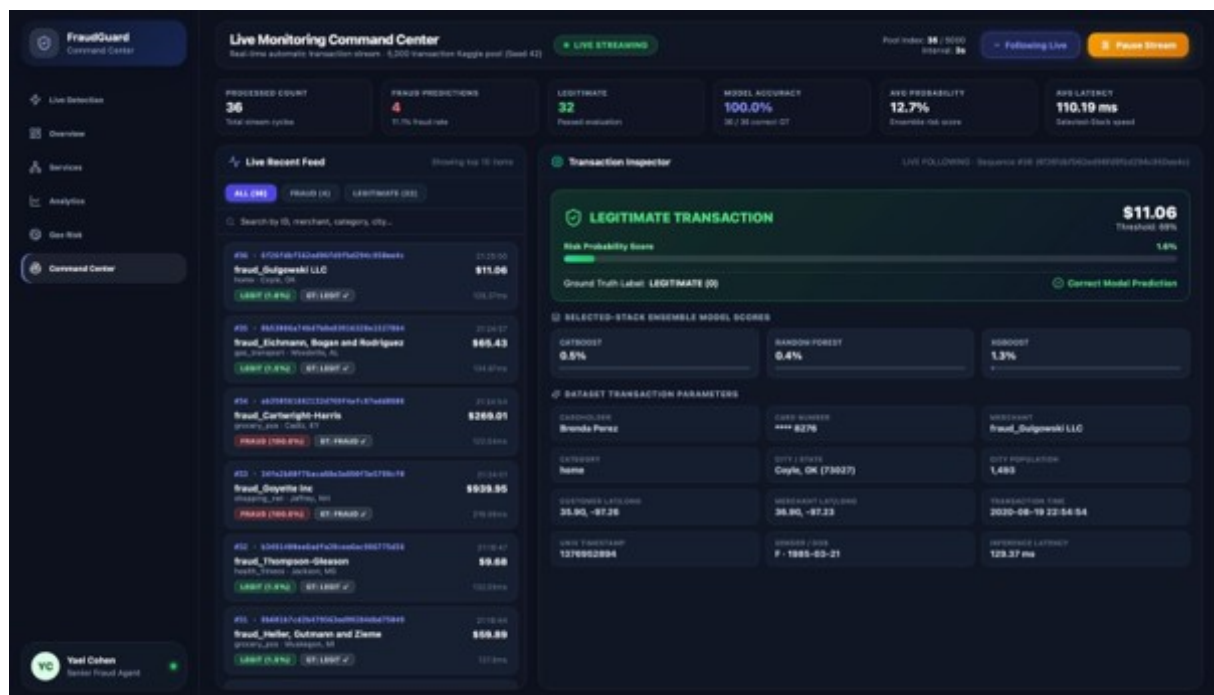
The application is divided into several views, including **Live Detection**, **Overview**, **Services**, **Analytics**, **Geo Risk**, and **Command Center**. Rather than performing model inference directly in the browser, the frontend communicates with the Flask backend and presents the transaction and prediction information returned by the system.

The **Command Center** serves as the primary real-time monitoring interface and is described separately in the following subsection.

5.3.1 Command Center

The Command Center provides a real-time view of transactions processed by FraudGuard. It combines a live transaction feed with aggregate monitoring statistics and a transaction inspector, allowing individual predictions to be examined as they are produced.

The interface also provides stream-level monitoring statistics, including the number of processed transactions, fraud and legitimate prediction counts, model accuracy against Ground Truth, average fraud-risk score, and average inference latency.

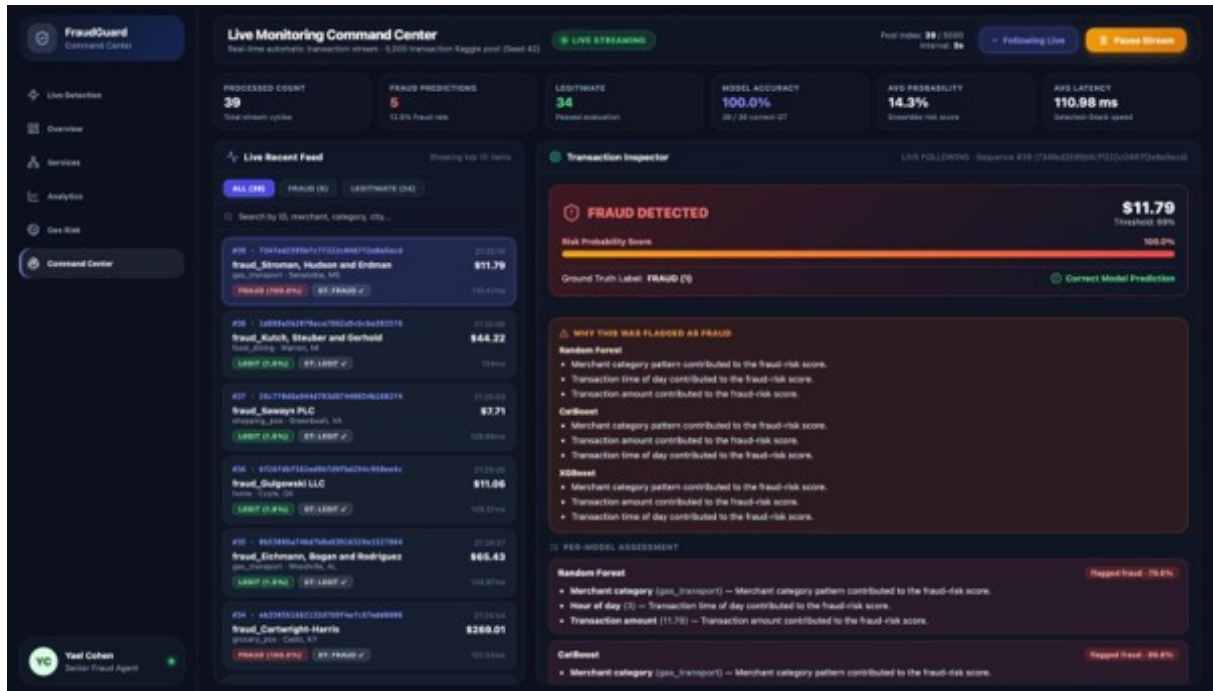


figureFraudGuard Command Center displaying a legitimate transaction.

For each selected transaction, the inspector presents the final fraud-risk score and classification, the production decision threshold, Ground Truth comparison, individual **Random**

Forest, CatBoost, and XGBoost probability scores, transaction parameters, and inference latency.

Transactions classified as fraudulent are presented with an expanded fraud-specific view. In addition to the standard transaction information, the Command Center displays a **“Why This Was Flagged as Fraud”** explanation and **per-model assessments**, providing model-specific information about the factors associated with the fraud prediction.

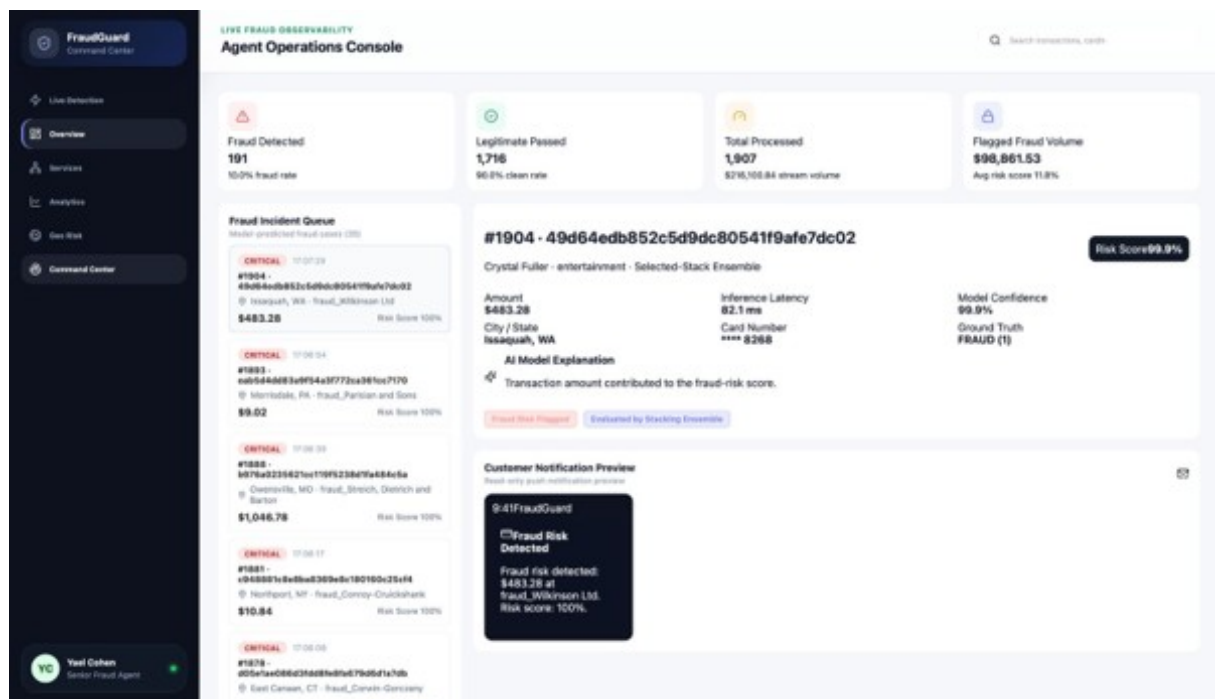


figureFraudGuard Command Center displaying a transaction classified as fraudulent, including fraud-specific explanatory information and per-model assessments.

5.3.2 Overview

The Overview provides a high-level operational summary of the FraudGuard system. It displays key statistics, including detected fraud, legitimate transactions, total processed transactions, and the monetary volume of transactions flagged as fraud.

A Fraud Incident Queue allows flagged transactions to be selected and inspected individually. For each incident, the interface presents the fraud-risk score, transaction details, model confidence, inference latency, and Ground Truth. The AI Model Explanation uses SHAP to highlight the features that contributed to the fraud prediction. The interface also includes a preview of the fraud notification that may be presented to the customer.



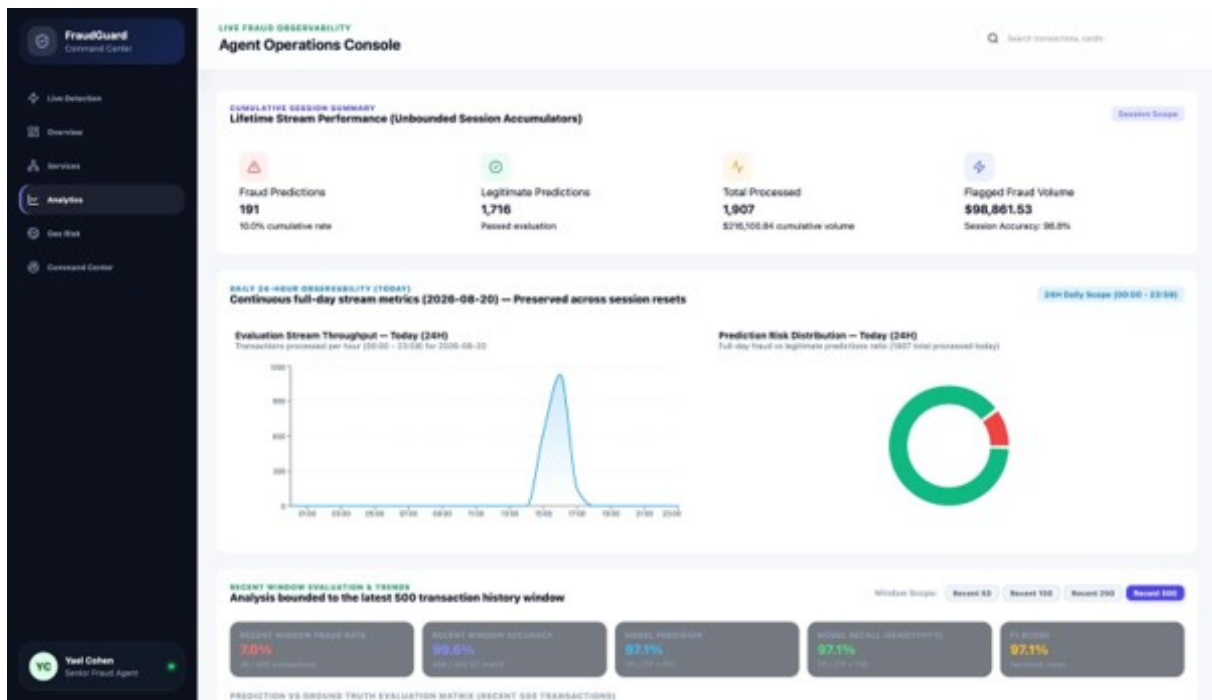
figureFraudGuard Overview displaying operational statistics, the fraud incident queue, SHAP-based model explanation, and customer notification preview.

5.3.3 Analytics

The **Analytics** view provides performance monitoring and evaluation of the FraudGuard prediction system across different time scopes. It combines cumulative session statistics, daily transaction activity, and recent-window model evaluation.

The upper section summarizes **lifetime session performance**, including fraud and legitimate predictions, total processed transactions, flagged fraud volume, and overall session accuracy. A separate **24-hour observability** section presents transaction throughput over time and the distribution between fraud and legitimate predictions for the current day.

The **recent-window evaluation** analyzes the latest **50, 100, 250, or 500 transactions** and reports key classification metrics, including **fraud rate, accuracy, precision, recall, and F1-score**, together with a comparison between model predictions and Ground Truth.



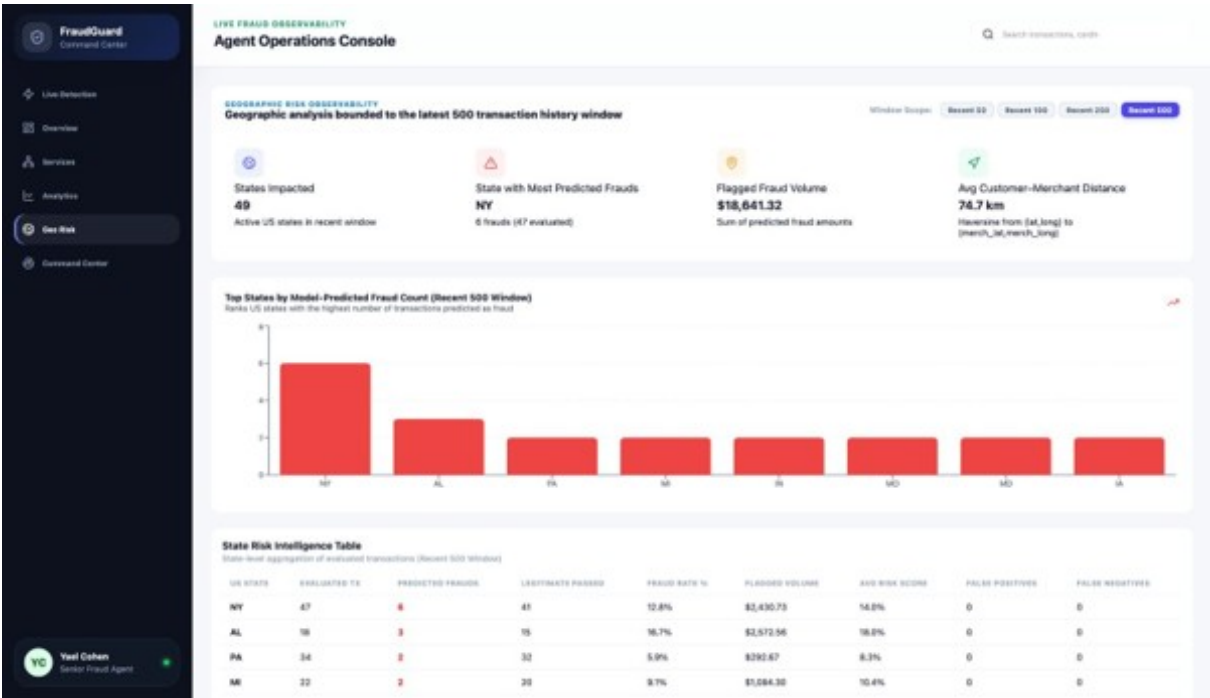
figureFraudGuard Analytics dashboard displaying cumulative performance, 24-hour transaction activity, prediction distribution, and recent-window model evaluation metrics.

5.3.4 Geo Risk

The **Geo Risk** view provides geographic analysis of FraudGuard predictions, allowing fraud activity and risk patterns to be examined across U.S. states. The analysis can be adjusted to the most recent **50, 100, 250, or 500 transactions**.

The dashboard summarizes the **number of states represented**, the **state with the highest number of predicted frauds**, total **flagged fraud volume**, and the **average customer-to-merchant distance**, calculated from the transaction coordinates using the Haversine distance.

A state-level visualization ranks locations according to the number of model-predicted fraud transactions. In addition, the **State Risk Intelligence Table** provides detailed statistics for each state, including evaluated transactions, predicted frauds, legitimate predictions, fraud rate, flagged transaction volume, average risk score, false positives, and false negatives.



figureFraudGuard Geo Risk dashboard displaying geographic fraud indicators, state-level fraud distribution, and risk statistics for the selected transaction window.

5.3.5 Services

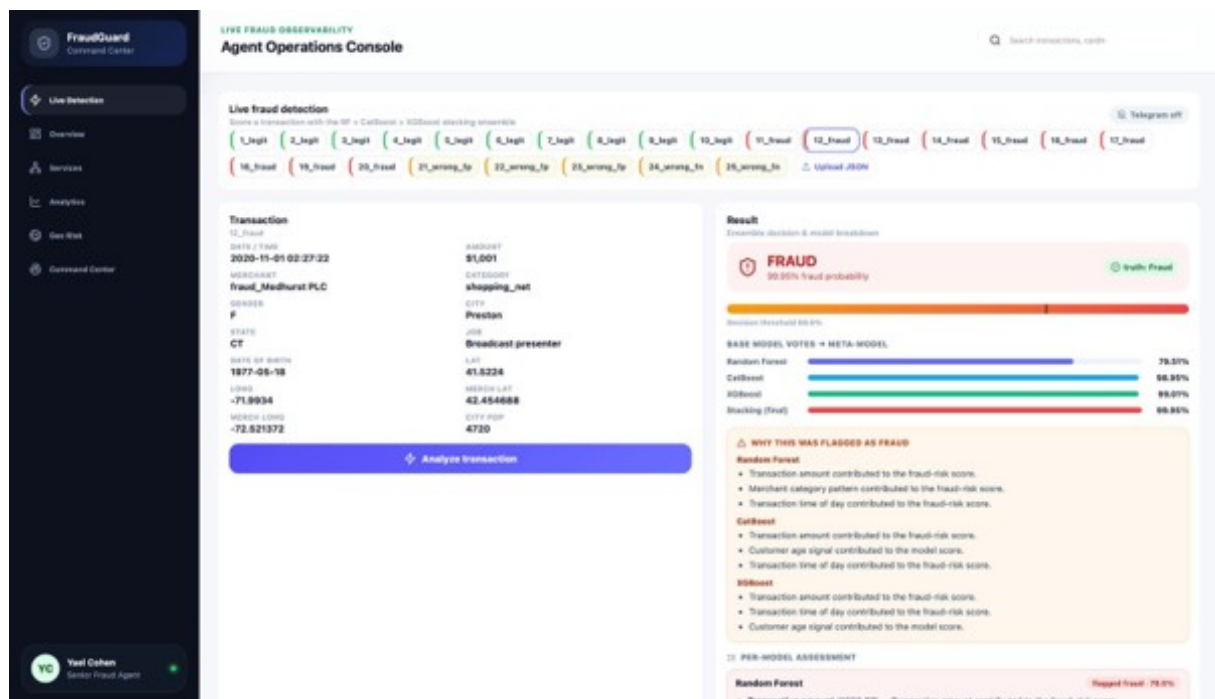
The **Services** view provides system-level information about the components supporting the FraudGuard application. It allows the deployed environment and its operational services to be reviewed from a single interface, complementing the transaction-focused monitoring and analytics views.

5.3.6 Live Detection

The **Live Detection** view provides an interactive interface for evaluating individual transactions using the deployed FraudGuard stacking ensemble. Users can select predefined transaction samples or upload a transaction in JSON format for evaluation.

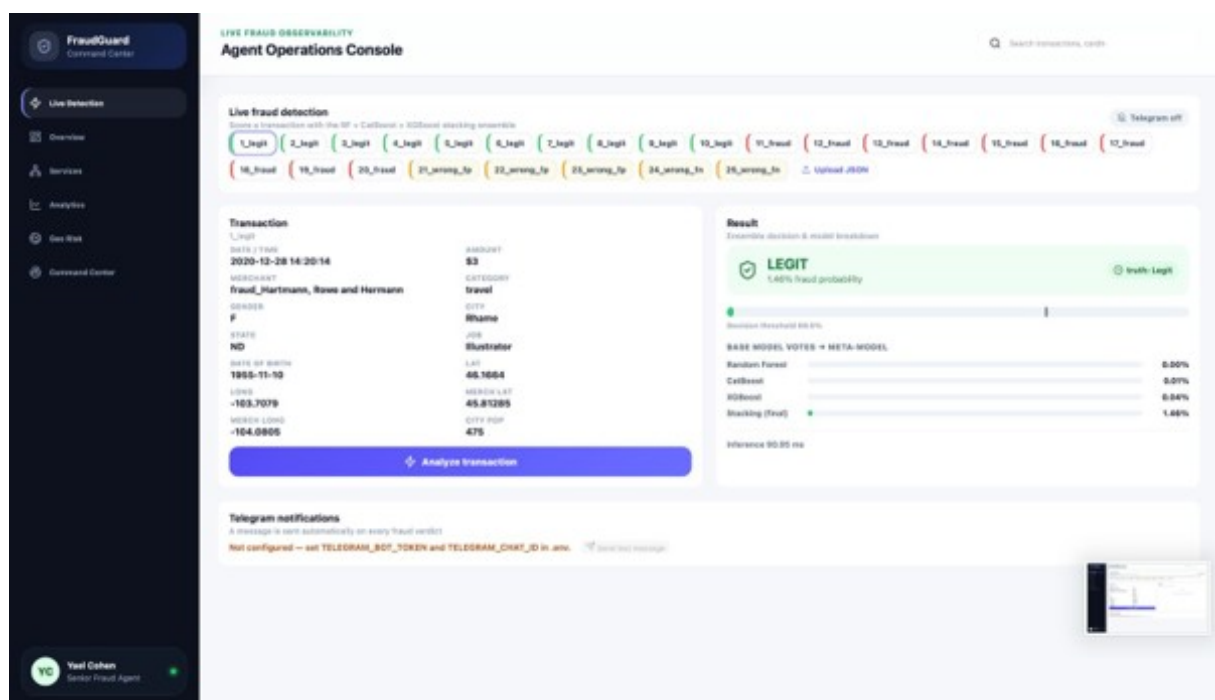
Once analyzed, the interface presents the final fraud probability and classification, the production decision threshold, the individual probability scores produced by Random Forest, CatBoost, and XGBoost, and the final stacking ensemble score. The prediction is also compared with the transaction’s Ground Truth label.

For transactions classified as fraudulent, the interface additionally provides SHAP-based explanations describing the features that contributed to the prediction for each base model, together with per-model assessments.



figureFraudGuard Live Detection interface displaying a fraudulent transaction, including the final stacking prediction, base-model probability scores, Ground Truth comparison, and SHAP-based explanations.

The interface also provides the same model-level information for transactions classified as legitimate, allowing the individual model outputs and final ensemble decision to be inspected.



figureFraudGuard Live Detection interface displaying a legitimate transaction, including the final stacking prediction, base-model probability scores, and Ground Truth comparison.

5.4 Continuous Transaction Simulation

To demonstrate FraudGuard under continuous transaction processing, the system includes a transaction simulator based on a predefined pool of **5,000 transactions**. The simulator submits transactions at approximately **three-second intervals**, creating a controlled transaction stream that can be observed through the monitoring environment.

Each simulated transaction is evaluated by the same production inference pipeline used by the application, and its resulting prediction becomes available to the Command Center. The simulator therefore acts only as a controlled transaction source and does not introduce an additional Machine Learning component.

5.5 Fraud Alerts and Telegram Integration

FraudGuard integrates **Telegram** [24] includes a web-based monitoring environment implemented using **React and Vite** [22, 23]. as an external notification channel for transactions classified as fraudulent. This provides an additional notification channel beyond the web-based monitoring environment by allowing fraud alerts to be delivered directly to a configured Telegram chat.

The notification summarizes the detected transaction and includes its fraud probability, transaction details, and the individual probability scores produced by the three base models. This provides the recipient with immediate information about the flagged transaction without requiring continuous observation of the Command Center.

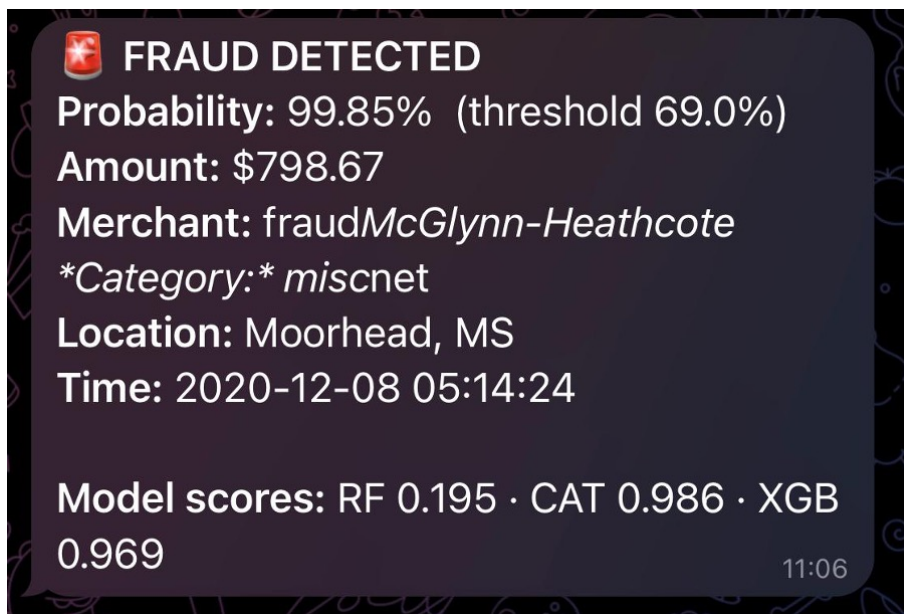


Figure 40: Telegram notification generated for a transaction classified as fraudulent by FraudGuard.

5.6 Chapter Summary

This chapter presented the implementation of FraudGuard as an integrated fraud-detection prototype built around the production Stacking Ensemble developed in Chapter 4. The system separates transaction acquisition, backend processing, predictive inference, and application-level interaction, allowing the deployed fraud-detection pipeline to support real-time transaction monitoring and analysis.

The resulting implementation demonstrates the transition from offline model development to an operational prototype in which transaction data can be processed, evaluated, and surfaced through user-facing monitoring and notification mechanisms.

6 Business Plan and “Shark Tank” Competition

Beyond its technical development and implementation, FraudGuard was also examined from a commercial perspective, including its target market, proposed business model, competitive value, and potential pathway toward deployment within financial institutions. The project was additionally presented in an academic “**Shark Tank**” competition, where it advanced to the **semi-final stage**.

6.1 “Shark Tank” Academic Competition

FraudGuard was presented in the academic “**Shark Tank**” competition, where it successfully advanced to the **semi-final stage**. The project was evaluated by a panel of industry experts, investors, and academic judges across areas including technical innovation, market feasibility, and financial sustainability.

The competition provided an opportunity to present FraudGuard not only as a technical fraud-detection system, but also as a potential commercial solution addressing the financial and operational impact of credit card fraud and false transaction declines.

6.2 Market Opportunity and Business Model

Fraud detection presents financial institutions with a dual challenge: identifying fraudulent transactions while minimizing false declines of legitimate transactions. FraudGuard’s proposed business model addresses this opportunity through a **performance-based Gain-Share approach**, linking the provider’s revenue to measurable financial value generated for the participating institution.

Under the proposed model, the company retains **40% of the net financial savings** generated through prevented fraud losses and preserved revenue from reduced false declines. This structure is intended to reduce upfront adoption risk and align the commercial interests of the technology provider with those of the financial institution.

The business plan defines a funding request of **\$2 million USD**, intended to provide approximately a **24-month operational runway** and support production deployment with a primary issuing bank.

6.3 Target Customers and Go-to-Market Strategy

The primary target customers include **commercial banks, credit card issuers, payment gateway providers, and FinTech institutions**, particularly organizations processing high volumes of cross-border transactions.

The proposed Go-to-Market strategy follows three stages:

1. **Performance-Based Pilot Deployment** – initial deployment based on the 40% Gain-Share model, allowing the participating institution to evaluate the system against measurable operational and financial outcomes.

2. **Production Integration** – integration of the FraudGuard fraud-detection pipeline into the participating institution’s transaction-processing environment.
3. **Data-Driven Expansion** – use of validated financial and operational results from initial deployments to support adoption by additional financial institutions and payment processors.

6.4 Competitive Advantage

FraudGuard’s proposed competitive value combines the technical capabilities developed throughout the project with a performance-based commercial model. The production Stacking architecture combines **Random Forest, CatBoost, and XGBoost through a Logistic Regression meta-model**, while the selected operating threshold prioritizes the reduction of False Positives under a minimum **Recall requirement of 0.95**.

The system additionally incorporates **SHAP-based explainability** to provide feature-level insight into the CatBoost component’s predictions. From a business perspective, these capabilities support the objective of reducing unnecessary transaction blocking while maintaining strong fraud detection.

According to the project’s verified business analysis, the proposed solution has the potential to achieve **up to a 50% reduction in credit card fraud losses** while minimizing disruption to legitimate users.

7 Discussion, Conclusions, Limitations and Future Work

7.1 Discussion

The development of FraudGuard demonstrated that effective fraud-detection model selection cannot be based on a single aggregate performance metric. During the preliminary experimental stage, Accuracy initially appeared informative; however, the substantial class imbalance showed that high Accuracy could coexist with weak fraud-class performance. The evaluation methodology therefore progressed toward Balanced Accuracy and subsequently toward Precision, Recall, F1-Score, and direct analysis of the Confusion Matrix. This shifted the evaluation focus toward the practical trade-off between detecting fraudulent transactions and incorrectly flagging legitimate activity.

The systematic experiments further demonstrated that the candidate models exhibited different predictive characteristics. Random Forest produced a comparatively high-Precision, low-False-Positive signal, whereas CatBoost and XGBoost provided stronger fraud Recall. These differences motivated the transition from standalone model selection toward Stacking based on complementary model probability outputs.

The initial Stacking architecture incorporated five supervised base models using leakage-aware Out-of-Fold predictions [?]. Evaluation of all **31 non-empty combinations** of the five meta-features subsequently demonstrated that increasing ensemble size did not

necessarily improve operational performance. Under the requirement of fraud Recall of at least **0.95**, while prioritizing False Positive reduction, the combination of **Random Forest, CatBoost, and XGBoost** preserved the same True Positive and False Negative counts as the five-model configuration while reducing False Positives by **233**.

This resulted in the final three-model Stacking architecture, with **Logistic Regression as the meta-model and a decision threshold of 0.69**. The result demonstrates that effective ensemble design depends on selecting complementary predictive signals rather than simply maximizing the number of models.

7.2 Conclusions

FraudGuard successfully progressed from preliminary experimentation and systematic model evaluation to a complete end-to-end fraud-detection prototype. Beyond the final predictive architecture, the project demonstrated the importance of fraud-specific evaluation, leakage-aware ensemble development, systematic model-combination analysis, and operating-threshold selection.

The selected architecture was integrated with **SHAP-based explainability, real-time transaction processing and monitoring, transaction simulation, and Telegram fraud notifications**, extending the Machine Learning research into a functional system prototype.

7.3 Limitations

Several limitations remain in the current FraudGuard framework. Despite maintaining fraud Recall above **0.95**, the selected operating point still produces **4,794 False Positives**, demonstrating that unnecessary fraud alerts remain an important challenge.

The models were developed and evaluated using the project dataset rather than transaction data from a deployed financial institution. Therefore, the reported performance cannot establish equivalent generalization across different customer populations, transaction distributions, or previously unseen fraud patterns.

The current SHAP layer explains original-feature contributions through the **CatBoost component**, rather than directly explaining the complete Stacking Ensemble at the original-feature level. Furthermore, although FraudGuard was integrated into a functional real-time monitoring and fraud-detection environment, it remains an **academic prototype** and has not been evaluated using real financial-institution infrastructure or under production-scale transaction workloads.

7.4 Future Work

Future development should focus on further reducing **False Positives** while preserving high fraud Recall. One direction is the incorporation of richer transaction and customer-behavior information, including historical spending patterns, typical locations, merchant preferences, transaction frequency, and deviations from established cardholder behavior.

The use of **real-world longitudinal transaction data** could enable FraudGuard to move toward **personalized behavioral modeling**. With sufficient transaction history, the system could learn an individual cardholder’s normal behavior and evaluate new transactions relative to that personal baseline. A future hybrid architecture could therefore combine global fraud patterns learned across customers with personalized risk assessment for individual cardholders.

Further work should evaluate FraudGuard using **real financial-institution data and production-scale conditions**. This would allow the system to be examined as transaction behavior and fraud patterns evolve over time, while also supporting periodic model evaluation and retraining under changing data distributions.

The system could also be extended from fraud detection and notification toward **real-time transaction intervention**, including temporarily holding or blocking high-risk transactions and requesting additional verification. This would allow FraudGuard to progress from a monitoring and decision-support system toward a more active fraud-prevention framework.

As an additional future extension, FraudGuard could be evaluated within an **NFC-based payment environment**. Transaction data generated as part of a contactless payment flow could be forwarded to the FraudGuard backend for real-time fraud assessment. This would enable the existing detection pipeline to be demonstrated within a more realistic end-to-end payment scenario, while NFC itself would serve as the payment interface rather than as an additional fraud-detection feature.

Finally, future development could extend the current **CatBoost-based SHAP analysis** toward interpretation of the complete **Stacking architecture**. The current implementation uses SHAP to explain the CatBoost base model, whereas the final fraud probability is produced by the Stacking meta-model from the predictions of the base models. Future work could therefore investigate methods for explaining both the contribution of the original transaction features and the contribution of each base model to the final Stacking decision, providing greater transparency into the complete decision-making process. ““

Project Repository and Supplementary Materials

The complete FraudGuard implementation, experimental notebooks, trained artifacts, and supplementary project materials are available through the following resources.

- **GitHub Repository:** FraudGuard – Credit Card Fraud Detection
- **Google Drive Folder:** FraudGuard Project Files
- **Notebook 1 – Research and Data Preparation:** Open in Google Colab
- **Notebook 2 – Out-of-Fold Stacking Ensemble:** Open in Google Colab
- **Notebook 3 – Inference:** Open in Google Colab
- **Notebook 4 – Meta-Feature Combination Analysis:** Open in Google Colab
- **Notebook 5 – Selected Models Inference:** Open in Google Colab

- **Notebook 6 – SHAP Explainability Layer:** Open in Google Colab

References

- [1] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets,” *PLOS ONE*, vol. 10, no. 3, 2015.
- [2] The Nilson Report, “Global card fraud losses at \$33 billion,” January 2026, payment card fraud losses worldwide totaled \$33.41 billion in 2024 and are projected to reach \$41.06 billion by 2030.
- [3] D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [4] A. I. Naimi and L. B. Balzer, “Stacked generalization: An introduction to super learning,” *European Journal of Epidemiology*, vol. 33, no. 5, pp. 459–464, 2018.
- [5] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [6] Feedzai, “Transaction fraud,” <https://www.feedzai.com/transaction-fraud/>, accessed: 2026-08-20.
- [7] Featurespace, “The featurespace platform,” <https://www.featurespace.com/solutions>, accessed: 2026-08-20.
- [8] SAS Institute Inc., “Sas fraud management,” https://www.sas.com/en_us/software/fraud-management.html, accessed: 2026-08-20.
- [9] K. Shenoy, “Credit card transactions fraud detection dataset,” Kaggle, accessed: 2026.
- [10] C. C. Robusto, “The cosine-haversine formula,” *The American Mathematical Monthly*, vol. 64, no. 1, pp. 38–40, 1957.
- [11] Scikit-learn Developers, “Ordinalencoder,” Scikit-learn Documentation, 2026, accessed: 2026. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OrdinalEncoder.html>
- [12] K. H. Brodersen, C. S. Ong, K. E. Stephan, and J. M. Buhmann, “The balanced accuracy and its posterior distribution,” in *2010 20th International Conference on Pattern Recognition*. IEEE, 2010, pp. 3121–3124.
- [13] C. Guo and F. Berkhahn, “Entity embeddings of categorical variables,” *arXiv preprint arXiv:1604.06737*, 2016.
- [14] M. Sakurada and T. Yairi, “Anomaly detection using autoencoders with nonlinear dimensionality reduction,” in *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*. ACM, 2014, pp. 4–11.

- [15] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Advances in Neural Information Processing Systems*, vol. 27, 2014.
- [16] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.
- [17] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.
- [18] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “Lightgbm: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [19] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “Catboost: Unbiased boosting with categorical features,” in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [20] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *2008 Eighth IEEE International Conference on Data Mining*. IEEE, 2008, pp. 413–422.
- [21] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [22] Meta Platforms, Inc., “React documentation,” <https://react.dev/>, accessed: 2026.
- [23] Vite, “Vite documentation,” <https://vite.dev/>, accessed: 2026.
- [24] Telegram, “Telegram bot api,” <https://core.telegram.org/bots/api>, accessed: 2026.